

Como muitos aplicativos foram escritos durante a década de 80 em máquinas que possuíam apenas o modo real, o DOS 5.0²⁰ o sistema continuou funcionando dessa forma e, como resultado, suas rotinas e chamadas de sistema tornaram-se incompatíveis com o modo protegido. Isso criou uma situação incômoda, na qual o sistema operacional padrão fornecido com todas as máquinas vendidas impedia os programadores de usar a máquina em sua capacidade máxima.

potência total I. Os desenvolvedores foram forçados a ignorar todos os recursos de um **1992 CPU** e, em vez disso, use-o como um muito rápido **CPU Intel 8086** de . **1976** Elavavam, portanto, limitados às seguintes características:

- ALU
- 16 registros:
 - Registros de uso geral de 16 bits: AX, BX, CX, DX
 - Registros de índice de 16 bits: SI, DI, BP, SP
 - Contador de programa de 16 bits: IP
 - Registros de segmento de 16 bits: CS, DS, ES, FS, GS, SS
 - Registro de status de 16 bits
- Até 1 MiB de RAM

ate-
Curiosidades: Um ano antes, em **1991**, um estudante do **Universidade de Helsinque** Comecei a trabalhar sobre um hobby ("nada demais") dele: um sistema operacional que, ao contrário do **FAZER** S conseguiu uso do **CPU em modo protegido** e aproveite o **MMU** e o **Registradores de 32 bits**. Isso se tornaria o pior da Microsoft. **pesadelo** e. **Linus Torvalds** tinha acabado de começar o que seria tornar-se **Linux**

2.2.2 O infame Modo Real: limite de 1 MiB de RAM

largura
Com modo protegido **indisponível** e, **1991** desenvolvedores programaram como se fosse **1976**: com um O barramento de endereços de 20 bits oferecia apenas 1 MiB de RAM endereçável. Independentemente da quantidade de memória instalada na máquina, apenas 1 MiB podia ser endereçado. Para piorar a situação, o endereçamento tinha que ser feito não com os registradores de 32 bits disponíveis, mas combinando dois registradores de 16 bits. Um representava o segmento, o outro um deslocamento dentro desse segmento. Daí o nome: 'programação segmentada de 16 bits'.

A disposição da memória é a seguinte:

- De 00000h a 003FFh: a **tabela de vetores de interrupção**.
- De 00400h a 004FFh: Dados da BIOS.
- De 00500h a 005FFh: **command.com+io.sys**.

²⁰Lançado em junho de 1991.

- De 00600h a 9FFFFh: Utilizável por um programa (cerca de 620 KiB no melhor caso).
- De A0000h a FFFFFh: UMA (Área de Memória Superior): Reservada para a ROM da BIOS, E/S mapeada da placa de vídeo e da placa de som.

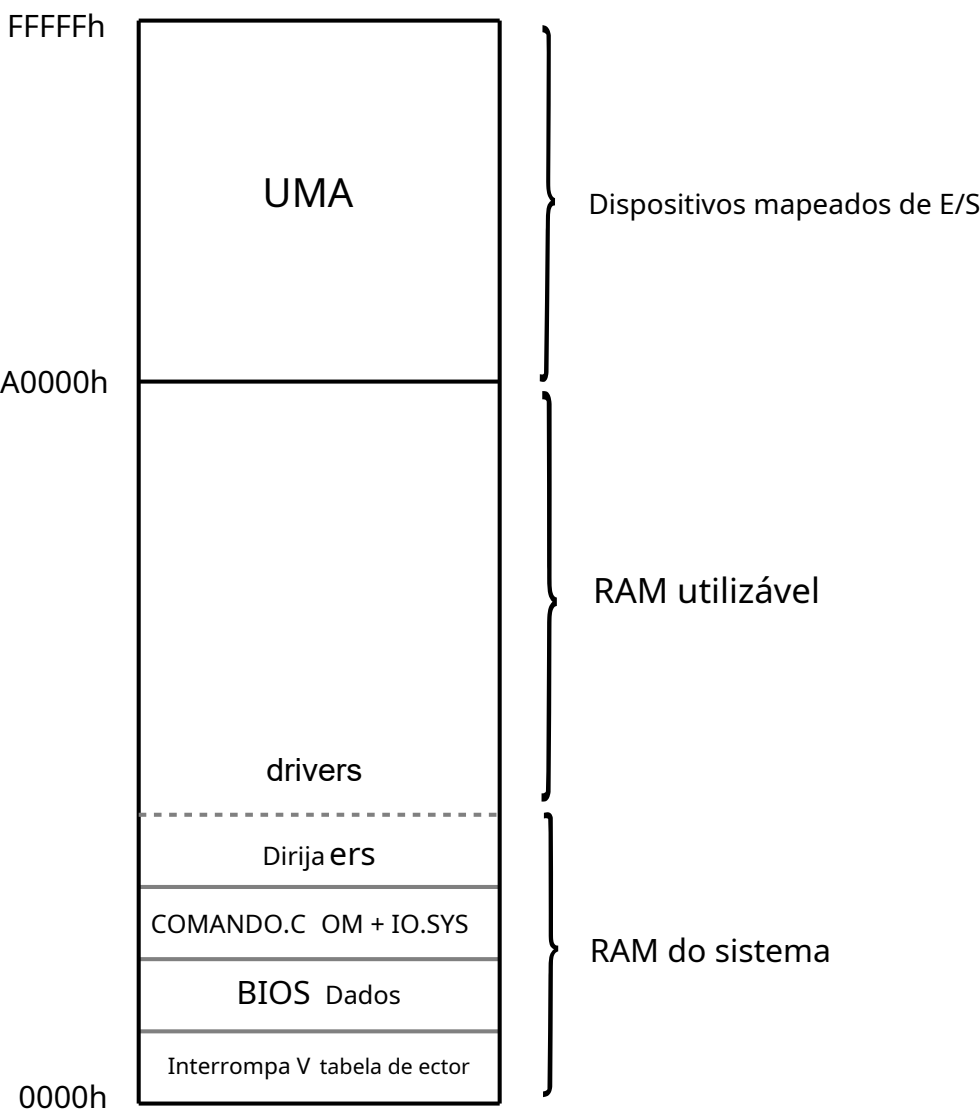


Figura 2.15: Layout dos primeiros 1 MiB de RAM.

Dos 1024 KiB originais, apenas 640 KiB (chamados de **Memória Convencional**) eram acessíveis a um programa. 384 KiB eram reservados para a UMA e para cada driver instalado. O **SISTEMA** (e .COM) retirou espaço dos 640 KiB restantes.

Curiosidades: Na França, as pessoas precisavam instalar o driver KEYBFR.SYS para que as teclas do teclado AZERTY fossem mapeadas corretamente. O driver consumia impressionantes 5 KiB de memória convencional. Desnecessário dizer que os franceses aprenderam rapidinho que o modo Deus era o que eles chamavam de **IDDAD**.²¹

2.2.3 O infame Modo Real: Endereçamento segmentado de 16 bits

Com um **barramento de endereços** de 20 bits e registradores muito pequenos para conter um endereço completo (16 bits de largura), a Intel teve que criar um **sistema de endereçamento**. Sua solução foi combinar dois registradores de 16 bits, um designando um segmento e o outro um deslocamento dentro desse segmento.

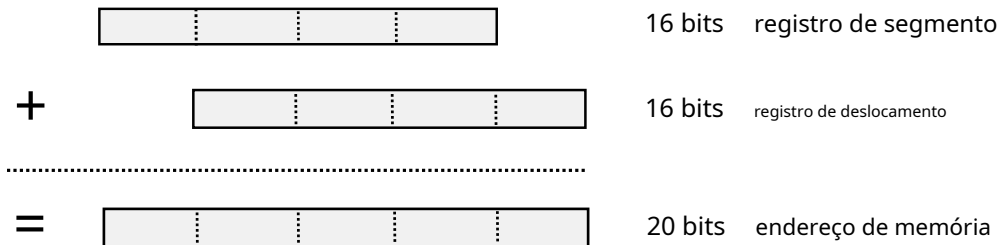


Figura 2.16: Como os registradores são combinados para endereçar a memória.

Existem dois tipos de indicadores: **aproximar** e **distante**. O **ponteiro** tem 16 bits e é considerado **rápido** porque pode ser usado tal como está (mas só permite um salto no segmento de código atual). O **distante** pode acessar qualquer coisa e permite um salto em qualquer lugar, mas é mais lento, pois um registrador de segmento de 16 bits precisa ser deslocado 4 bits para a esquerda e combinado com o outro registrador de deslocamento de 16 bits para formar um endereço de 20 bits.

Isso pode não parecer tão ruim, mas na prática esse endereçamento segmentado leva a muitos problemas. O menos problemático diz respeito à linguagem. Como o C foi inventado em uma máquina com memória plana, ele precisou ser aprimorado pelos fabricantes de compiladores para PC. É assim que... **aproximar** e **distante** surgiram as palavras-chave. **MacroMK_FP** construíram-nas **eFP_SEG/FP_OFF** acessou componentes individuais. **libct** também é "diferente": **malloc** retorna um **aproximar** ponteiro e, portanto, só pode alocar até **64 KiB**. Para obter mais de 64 KiB, **farmalloc** é necessário.

A questão mais importante é que dois indicadores se referem a **O mesmo endereço** pode falhar em um teste de igualdade. Neste modelo, **1 MiB** de RAM é dividido em **65536 partes pelo ponteiro de segmento**.

²¹ O modo de invencibilidade em Doom é **IDDQD** em um teclado QWERTY, mas **IDDAD** em um teclado AZERTY (sem o "d" francês). rio carregado.

Um parágrafo tem 16 bytes, mas um deslocamento pode chegar a 65536 bytes, o que resulta em muitas sobreposições. Isso pode ser explicado com os exemplos a seguir.

Ponteiro A definido como:

0000 0000 0000 0000	Segmento	16 bits
+ 0000 0001 0010 0000	Desvio	16 bits
=====		
0000 0000 0001 0010 0000	Endereço	20 bits

Ponteiro B definido como:

0000 0000 0001 0000	Segmento	16 bits
+ 0000 0000 0010 0000	Desvio	16 bits
=====		
0000 0000 0001 0010 0000	Endereço	20 bits

Ponteiro C definido como:

0000 0000 0001 0010	Segmento	16 bits
+ 0000 0000 0000 0000	Desvio	16 bits
=====		
0000 0000 0001 0010 0000	Endereço	20 bits

Conforme definido, A, B e C apontam para o mesmo local de memória, porém falharão em um teste de comparação.

```
# incluir <stdio.h>
# incluir <dos.h>

inteiroprincipal(inteiroargc,personagem** argv){

    vaziolong *a = MK_FP (0x0000 , 0x0120); vaziofar
    *b = MK_FP (0x0010 , 0x0020); vaziofar *c = MK_FP
    (0x0012 , 0x0000);

    printf("%d\n",a==b);
    printf("%d\n",a==c);
    printf("%d\n",b==c);
}
```

O resultado será:

```
0
0
0
```

Nesse sistema, a aritmética de ponteiros também deve receber atenção especial. O incremento de ponteiro incrementa apenas o deslocamento, não o segmento. Se você iterar sobre um array maior que 64 KiB, acabará dando a volta completa. Você poderia usar outro tipo de ponteiro, inteiro enorme*Para fazer a aritmética de ponteiros funcionar além de 64 KiB, mas na realidade, ninguém quer chegar a esse ponto.

2.2.4 Memória Estendida

O barramento de endereços de 20 bits do modo real limita a RAM endereçável a 1 MiB. Os computadores de 1992 vinham equipados com mais memória, tipicamente 2 MiB e até mesmo 4 MiB para os clientes mais afortunados. Essa memória localizada além do espaço endereçável é chamada de "Memória Estendida". A solução alternativa na época para acessar esses recursos era instalar drivers especializados.²²

Infelizmente, o acesso à memória estendida não foi padronizado. Os usuários podiam carregar qualquer um dos drivers fornecidos com o DOS:

- Drivers de especificação de memória expandida (EMS): EMM386.EXE.
- Drivers de especificação de memória estendida (XMS): HIMEM.SYS.

Ou talvez não tivessem ideia de que precisavam instalar um driver, não carregar nada na inicialização e não usar mais de 1 MiB de RAM. Esse caso de uso era um grande problema. Muitos clientes não conseguiam entender o porquê, mesmo após a instalação (extremamente cara).²³ megabytes de RAM em sua máquina, o jogo que acabaram de comprar se recusava a iniciar, alegando "Memória insuficiente". A id Software teve que publicar uma explicação (veja o Apêndice "A Barreira dos 640 KB") junto com o jogo.

As duas APIs tinham funcionalidades semelhantes, mas arquiteturas completamente diferentes.

2.2.4.1 API XMS

O driver XMS funciona²⁴ como `malloc/freed` e é a mais intuitiva para um programador. Ela permite a manipulação de dados na memória estendida não endereçável por meio de operações como `alocar`, `liberar`, `realocar`, `remover`. O aspecto fundamental do XMS é que a memória precisa ser copiada entre a RAM estendida e a RAM convencional.

²²Consulte o arquivo `CONFIG.SYS` nos Apêndices.

²³Em 1992, 4 MiB de RAM custavam US\$ 149, o que, ajustado pela inflação, equivaleria a US\$ 256 em 2017.

²⁴Especificação de Memória Estendida (XMS) 19 de julho de 1988.

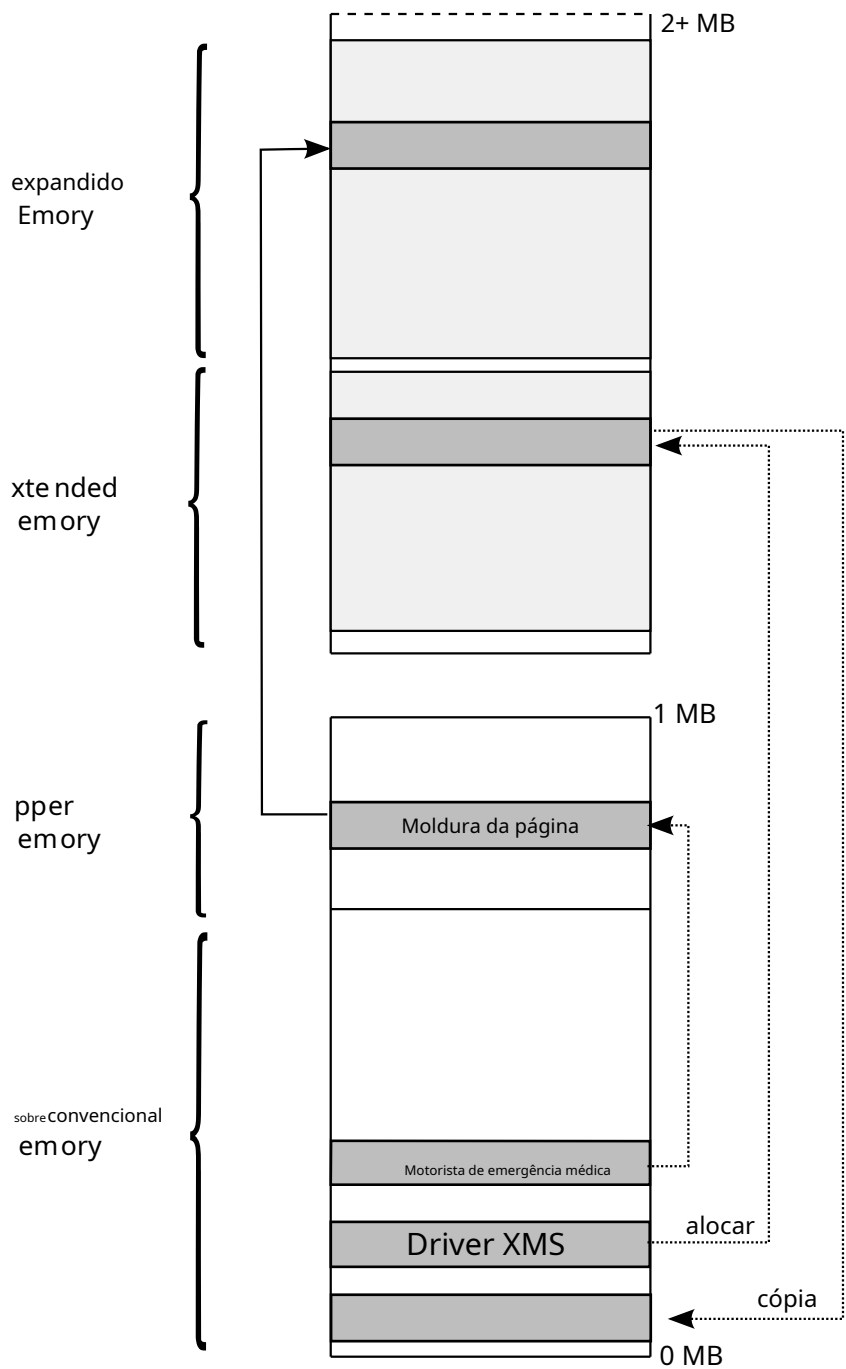


Figura 2.17:Exp uen Layout de memória estendida/ed

2.2.4.2 API EMS

O driver EMS abre uma janela além da RAM endereçável, como mostrado na página 45. A ideia gira em torno do **mapeamento de memória**. O driver permite manipular quatro unidades de 16 KiB chamadas "páginas" através de uma área de 64 KiB chamada "Quadro de Página". Mediante solicitação ao driver, uma página pode ser trocada para o quadro de página sem copiar nada.

2.2.4.3 EMS vs XMS

Como os **drivers** conseguiram o impossível (afinal, como acessar a **RAM** além de 1 MiB com um **ponteiro de 20 bits?**) e quais foram as **vantagens** e **desvantagens** de cada abordagem é um tópico fascinante explicado em detalhes no **Apêndice B**, na página 297.

Se preferir não entrar em muitos detalhes, basta lembrar que a abordagem de **mapeamento EMS** era várias vezes mais **rápida** do que a abordagem de **cópia XMS**. Essa questão de **velocidade** impactou consideravelmente o **gerenciamento de memória** do Wolfenstein 3D.

2.2.4.4 Um sistema "impossível de amar"

Neste ponto, se você está confuso com a **CPU** e seu **design**, saiba que não está sozinho. Ao longo dos anos, encontrei muitas maneiras de descrever essa loucura, mas três se destacam particularmente.

"

A arquitetura **x86** é difícil de **explicar** e impossível de **amar**.

"

David Patterson e John Hennessy - Organização e Projeto de Computadores.

"

Isso pode parecer estranho, mas a **Intel** o construiu, a **Microsoft** o escreveu e o **DOS** cresceu em torno dele.

Gatilho Eccles-Jordan - Codeproject.com.

"

"

Veneno de software.

Steve Morris - Co-arquiteto do Intel 8086.

"

Curiosidades: 640 KiB era o **máximo** que um jogo podia ter para **código executável**. Mas os **programadores de compiladores** foram espertos. **Strike Commander** (um famoso jogo de arcade de voo lançado em 1993) exe-

O espaço disponível para recorte é de 745 KiB, o que obviamente não cabe na memória convencional. O truque é usar uma técnica de páginas de "sobreposição", onde apenas algumas sobreposições são carregadas quando o jogo inicia. Quando a CPU está prestes a atingir o final de uma sobreposição, instruções especiais (inseridas pelo compilador) carregam as próximas sobreposições do disco rígido e configuram uma nova página.saltoinstrução para a CPU seguir. A técnica pode ser vista como um exercício de paginação de grafos, semelhante a como Gromit coloca trilhos na frente de um trem em miniatura em movimento enquanto está sentado em cima dele em "As Calças Erradas".

Curiosidades: Em 2017, mais de quarenta e cinco anos após o lançamento do 8086, em nome da retrocompatibilidade, todos os PCs do mundo ainda iniciavam em modo real. Um carregador de inicialização alternava para o modo protegido, carregava o kernel e, então, a inicialização propriamente dita podia começar.

2.3 Vídeo

Os PCs eram conectados a monitores CRT: telas grandes, pesadas, de diagonal pequena, baseadas em raios catódicos e com superfície curva. A maioria tinha 14 polegadas na diagonal e proporção de 4:3.

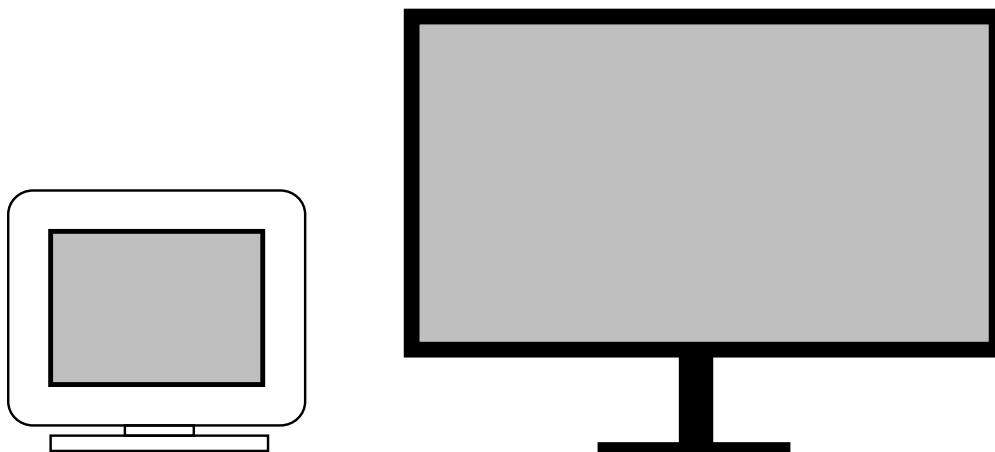


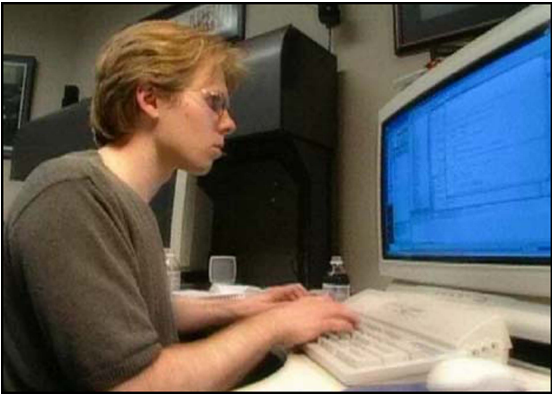
Figura 2.18: CRT (esquerda) vs LCD (direita)

Para que você tenha uma ideia do tamanho e da resolução, a figura 2.18 mostra uma comparação entre um monitor CRT de 14 polegadas de 1992 (capaz de resolução de 640x480) e um Apple Cinema Display de 30 polegadas de 2014 (capaz de resolução de 2560x1600).

Curiosidades: Apesar das diferenças em suas capacidades, ambos os monitores têm o mesmo peso: 12 kg (27,5 libras).

Curiosidades: Qual era o tamanho e o peso máximo que um monitor CRT poderia ter? O InterView 28hd96 da Intel tinha uma diagonal de 28 polegadas, permitindo uma resolução de 1920x1080 numa época em que a maioria dos monitores exibia 640x480. Ele pesava 45 kg (99,5 lb). Para comparação, um monitor LCD moderno da Dell de 27 polegadas pesa 7,8 kg (17 lb).

Na foto à direita: John Carmack em 1996 trabalhando em um 28hd96 enquanto programava Quake 2 usando o Visual Studio C++.



O principal problema desse sistema era que os monitores CRT eram analógicos, enquanto os computadores eram digitais. Era necessária uma interface entre os dois, e essa interface surgiu na forma de uma série de chips chamados "Adaptadores".

2.3.1 Histórico dos Adaptadores de Vídeo

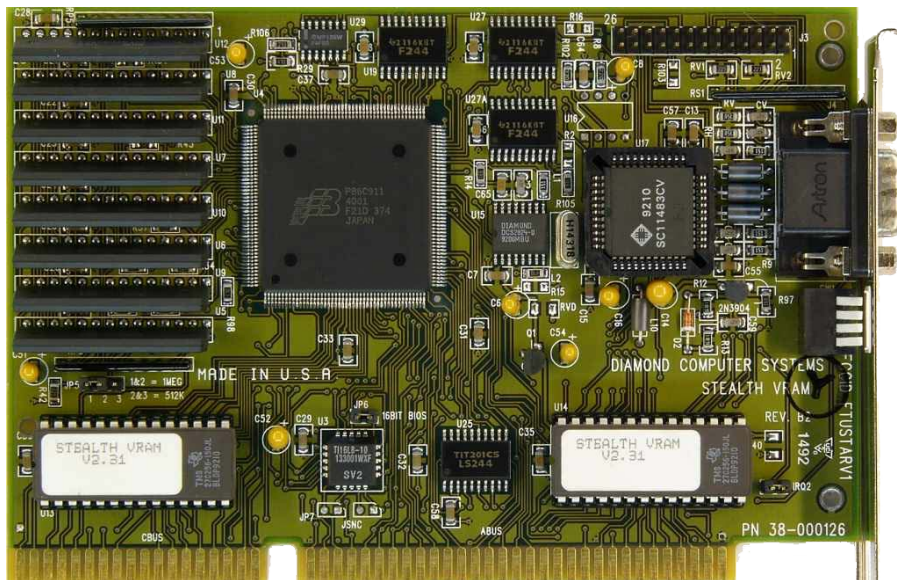
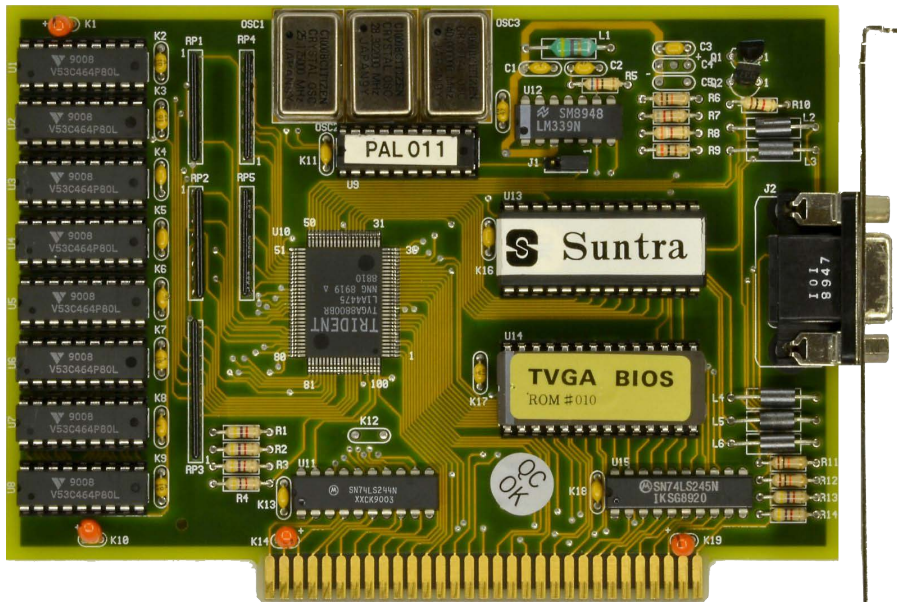
O Adaptador de Tela Monocromática (MDA) Foi lançado em 1981 com o IBM PC 5150. Oferecia duas cores, permitindo 80 colunas por 25 linhas de texto. Embora não fosse excelente, era padrão em todos os PCs. Muitos outros sistemas surgiram ao longo dos anos, cada um deles preservando a retrocompatibilidade.

Nome	Ano de lançamento
MDA (Monocromo DisplayUMadaptador) CGA	1981
(CcorGgráficosUMadaptador) EGA (E	1981
aprimoradoGgráficosUMadaptador) VGA (V	1985
vídeoGgráficosUMrray)	1987

Figura 2.19:Histórico da interface de vídeo.

Cada versão adicionava novos recursos e, em 1991, o sistema gráfico predominante era o VGA. Todas as placas de vídeo instaladas em PCs tinham que seguir o padrão estabelecido pela IBM. A universalidade desse sistema era uma faca de dois gumes. Embora os desenvolvedores tivessem que programar para apenas um sistema gráfico, não havia como escapar de suas limitações.

Na página oposta, na parte superior, está a placa VGA Trident 8800 (ISA de 8 bits). Os oito chips à esquerda da placa formam a VRAM, onde os buffers de quadros são armazenados.25Na parte inferior, um Diamond Stealth (ISA de 16 bits). Ambas as fotos são cortesia devgamuseum.info.



25Esta placa Trident TVGA8800BR é, na verdade, mais do que apenas VGA, já que seus oito chips V53C464P80L podem armazenar 64 KiB cada, totalizando 512 KiB de VRAM. Placas com mais de 256 KiB eram chamadas de Super-VGA, mas isso é outra história.

Curiosidades: Naquela época, não existia mercado de GPUs. Como todas as placas de vídeo "apenas" precisavam ser compatíveis com VGA e ter 256 KiB de RAM, muitos compravam a opção mais barata disponível. Observe, no entanto, que algumas placas tinham um conector de barramento ISA de 8 bits (como a Trident) e outras tinham um conector ISA de 16 bits (como a Diamond), o que as tornava quase duas vezes mais rápidas.

2.3.2 Arquitetura VGA

O VGA pode ser resumido em três sistemas principais: entrada, armazenamento e saída:

- O Controlador Gráfico e o Controlador de Sequência controlam como a RAM VGA é acessada (a interface CPU-VRAM)
- O framebuffer (a VRAM) é composto por quatro bancos de memória de 64 KiB (em vez de um banco de 256 KiB)
- O controlador CRT e o DAC²⁶ Responsável por converter o framebuffer indexado por paleta para RGB e, em seguida, para sinal analógico (interface VRAM-CRT).

A parte mais surpreendente da arquitetura é, obviamente, o framebuffer. Por que usar quatro pequenos bancos fragmentados em vez de um único banco linear grande?

Parte da explicação reside na retrocompatibilidade. O EGA, antecessor do VGA, possuía apenas 64 KiB de RAM. Era muito fácil projetar um sistema retrocompatível que utilizasse apenas um banco de 64 KiB.

A razão mais significativa era a latência da RAM e a necessidade de largura de banda mínima. Um monitor CRT rodando a 60Hz e exibindo 640x480 em 16 cores precisa de um pixel a cada $\frac{640 \times 480 \times 16}{60}$ em milésimos de segundo. Nessa resolução, um pixel é codificado com 4 bits. Cada nibble é traduzido para uma cor RGB de 666 bits pelo DAC. Essa codificação divide a largura de banda por 4,5, mas ainda requer um byte a cada 108 nanossegundos.

Infelizmente, a latência de acesso à RAM foi de 200 ns - muito aquém do necessário.²⁷ Para atualizar a tela a 60 Hz, o DAC ficaria sobrecarregado. Se a latência não pudesse ser reduzida, a taxa de transferência ainda poderia ser melhorada lendo de quatro bancos simultaneamente. A leitura em paralelo resultou em uma latência de RAM amortizada de $200/4 = 50$ ns, o que era suficientemente rápido.

Lembre-se de que essa arquitetura reduziu a penalidade das operações de leitura, mas plotar um pixel no framebuffer com uma operação de escrita ainda era lento. Escrever na VRAM o mínimo possível era crucial para manter uma taxa de quadros decente.

²⁶Conversor Digital para Analógico.

²⁷Computação Gráfica: Princípios e Prática, 2ª Edição, página 168.

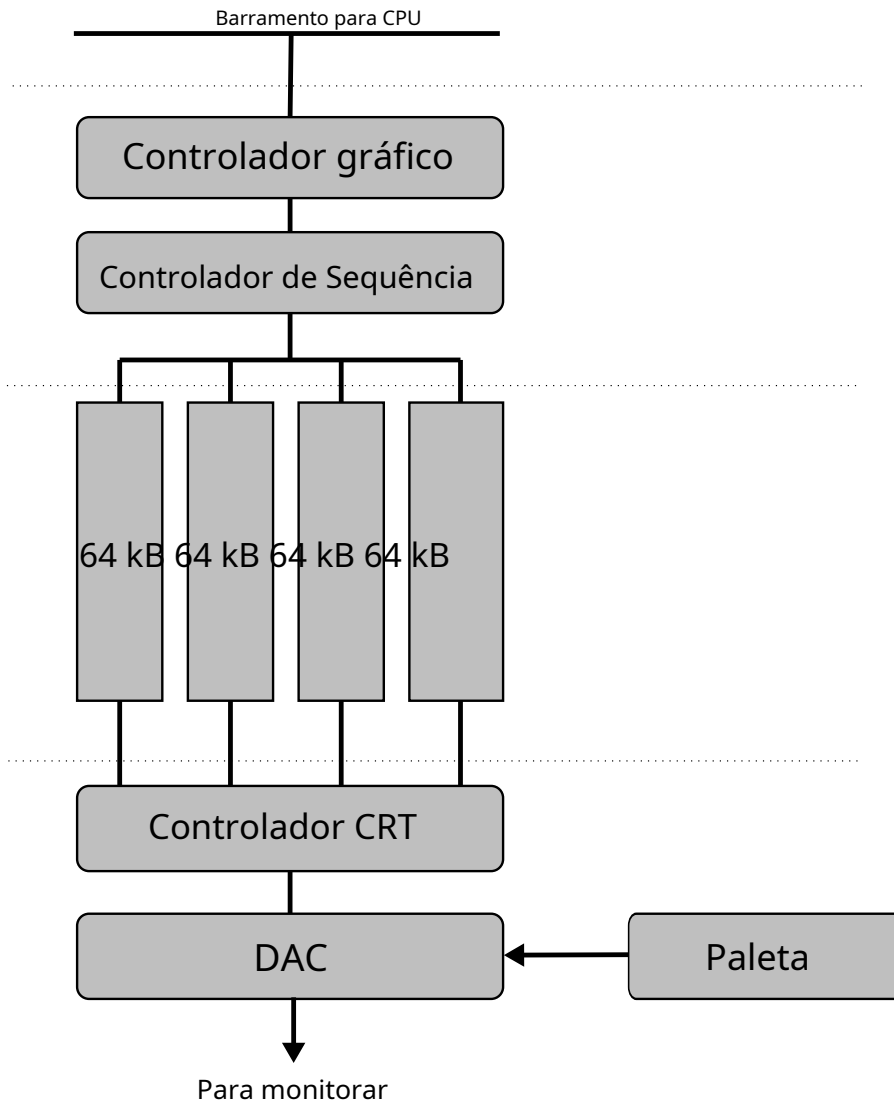


Figura 2.20:Arquitetura VGA.

2.3.3 Loucura Planar VGA

Enquanto os bancos de órfãos garantem taxa de transferência suficiente para atingir altas resoluções a 60Hz/70Hz. O preço reside na complexidade da programação, como reconhecido até mesmo pelos melhores programadores da época.

“

Direito de f o bAlém disso, gostaria de tornar uma coisa muito perfeitoClaramente claro: O VGA é às vezes es ve difícil de programar para um bom desempenho. rforma difícil.

Miguel I Abra sh - Programação Gráfica B falta Book

”

O primeiro problema lem wiO problema desse design é que ele é unilateral. itivo. T não há nenhum framebuffer linear aqui. e Descobrir qual byte corresponde a qual pixel na tela é difícil.

Esse tipo de arquitetura é chamado de "planar". No modo 13h, onde um pixel é codificado em um byte, para escrever quatro pixels lado a lado em uma linha na tela, é necessário escrever um byte em cada banco. Cada um desses bancos é mapeado para o mesmo endereço de memória UMA. Esse layout é melhor explicado com um desenho.

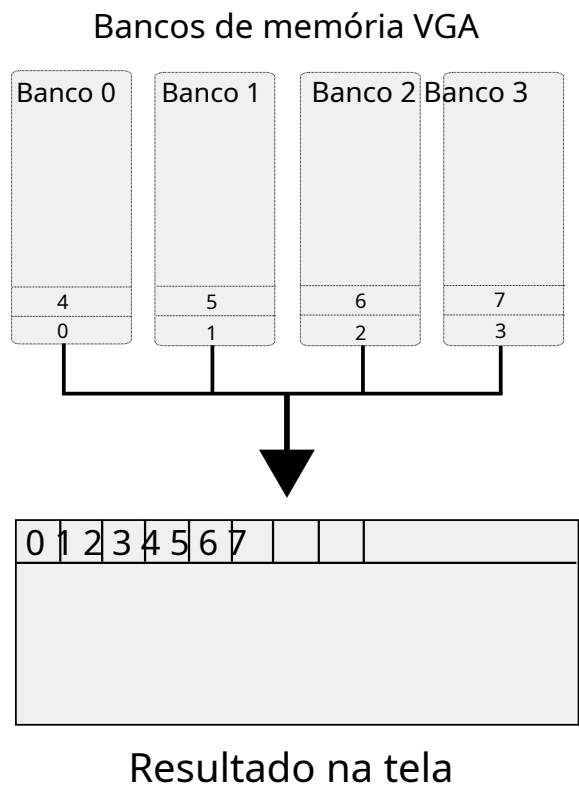


Figura 2.21: Modo VGA 13h, como o layout do banco aparece na tela.

Para configurar essa confusão de aviões e controladores, é preciso ajustar 5 registradores0 pobre y documentado int ernal
Escusado será dizer que poucos programadores se aventuraram nessa tarefa. para dentro de componentes internos do VGA.

A Figura 2.20, que descrevia a arquitetura, na verdade era aengano Muito simplificado. F figura 2.22, que mostra como a documentação de referência da o VG A. O labirinto de arame IBM explicava a complexidade real do sistema.

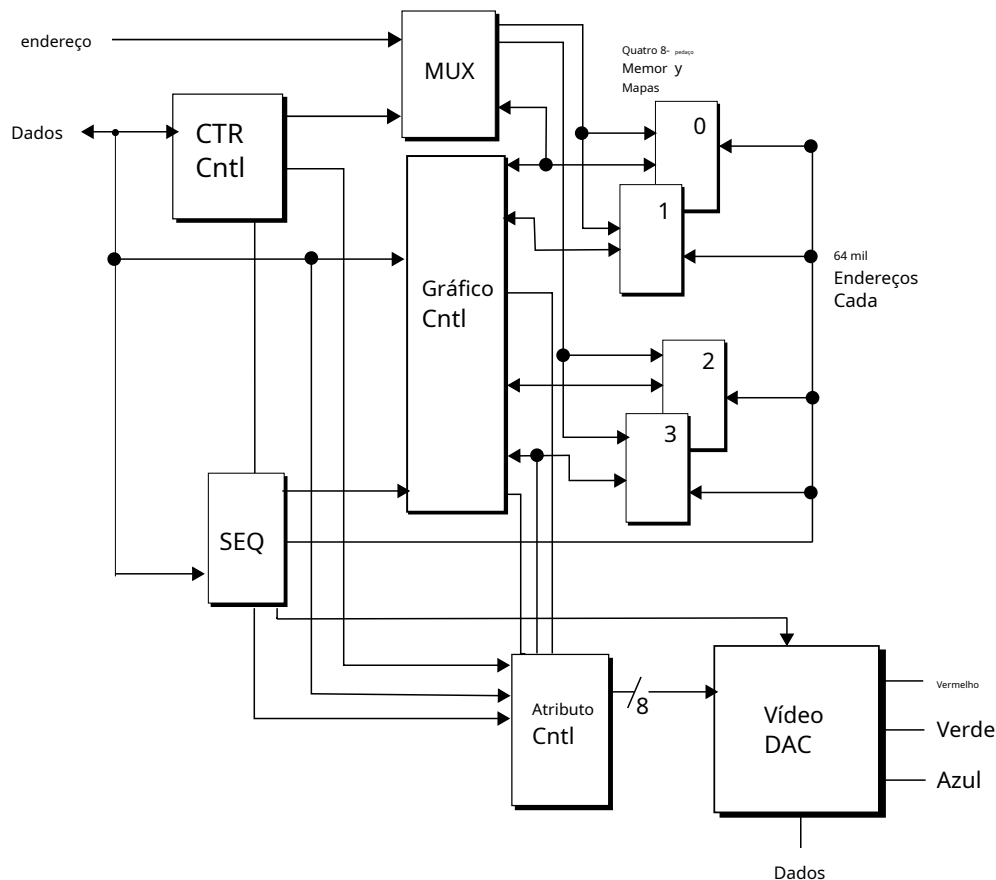


Figura 2.22: Documentação VGA da IBM.

Para compensar a complexidade, a IBM forneceu uma rotina para inicializar todos os registradores por meio de uma única chamada de BIOS. Um modo pode ser selecionado dentre os 15 disponíveis, cada um com sua respectiva resolução, número de cores e layout de memória.

2.3.4 Modos VGA

É possível acessar a BIOS para configurar a placa de vídeo (VGA) da seguinte forma.

Modo	Tipo	Formatar	Cores	Mapeamento de RAM	Hz
0	texto	40x25	16 (monocromático)	B8000h	70
1	texto	40x25	16	B8000h	70
2	texto	80x25	16 (monocromático)	B8000h	70
3	texto	80x25	16	B8000h	70
4	Gráficos CGA	320x200	4	B8000h	70
5	Gráficos CGA	320x200	4 (monocromático)	B8000h	70
6	Gráficos CGA	640x200	2	B8000h	70
7	Texto MDA	9x14	3 (monocromático)	B0000h	70
0Dh	Gráfico EGA	320x200	16	A0000h	70
0Eh	Gráfico EGA	640x200	16	A0000h	70
0Fh	Gráfico EGA	640x350	3	A0000h	70
10h	Gráfico EGA	640x350	16	A0000h	70
11h	Gráficos VGA	640x480	2	A0000h	70
12h	Gráficos VGA	640x480	16	A0000h	60
13h	Gráficos VGA	320x200	256	A0000h	70

Figura 2.23: Modos VGA disponíveis.

Os programadores faziam referência aos modos VGA por seus IDs. Era comum encontrar tutoriais sobre modos.12hou modo13h,que eram os dois modos mais adequados para programação de jogos.

2.3.5 Programação VGA: Mapeamento de Memória

Para escrever na VRAM, o espaço de endereçamento de 1 MiB da RAM mapeia 64 KiB, começando conforme indicado na tabela 2.23. No modo13hPor exemplo, a VRAM é mapeada de 0xA0000 a 0xAFFFF. Uma das primeiras perguntas que vem à mente é: "Como posso acessar 256 KiB de RAM com apenas 64 KiB de espaço de endereçamento?" A resposta é "comutação de bancos", conforme resumido na figura 2.24. As operações de escrita e leitura são roteadas com base em um registrador de máscara que indica em qual banco a leitura ou a escrita deve ser realizada.

2.3.6 Programação VGA: Modo 12h

O primeiro modo geralmente considerado para programação de jogos é o modo.12h.Oferece uma resolução de 640x480 a 60 Hz.28com 16 cores. Cada pixel é codificado em 4 bits (um nibble) distribuídos.

²⁸Devido aos requisitos de largura de banda, este modo é o único que funciona a 60Hz. Todos os outros funcionam a 70Hz.

nas quatro margens.

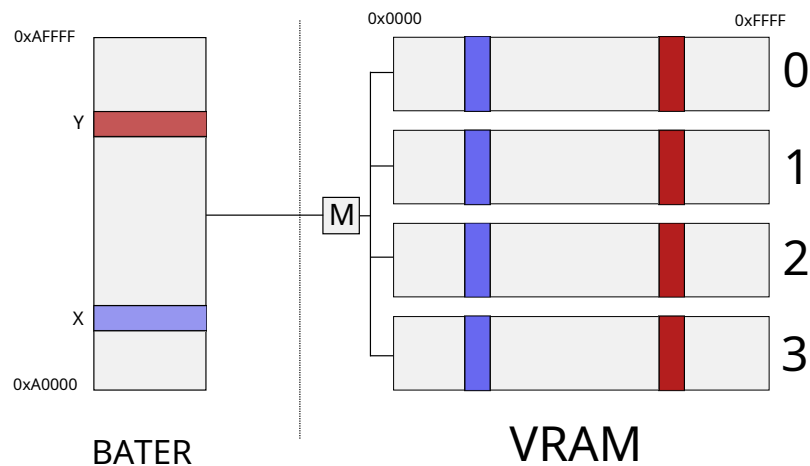


Figura 2.24: Mapeamento da RAM do PC para os bancos de VRAM da placa de vídeo.

Para escrever a cor do primeiro pixel, o desenvolvedor precisa escrever o primeiro bit do nibble no plano 0, o segundo no plano 1, o terceiro no plano 2 e o quarto no plano 3. O controlador do CRT lê então 4 bytes por vez (um de cada plano), resultando em 8 pixels na tela.

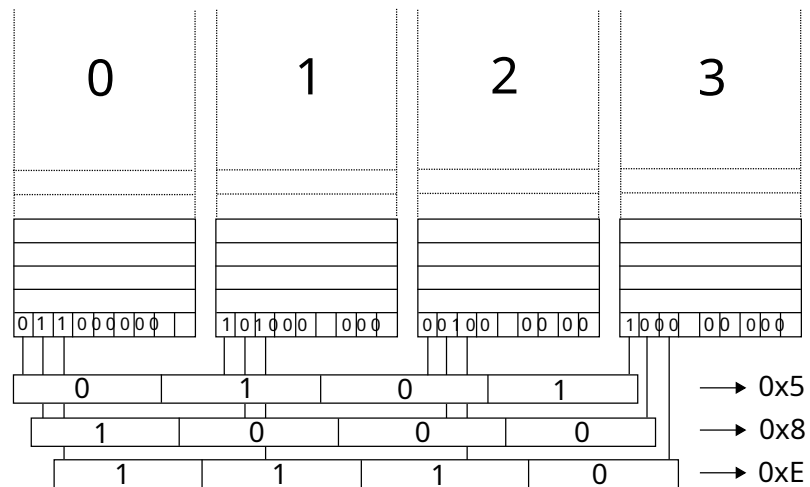


Figura 2.25: Um banco de dados em modo 12h.

A única vantagem desse modo é que ele tem pixels quadrados. A proporção de 640x480 corresponde ao 4:3 de um monitor CRT, então não há distorção do framebuffer quando ele é exibido na tela. Praticamente todo o resto é ruim.

- Sem buffer duplo: $640 \times 480 / 2 = 0x25800$ bytes, o que é mais da metade dos 256KiB (0x40000) de VRAM disponíveis.
- Alta resolução significa mais pixels, o que significa mais cálculos e mais desenho.
- Dezesesseis cores limitavam severamente os artistas. Jogos que optavam por usar esse modo geralmente tinham uma aparência pior do que aqueles que usavam o modo de 256 cores.²⁹



Figura 2.26: Como Wolfenstein 3D teria ficado se tivesse usado dezesesseis cores.

²⁹A exceção à regra é "Dark Seed", um jogo de aventura desenvolvido e publicado pela Cyberdreams em 1992. Inspirado nas obras de H.R. Giger, é notório por seus visuais deslumbrantes em 640x480, com 16 cores.

2.3.7 Programação VGA: Modo 13h

Modo13h é muito mais atraente, pois oferece uma resolução mais baixa de 320x200 com 256 cores a 70Hz. Também tem a vantagem de simular um buffer linear. Um chip especial chamado Chain-4 usa os 2 bits menos significativos do endereço da RAM para programar automaticamente a máscara e direcionar a operação para o banco de VRAM apropriado. Esse mecanismo de conveniência foi adicionado originalmente porque o modo13h foi projetado para exibir imagens estáticas e um espaço de endereçamento linear facilitou aos desenvolvedores a cópia da RAM para a VRAM.

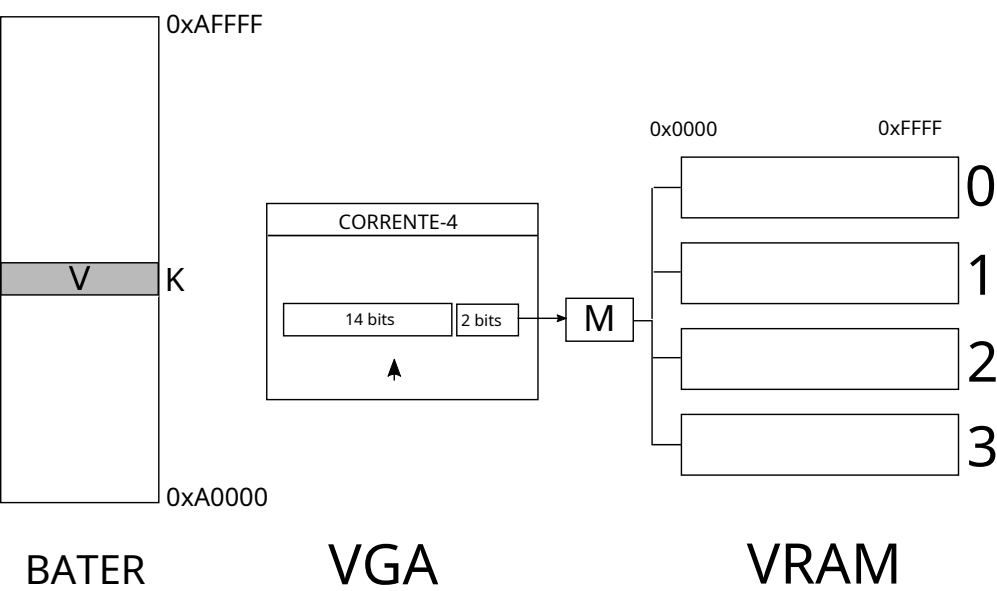


Figura 2.27: O chip Chain-4 encaminha as entradas e saídas entre a RAM e a VRAM.

O quê? Ao escrever ou ler um valor V no endereço K na RAM, a Chain-4 se divide em duas partes. S:

- Os 2 inferiores Os bits são usados para configurar a máscara 1, automaticamente. 00bg para o banco 0, 01bpara banco 10bpara o banco 2 e11bpara o banco 3.
- Os 14 mais altos r bits são deslocados para a direita em dois bits. d nós ed como um desvio no banco.

Para exemplo, escrevendo g em 0xABF13 na RAM resultaria em mandad ing em ba nk 3 em deslocamento 0xABF13 » 2 = 0x2AFC4. Si nce tudo isso está gravado na cadeia- 4, este trabalho extra é rápido e totalmente convíte sível para o uso r.

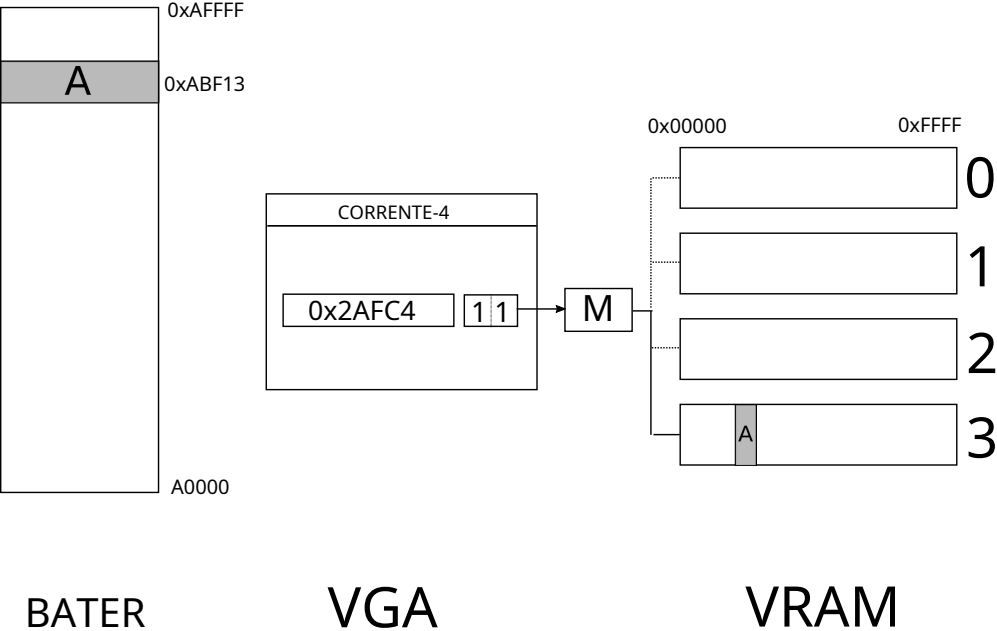


Figura 2.28

Um efeito colateral desse mecanismo de conveniência é que 75% da RAM é desperdiçada (já que 14 bits são apenas utilizáveis para o deslocamento).

2.3.7.1 Configuração

Para configurar o VGA no Modo 13h Usar a BIOS é incrivelmente fácil. Basta seguir as instruções feitas com apenas duas instruções:

```
movimento machado, 0x13 ; AH=0 (Alterar modo de vídeo) e, AL=13h (Mode)
inteiro 0x10. ; Interrupção de BIOS de vídeo
```

O `int 10` instrução A ação é uma interrupção de software capturada que para a BIOS Rotina Se é responsável por gerenciar a configuração física. consulte o `machado` registre-se para configurar tudo. 50 registradores VGA com o correspondente modo de inicialização.

Depois de inicializar a VGA, é possível escrever na memória mapeada em `0xA000`. Isto se demonstra com um exemplo de código; aqui está um código para limpar a tela. preto.

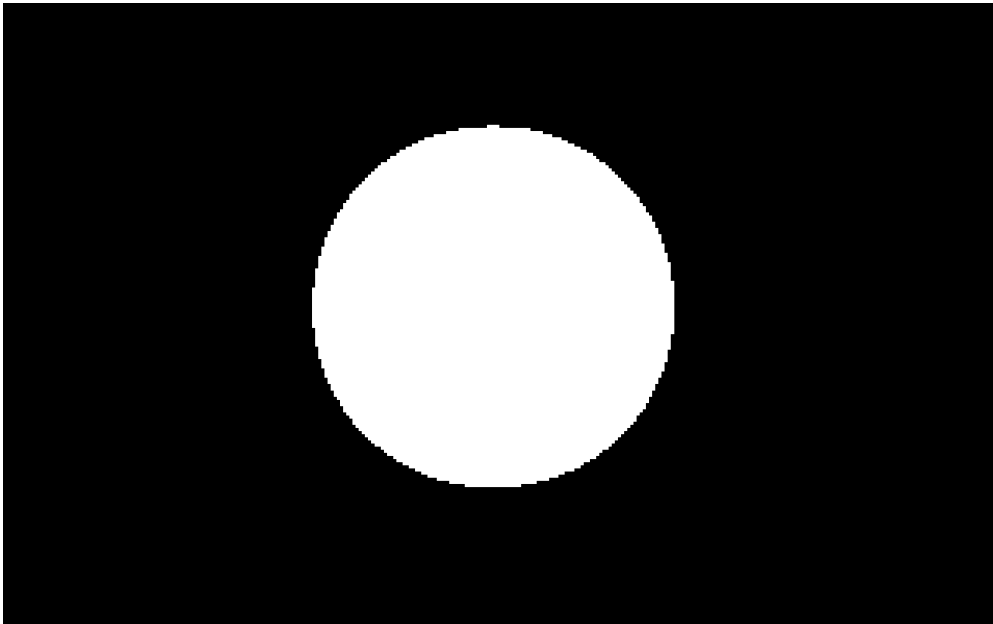
```
personagem far *VGA = (byte far*)0xA0000000L;

vazio ClearScreen(vazio){
    asm movimento eixo ,0x13
    asm inteiro 0x10

    para(inteiro i=0 ; i < 320*200 ; i++)
        VGA[i] = 0x00;
}
```

Modo13h Parece um pouco melhor do que 12h mas ainda é bastante ruim para jogos ou mesmo imagens estáticas:

- Com Chain-4, todo o espaço de endereçamento da RAM é utilizado e não há como criar um buffer duplo.
- Como a resolução é 320x200, a proporção da tela (1,6) não corresponde à do monitor (1,333). Consequentemente, o framebuffer armazenado na VRAM é distorcido ao ser transferido para o CRT. Isso pode parecer uma pequena distorção, mas é bastante problemático. Um exemplo com um círculo no framebuffer sendo exibido como uma elipse na tela ilustra bem o problema.



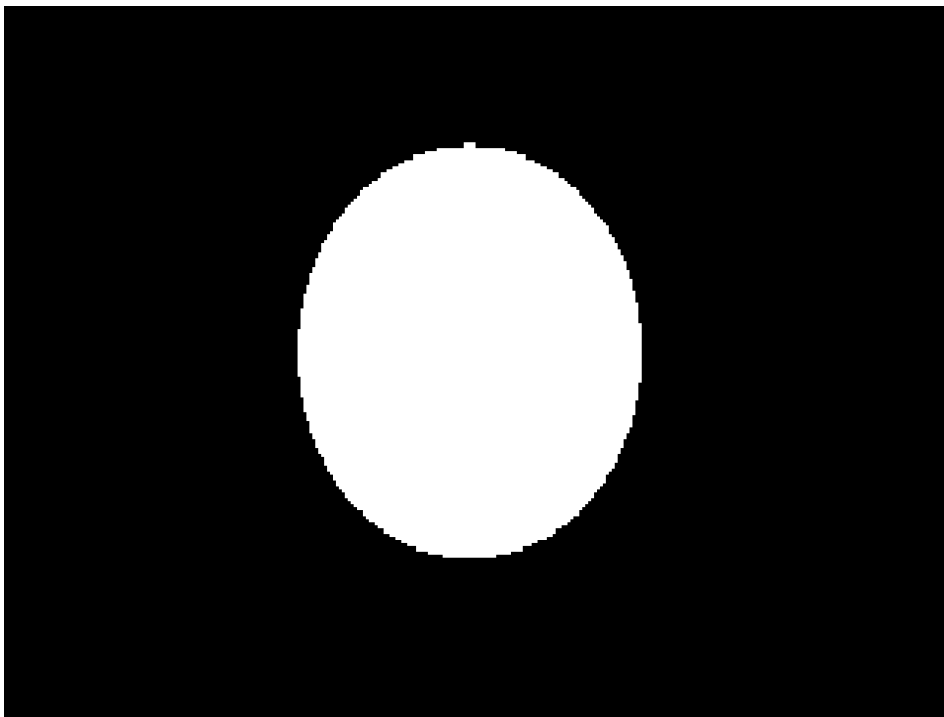
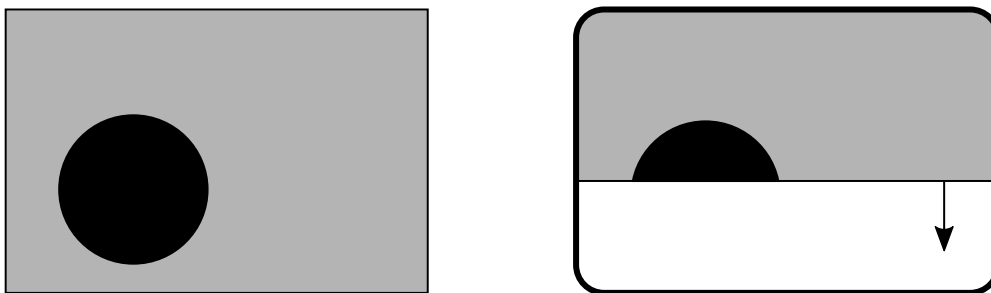


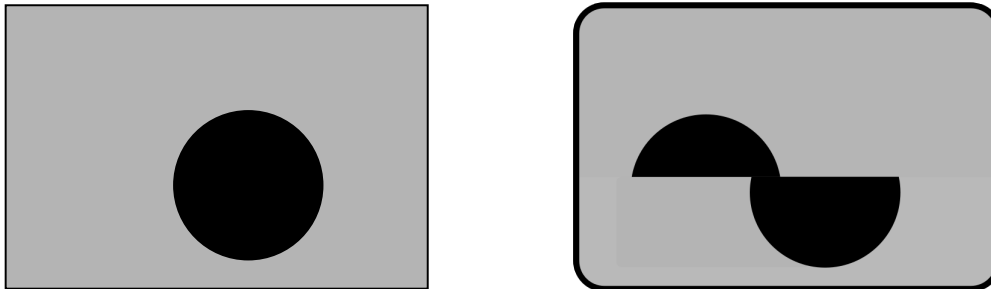
Figura 2.29: Como o framebuffer aparece quando esticado no monitor CRT.

2.3.8 A importância do buffer duplo

O buffer duplo tem sido mencionado frequentemente ao descrever o hardware, mas até agora não analisamos por que ele é fundamental para obter animações fluidas. Com apenas um buffer, o software precisa operar exatamente na frequência do CRT (70Hz). Caso contrário, ocorre um fenômeno conhecido como "rasgo" (tearing). Vejamos o exemplo de uma animação que renderiza um círculo se movendo da esquerda para a direita:



Neste exemplo, a CPU terminou de escrever o framebuffer (à esquerda) e o feixe de elétrons do CRT (à direita) começou a escaneá-lo na tela. Neste momento, o feixe de elétrons escaneou metade do framebuffer, então o círculo foi parcialmente desenhado na tela.

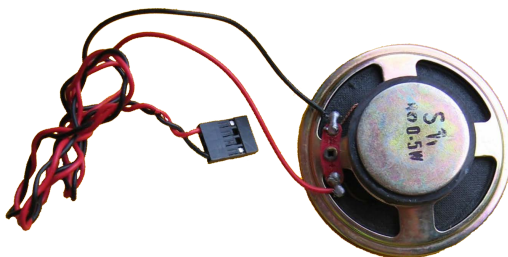


Se a CPU for mais rápida que a frequência do CRT (70 Hz), ela pode reescrever o framebuffer antes que a varredura seja concluída. Foi o que aconteceu aqui. O próximo quadro foi desenhado com o círculo movido para a direita. O feixe de elétrons não percebeu isso e continuou varrendo o framebuffer. O resultado na tela agora é uma composição de dois quadros. Parece que dois quadros foram rasgados e colados novamente. Daí o nome "rasgo".

Com dois buffers (também conhecido como buffer duplo), a CPU pode começar a escrever no segundo framebuffer sem interferir no framebuffer que está sendo exibido na tela.³⁰ Chega de rasgos!

2.4 Áudio

Os PCs vinham equipados com um pequeno dispositivo sonoro, do tamanho de uma moeda de um dólar, comumente conhecido como "alto-falante de PC", capaz de gerar uma onda quadrada através de 2 níveis de saída.



³⁰Agora, a velocidade da CPU representa a latência.

limitado pela taxa de atualização do CRT. O buffer triplo pode resolver isso, ao custo de perda de quadros.

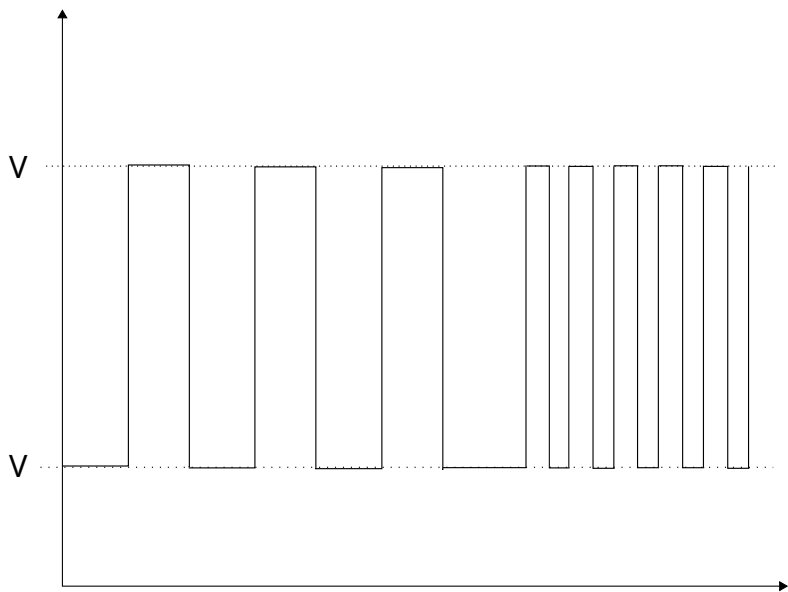


Figura 2.30: Dois bipes de frequências diferentes gerados pelo alto-falante do PC.

Até hoje, o alto-falante do PC é o primeiro dispositivo de saída a ser ativado durante o processo de inicialização. O propósito desse alto-falante primitivo é sinalizar problemas de hardware com códigos sonoros. Ele foi projetado para permanecer em silêncio após uma inicialização bem-sucedida.

Abelha p	Código	Significado g
Não Beeps		Curto, B anúncio CPU/ MB, Peri solto pela também
Um Bip		Tudo ng não é mal
Dois Bipes		PUBLICAR/ C Erro MOS r
Um Abelha Longa p, Um Sho rt Bip		Mãe quadro Prob lem
Um Abelha Longa p, Dois Sho rt Bipes		Vídeo P r problema
Um Abelha Longa p, Três S hbipes ort		Vídeo P r problema
Trêse Long Beeps		Teclado d Erro
el On ado g Bipes		Memória Erro
Continuoso Hi-	Bipes baixos	CPU Over superaquecimento

However, quadrado e ondas a re não usar completo para profissionais produzindo qualquer coisa agradável t. Son ep eople viu um potencial mmercado e empresas s começou m uma facturi ng o que eram knão usn' sound cartas "Usuários c compraria isso esse separador ateia e eu nservê-los em um dos em achine's esUM slots. Esses carros ds poderia ser conectar d para AU realdio falante rs via j de 3,5 mm ucks and temen- dously melhorou som ca phabilidades. Eu n 1991, o re eram fo ur cartas no e market:

- Cartão de música AdLib
- SoundBlaster 1.0
- SoundBlaster Pro
- Fonte de som da Disney

Embora a adoção estivesse crescendo (a Creative chegaria a vender um milhão de placas SoundBlaster em 1991), a maioria dos PCs não possuía placa de som, o que mais uma vez representava um enorme problema para os desenvolvedores de jogos.

2.4.1 AdLib

O cartão de música da AdLib foi o primeiro do mercado. A empresa foi fundada em 1988 por Martin Prevel, um ex-professor de música do Quebec. Após uma dificuldade inicial para convencer os desenvolvedores de jogos a usar seu cartão (o SDK custava US\$ 300), a AdLib conseguiu persuadir a Taito, a Velocity e a Sierra On-Line a dar suporte ao seu hardware. A Sierra, em particular, contribuiu muito para aumentar a adoção, com King's Quest IV vendendo quase 3 milhões de cópias. Logo depois, todos os jogos passaram a ser compatíveis com o "cartão de música".

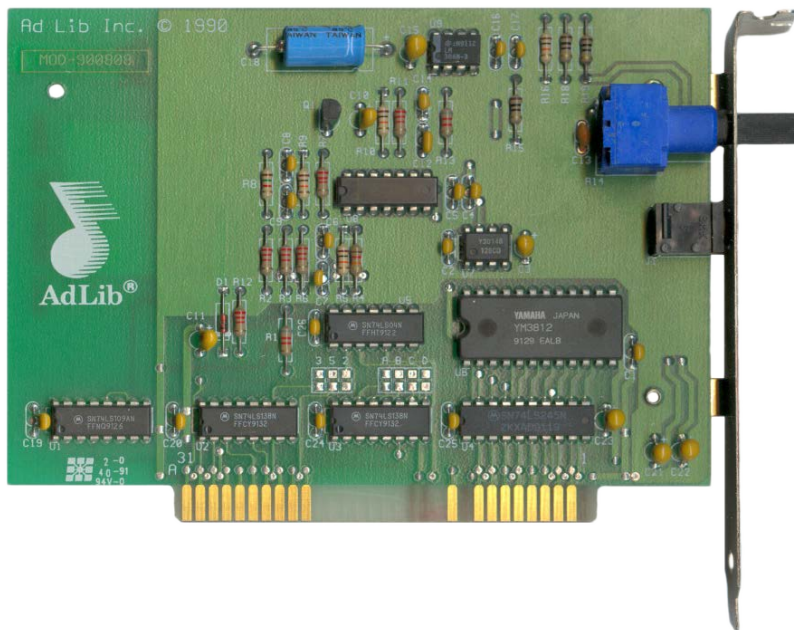


Figura 2.31: Uma placa de som AdLib. Observe o grande chip YM3812 e o conector ISA de 8 bits.

Equipada com um Yamaha YM3812, também conhecido como OPL2, a placa pode produzir 9 canais de som, cada um capaz de simular um instrumento. Baseada em síntese FM, os canais eram limitados, mas permitiam a produção de música agradável.

Curiosidades: As empresas canadenses, especialmente as do Quebec, predominavam no início dos anos 90 devido à sua capacidade tecnológica. A AdLib fabricava placas de som, a Matrox lucrou muito com sua placa gráfica Millennium e a Watcom vendia o melhor compilador C para DOS.³¹ATI³²Mais tarde, emergiria como um dos principais inovadores em GPUs na década de 2000.

2.4.2 Sound Blaster

O Sound Blaster 1.0 (codinome "Killer Kard") foi lançado em 1989 pela Creative. Era um produto inteligente que visava claramente a posição dominante do AdLib.

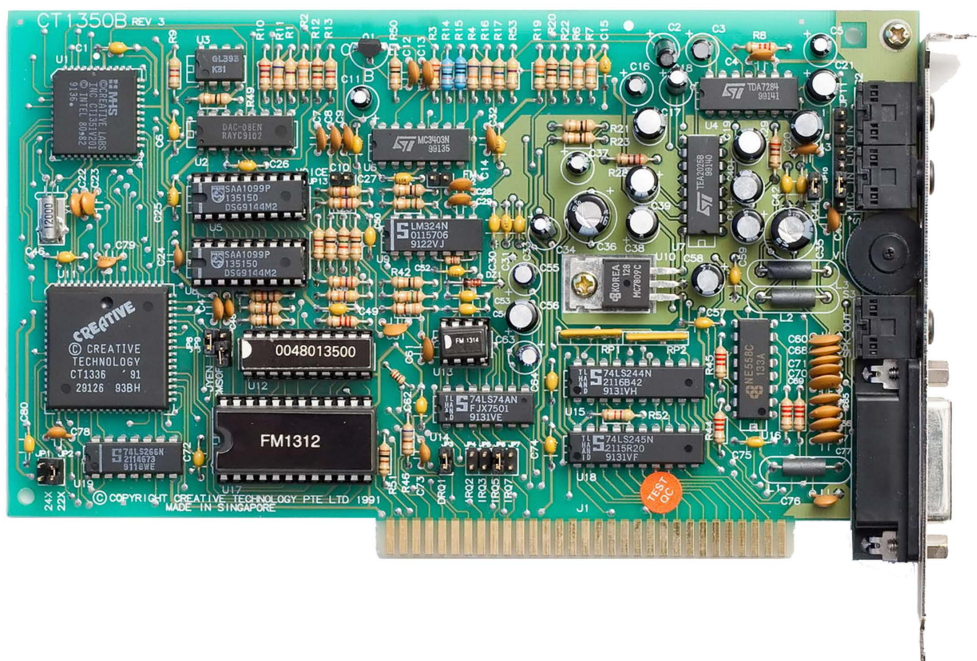


Figura 2.32:Um SoundBlaster (v1.2).

³¹O compilador da Watcom era tão bom que a id Software o usava para compilar Doom.

³²A história se repetiria no final dos anos 90 no campo das placas gráficas: Nvidia versus ATI.

Além de estar equipado com o mesmo chip OPL2, proporcionando 100% de compatibilidade com a reprodução de música AdLib, ele também era tecnologicamente superior com um DSP.³³ Permitindo a reprodução de PCM (sons digitalizados) a 8 bits por amostra e taxa de amostragem de até 22,05 kHz. A placa também vinha com uma porta DA-15, permitindo a conexão de joystick. E o mais importante: a Sound Blaster era US\$ 90 mais barata que a AdLib.

A Figura 2.32 mostra o modelo Sound Blaster CT1350B. Observe o chip OPL2 (rotulado como FM1312), a grande interface de barramento CT1336 (rotulada como "CREATIVE") no centro à esquerda, o DSP CT1351 no canto superior esquerdo e o conector de barramento ISA de 8 bits.

Curiosidades: As inúmeras vantagens da placa Sound Blaster sobre a AdLib fizeram dela o padrão de facto pouco depois do seu lançamento, levando eventualmente a AdLib à falência.³⁴

2.4.3 Sound Blaster Pro

O Sound Blaster Pro tinha todas as funcionalidades do Sound Blaster 1.0, mas adicionava suporte para reprodução estéreo de 22,05 kHz e mono de 44,1 kHz. Também incluía um "mixer" para combinar fontes de áudio (microfone, entrada de linha e CD) e escolher o nível de atenuação das saídas esquerda e direita. O som estéreo era obtido com um par de chips YM3812 (um para cada canal de áudio).

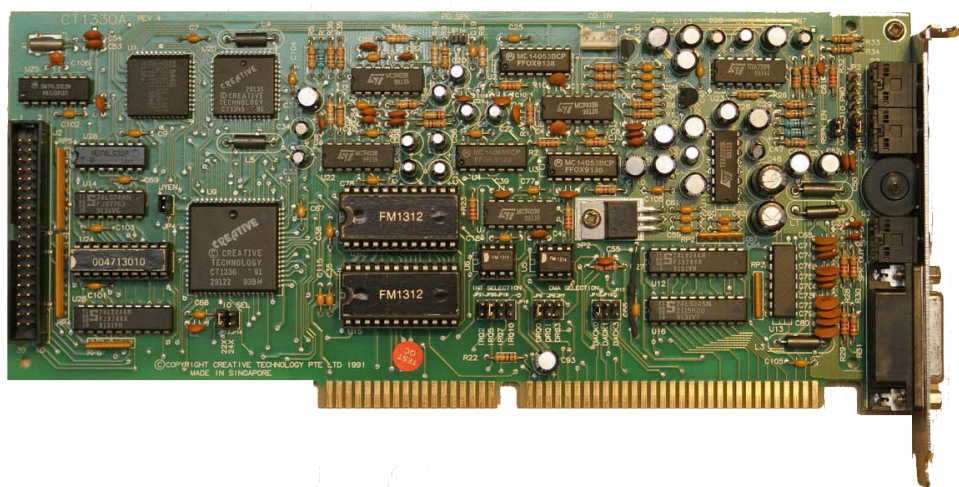


Figura 2.33: Uma placa de som Sound Blaster Pro.

³³Trata-se de um "Processador de Som Digital" Intel MCS-51, e não de um "Processador de Sinal Digital".

³⁴O reinado do Sound Blaster chegou ao fim com o Windows 95, que padronizou a interface de programação em nível de aplicativo e eliminou a importância da compatibilidade com o Sound Blaster.

A Figura 2.33 mostra uma placa de som Sound Blaster Pro, modelo CT1330A. A maioria dos componentes foi duplicada para fins estéreo. Observe o chip duplo FM1312 no centro. Os dois chips grandes no canto superior esquerdo são o DSP (CT1341) e o Mixer. Abaixo, com a etiqueta "Creative", está a grande interface de barramento CT1316.

Curiosidades: A placa parece ter um conector de barramento ISA de 16 bits (verifique a diferença com a Sound Blaster 1.0). No entanto, ela não possui "pinos" para transferência de dados na porção superior "AT" do conector de barramento. Ela usa a extensão de 16 bits para o barramento ISA para fornecer ao usuário uma opção adicional para um canal IRQ (10) e DMA (0) encontrado apenas na porção de 16 bits do conector de borda.

Observe que, bem à esquerda do cartão, em preto, há uma interface Panasonic para conectar um CD-ROM. Essa era a única maneira de conectar um CD-ROM a um PC naquela época.

2.4.4 Fonte de som da Disney

Em 1990, a Disney começou a vender o Disney Sound Source (DSS). Conectado à porta de impressora (porta paralela) do PC, um DAC de 8 bits semelhante ao "Covox Speech Thing" era conectado a uma caixa de som.



Figura 2.34:Caixa de som (DAC não mostrado).

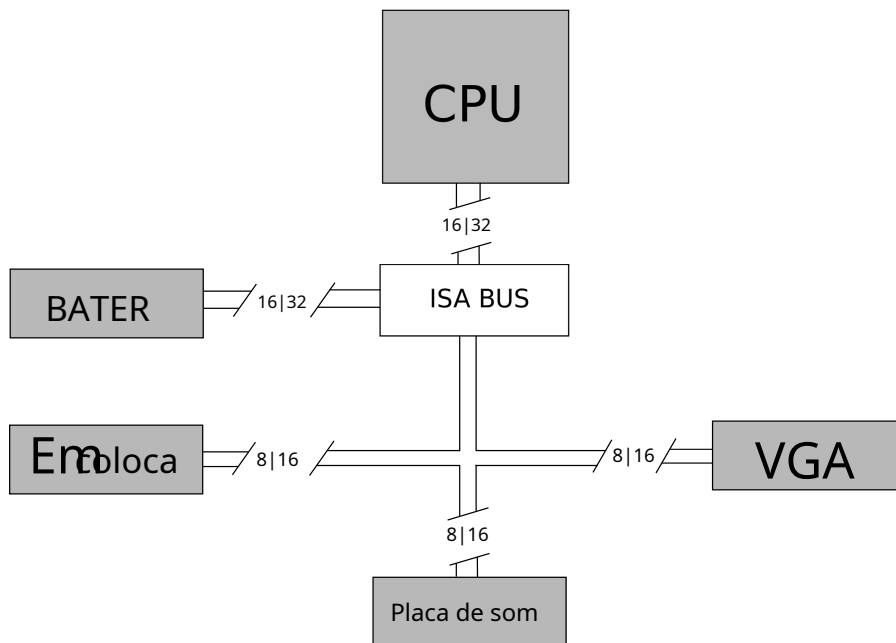
Foi incrivelmente fácil de configurar, simples de programar (só conseguia reproduzir um tipo de PCM e

Não possuía sintetizador FM e era muito barato em comparação com outras soluções de áudio (US\$ 14). Teria agradado programadores e clientes se não fosse por uma limitação grave: a largura de banda da porta paralela.³⁵ permitia uma taxa de amostragem de até 18.750 Hz, mas o design do DSS limitava a taxa de amostragem PCM a 7.000 Hz. Isso ainda era suficiente para produzir sons agradáveis, mas ficava aquém quando comparado aos 22 kHz ou mesmo aos 44 kHz de uma Sound Blaster Pro.

2.5 Ônibus

Embora os desenvolvedores não tivessem controle sobre eles, ainda vale a pena mencionar como esses componentes estavam conectados entre si.

A ISA³⁶ O barramento conecta a CPU a todos os dispositivos, incluindo a RAM. Ele já tinha 10 anos em 1991, mas ainda era usado universalmente em PCs. O caminho de dados para a RAM tem 16 bits de largura para os processadores 286 e 386SX ou 32 bits em máquinas baseadas no 386DX. Ele opera na mesma frequência da CPU.



³⁵ A largura de banda máxima da porta paralela era de 150 kbits/s na época. A Porta Paralela Aprimorada (Enhanced Parallel Port) e, posteriormente, a Porta de Capacidade Aprimorada (Enhanced Capability Port) aumentaram significativamente a taxa de transferência necessária para scanners e impressoras a laser.

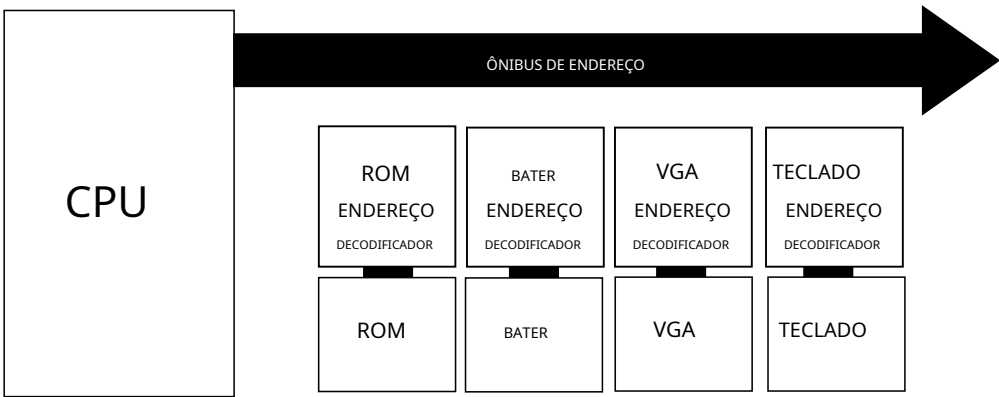
³⁶ Arquitetura padrão da indústria.

O restante do barramento, que se conecta a tudo que não seja RAM, pode ser:

- 8 bits de largura a 4,77 MHz para 19,1 Mbit/s
- 16 bits de largura a 8,33 MHz para 66,7 Mbit/s³⁷.

Também é compatível com versões anteriores, e uma placa ISA de 8 bits pode ser conectada a um barramento ISA de 16 bits.

Curiosidades: Na arquitetura ISA, todos os dispositivos estão conectados ao barramento o tempo todo e escutam na via de endereço do barramento. Cada dispositivo possui um "decodificador de endereço" para detectar se deve responder a uma solicitação do barramento. É assim que a RAM da placa de vídeo é "mapeada" na RAM. O "decodificador de endereço" da placa de vídeo filtra tudo o que não está dentro do intervalo definido. A0000h - AFFFFh. Consequentemente, a RAM ignora qualquer solicitação que esteja dentro do intervalo [A0000h - AFFFFh].



Na prática, a largura de banda efetiva do barramento é dividida por dois devido à sobrecarga de pacotes e interrupções. Como resultado, um PC equipado com uma placa VGA ISA de 8 bits pode atingir 19,1 Mbits/s / 2 / 8 = 1,1 MB/s. No modo 13h, Como um quadro tem 320x200 = 64.000 bytes, a taxa de quadros máxima teórica com uma CPU levando 0 ms para renderizar um quadro é 1.100.000 / 64.000 = 17 quadros por segundo.

Em um sistema de 16 bits VG Uma placa de vídeo com taxa de transferência de 33.400.000 bits por segundo resulta em 33.400.000/8/64.000 = 65 quadros por segundo.

Se você fatorar em outras palavras, os dados que tinham que ser transportados pelo barramento, como uma placa, chave-quadro, interruptor, rato, entradas e módulos, são transportados no quadro de amostragem de uma escavação, então o efeito consumido é 23,000/70 = 328 bytes por de 23k) é fácil para entender agora importante

³⁷https://en.wikipedia.org/wiki/Lista_de_dispositivos_por_taxas.

Isso servia para limitar a transferência de dados, e é por isso que poucos programas daquela época conseguiam atingir a taxa máxima de 70 quadros por segundo do VGA.³⁸

2.6 Entradas

Em uma época anterior à onipresença do USB, as entradas eram uma bagunça, com nada menos que quatro portas, todas programadas de forma diferente.

A porta paralela (DB-25) estava presente em todos os computadores e geralmente era usada para conectar impressoras matriciais (aparelhos barulhentos que imprimiam com agulhas). A porta paralela era multifuncional e o Disney Sound Source podia ser conectado a ela.

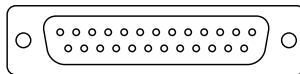


Figura 2.35: Porta paralela

A porta serial (DE9) foi usada para conectar o mouse.

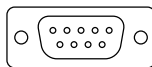


Figura 2.36: Porta serial

A porta PS/2 era usada para conectar um teclado.



Figura 2.37: Porta PS/2

Finalmente, um som Blaster sou A placa de rede conectada via barramento ISA fornecia uma porta de jogo (D) para 15) permitindo conexão um joystick.³⁹

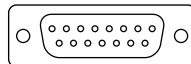


Figura 2.38: Game Port

³⁸Demonstrações especializadas poderiam atingir 70 fps em VGA por meio de um gerenciamento cuidadoso de regiões inalteradas ou usando placas planares.

Escreve para obter um aumento de velocidade de 4x.

³⁹Em 1981, o primeiro IBM PC podia ser adquirido com uma placa de expansão DA-15 "Game Port" ao preço de US\$ 55 (US\$ 159 em 2018).

Curiosidades: O monitor CRT era conectado à placa VGA através de uma porta DE15. Mais de 20 anos depois, os fabricantes ainda tentam se livrar da "porta VGA", como é comumente chamada hoje em dia. Ela se parecia muito com uma porta serial e os novatos frequentemente a danificavam ao forçar um mouse nela.

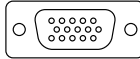


Figura 2.39: Porta VGA

2.7 Resumo

Dizer que programar um PC para jogos era difícil seria um eufemismo. Era um pesadelo. A CPU era boa em fazer a coisa errada, a melhor interface gráfica não permitia buffer duplo nem pixels quadrados, o modelo de memória permitia apenas 1 MiB padrão com um endereço composto por dois registradores de 16 bits separados, e o *operto/longeOs* proibia o uso da linguagem C padrão. Por último, mas não menos importante, o sistema de som padrão só conseguia produzir ondas quadradas e quem tinha uma placa de som instalada podia optar por qualquer uma das três principais marcas.

Apesar de todas essas condições desfavoráveis, equipes de desenvolvedores se uniram para domar a fera e liberar seu poder para os jogadores. Uma dessas equipes se autodenominava *id Software*.⁴⁰

⁴⁰Inicialmente, eles se chamavam *Ideas From the Deep*, mas depois decidiram abreviar para simplesmente *id*, que significa "em demanda" e se pronuncia como em "did" ou "kid". O nome também se refere ao *id*, a parte do cérebro que se comporta segundo o princípio do prazer na psicologia freudiana.

Capítulo 3

Equipe

Em 1990, uma pequena empresa sediada em Shreveport, Louisiana, prosperava no mercado de shareware. Como um serviço geral de assinatura de software, a Softdisk produzia e enviava títulos inovadores para seus membros. A divisão de jogos da Softdisk, chamada "Gamer's Edge", lançava títulos para Apple II, Apple IIgs, C64, Mac e PC a cada dois meses. Os negócios iam bem, mas alguns de seus funcionários tinham outras ambições. Eles acreditavam ter as habilidades necessárias para alcançar o sucesso e queriam provar isso.

Eles haviam criado uma nova maneira de programar jogos de rolagem lateral. Chamaram a tecnologia de "atualização adaptativa de blocos" e ela permitia a rolagem por hardware em um PC, capaz de rivalizar com o NES. No início de 1990, trabalharam sem parar durante um fim de semana para reimplementar Super Mario 3 em um PC e demonstrar suas habilidades à Nintendo. A equipe conseguiu criar um clone de Mario, mas infelizmente a "Ideias das Profundezas", como se autodenominavam, não conseguiu convencer a Nintendo a lhes oferecer um contrato.

Apesar de estarem impressionados, a empresa japonesa queria que a série Mario permanecesse exclusiva dos consoles da Nintendo.

“

Enviamos essa demo para a Nintendo da América, que por sua vez a enviou para Kyoto, para o **escritório central**, e os executivos de lá viram a demo e ficaram realmente **impressionados**. No entanto, eles não queriam sua propriedade intelectual em nada além de seus próprios **consoles**, então nos disseram "Bom trabalho e vocês não podem fazer isso".¹

John Romero - Programador

”

¹Demo de Super Mario Bros. 3 (1990) por John Romero: <https://vimeo.com/148909578>



Figura 3.1: Mario 3 no PC. Observe o "IFD", que significa "Ideias das Profundezas".

Este episódio foi suficiente para convencê-los de que não só tinham o talento para alimentar suas ambições, como também o espírito de equipe e a ética de trabalho necessários para potencialmente chegar ao topo. Em 1º de fevereiro de 1991, quatro funcionários da Softdisk deram um salto de fé e fundaram sua própria empresa. Deram a ela o nome de "id Software".³

Nome	Idade	Ocupação
João Carmack	21	Programador
João Romero	23	Programador
Adrian Carmack	22	Artista
Tom Hall	28	Diretor Criativo

Figura 3.2: Membros fundadores da id Software.

²"A id foi fundada em 1º de fevereiro", [rua](#), 1991", conta do Twitter de Tom Hall em 1º de fevereiro de 2020.

³"Id" também significa "Em Demanda". Sua pronúncia é semelhante à dos termos freudianos "id", "ego" e "superego".

Eles imediatamente utilizaram a tecnologia desenvolvida para "Mario 3 PC" para lançar seus próprios títulos e construir sua própria propriedade intelectual. Sem perder tempo, a equipe lançou nada menos que três títulos por ano.

- Commander Keen Episódios 1, 2 e 3: Invasão dos Vorticons (14 de dezembro de 1990)
- Commander Keen Episódios 4 e 5: Adeus, Galáxia (15 de dezembro de 1991)
- Jogo independente do Commander Keen: Aliens Ate My Baby Sitter (dezembro de 1991)

Os jogos, publicados pela Apogee Software, foram sucessos instantâneos e venderam muito bem. Eles também continuaram a desenvolver jogos para a Softdisk, a maioria dos quais apresentava atualização adaptativa de blocos.⁴

- Dangerous Dave na Mansão Assombrada (fevereiro de 1991)
- Commander Keen em Keen Dreams (abril de 1991)
- Rescue Rover (1991)
- Rescue Rover 2 (1991)
- Cavaleiros das Sombras (1991)
- Hovortank One (maio de 1991).
- Catacumba 3-D (novembro de 1991)

Na primavera de 1991, a próxima geração da tecnologia id Software começou a surgir.⁵Hovortank One colocava o jogador dentro de um tanque. Ainda não havia mapeamento de texturas e o ritmo era bastante lento. Catacomb 3-D marcou a introdução de texturas e levou a imersão um passo adiante, colocando o jogador no controle de um mago de batalha em primeira pessoa.

//

Após o lançamento do Catacomb 3-D no final de novembro de 1991, estávamos finalizando o Commander Keen Goodbye, Galaxy, e o lançamos em 15 de dezembro de 1991. Em seguida, tiramos duas semanas de férias enquanto John Carmack adquiria o computador NeXT e programava um **algoritmo de compressão de quantização vetorial VGA**. O Wolf3D começou a ser desenvolvido quando a equipe estava reunida, em janeiro de 1992.

João Romero

//

⁴O contrato com a Softdisk começou em fevereiro de 1991 e terminou em 31 de janeiro de 1992. Como precisavam continuar produzindo jogos, terceirizaram dois deles para se concentrarem em Wolfenstein 3D. "Tiles of the Dragon" e "Scubaventure" foram esses dois últimos jogos produzidos por terceirizados.

⁵Você pode ver capturas de tela na seção Apêndice, nas páginas 294 e 295.

Em novembro de 1991, a equipe ficou livre de quaisquer obrigações com a Softdisk. Seu próximo jogo **utilizaria** a tecnologia 3D que estavam desenvolvendo e se chamaria Wolfenstein 3D. Quatro novas pessoas se juntaram à equipe, totalizando oito membros.

Nome	Idade	Ocupação
Jay Wilbur	30	Negócios
Kevin Cloud ⁶	27	Artista de computação gráfica
Bobby Prince ⁷	37	Compositor
Jason Blochowiak ⁸	21	Programador



Figura 3.3:A equipe, como mostrada em Spear of Destiny, em um Easter Egg criado por John Romero. Para visualizar essa tela, o jogador precisava acessar o menu Alterar Visualização, diminuir o tamanho da tela em um nível, pressionar as teclas I e D simultaneamente e, em seguida, pressionar Enter.

⁶Jay e Kevin foram recrutados em abril de 1992, mas mesmo assim receberam os créditos.
⁷Bobby já havia trabalhado com a id Software anteriormente no Keen 4-6 como contratado, mas nunca foi um funcionário em tempo integral.
⁸Jason escreveu parte do **gerenciador de páginas** e é creditado por apresentar **John Carmack** ao desenvolvimento **Unix**, o que acabou levando à compra de um **NeXTstation Color**. Ele saiu alguns meses após a **criação** do id.



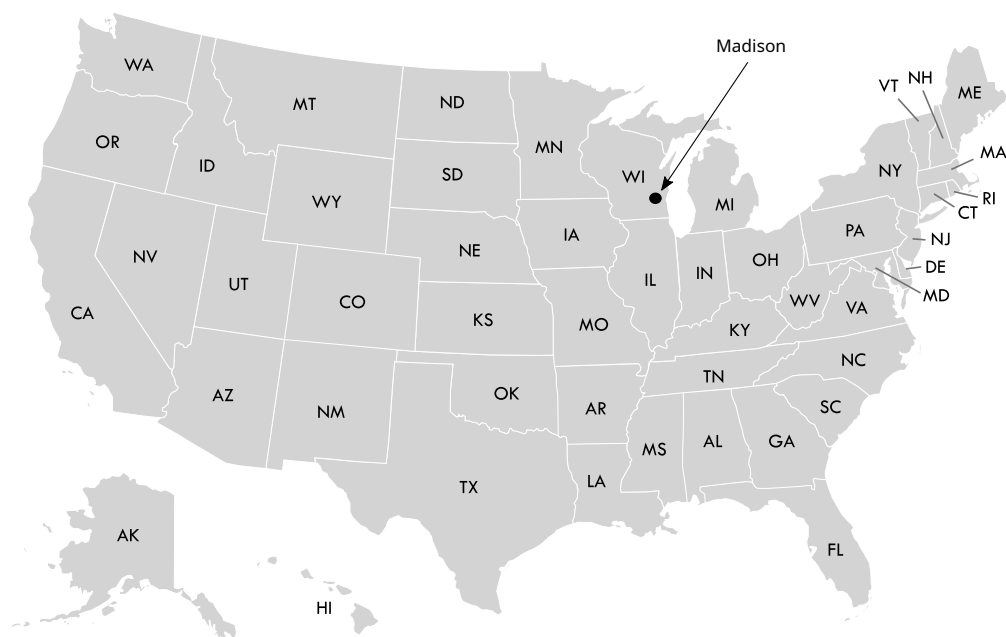
Figura 3.4: Eles estavam, de fato, usando calças.

3.1 Organização

Em setembro de 1991, inspirados pelas lembranças de Tom e Jason da época do ensino médio na região, a equipe se mudou de Shreveport, Louisiana, para Madison, Wisconsin. Eles estabeleceram seu escritório em um prédio de tijolos de dois andares no complexo de apartamentos The Pines, localizado no número 2622 da High Ridge Trail. Todos moravam a uma curta distância a pé do escritório, com exceção de John Carmack que, por não se importar, ocupava o segundo andar do apartamento.

O desenvolvimento de Wolfenstein 3D começou em janeiro de 1992. Com a queda das temperaturas e a neve caindo do céu, a equipe se manteve cada vez mais ocupada e quase não saía do escritório. O desenvolvimento durou quatro meses e Wolfenstein 3D foi lançado em maio de 1992.





Durante esses quatro meses, a organização da equipe foi praticamente padrão para um estúdio de jogos daquela época: quatro caras **amontoados** em uma sala, ditando um ritmo acelerado e um forte senso de camaradagem (e muita perturbação sonora, dado o tipo de interação entre John Romero e Tom Hall).

No mapa (página seguinte), você pode ver no andar superior o **SNES**, onde inúmeras partidas de F-Zero foram jogadas, e a área de Dungeons & Dragons, amplamente mencionada em "Masters of Doom". Ter um membro da equipe (John Carmack) com seu apartamento diretamente acima do estúdio não era algo incomum.⁹

"

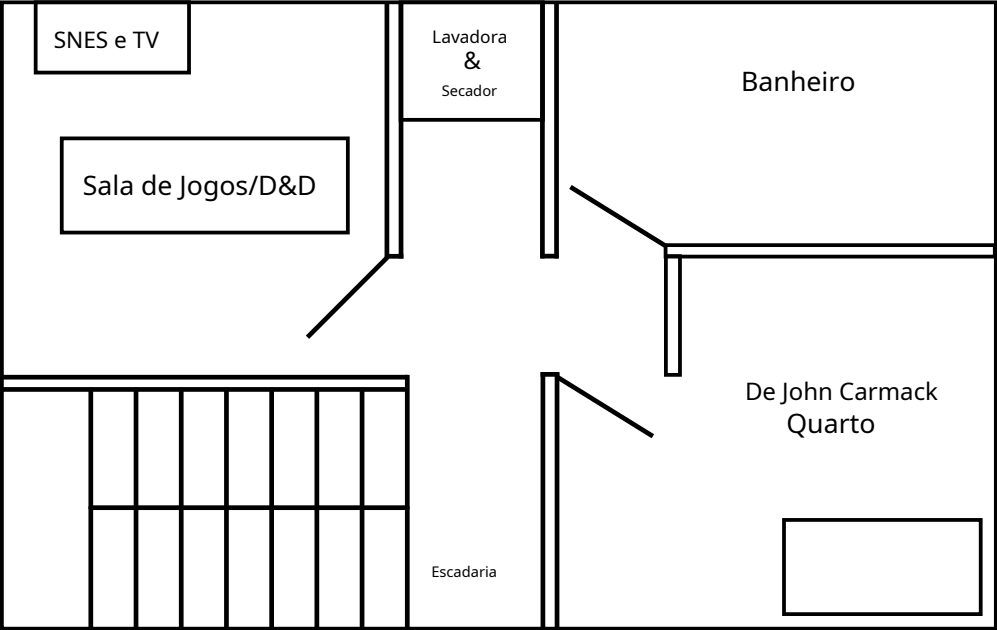
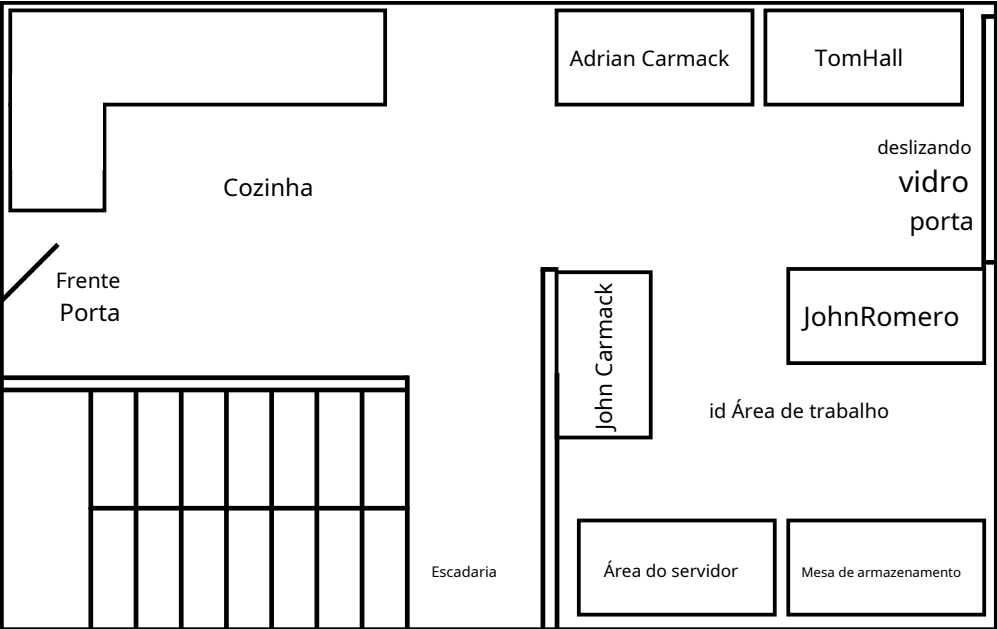
Começamos com **transferência de dados** por **disquete**, mas no final tínhamos uma **rede Novell** em **Ethernet coaxial**.¹⁰ Não tínhamos um sistema de controle de **versão**. Surpreendentemente, chegamos até o **Quake 3** sem um, e só depois começamos a usar o Visual Source Safe.

John Carmack - Programador

"

⁹"90 horas por semana e adorando!" por Andy Hertzfeld.

¹⁰Comentário de John Romero: "Compramos um sistema Novell Netware 3.11 de US\$ 7.000 em Madison, Wisconsin, em novembro de 1991. Isso incluía o servidor de arquivos, cabos e placas de rede."



Todos trabalhavam com o melhor PC que o dinheiro podia comprar, um 386-DX de última geração com 33MHz e 4MiB de RAM.

3.2 Programação

O desenvolvimento foi feito com Borland C++ 3.1 (mas a linguagem utilizada foi C), que por padrão rodava no modo VGA 3, oferecendo uma tela com 80 caracteres de largura e 25 caracteres de altura.

John Carmack cuidou do código de tempo de execução. John Romero programou muitas das ferramentas (editor de mapas TED5, compactador de recursos IGRAB, compactador de sons MUSE). Jason Blochowiak escreveu subsistemas importantes do jogo (gerenciador de entrada, gerenciador de páginas, gerenciador de som, gerenciador de usuários).

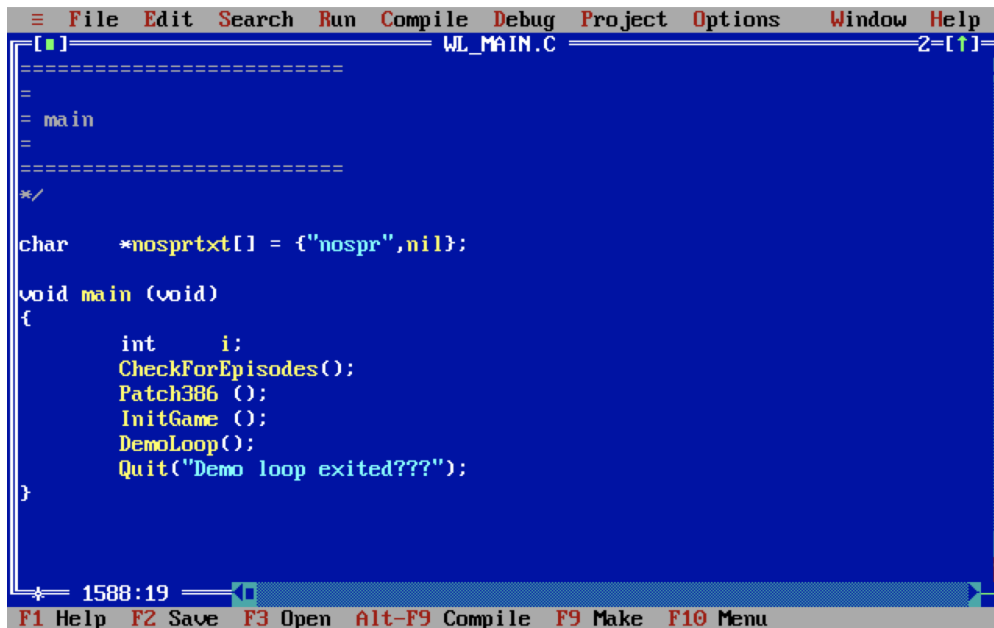


Figura 3.5: Editor Borland C++ 3.1

A solução da Borland foi um pacote completo. O IDE, BC.EXE, Apesar de algumas instabilidades, permitia edição de código rudimentar em múltiplas janelas com realce de sintaxe agradável. O compilador e o linker também faziam parte do pacote sob BCC.EXE e TLINK.EXE¹¹.

¹¹Fonte: Guia do Usuário do Borland C++ 3.1.

Não havia necessidade de entrar no **modo de linha de comando**. O IDE permitia criar um projeto, compilar, executar e depurar.

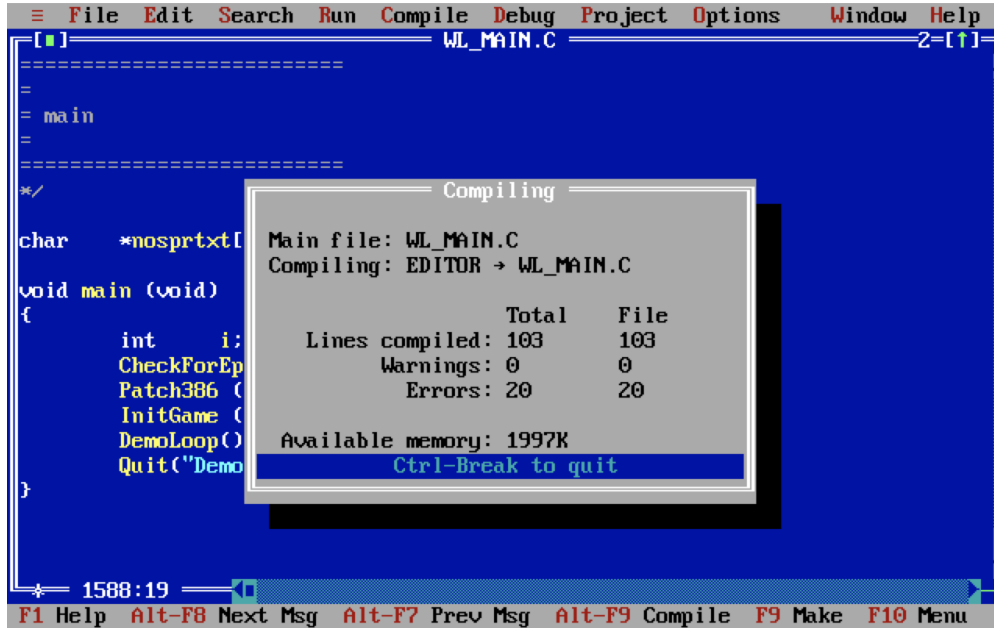


Figura 3.6:Compilando Wolfenstein 3D com Borland C++ 3.1

Para compensar o tamanho reduzido da **tela CRT**, alguns desenvolvedores usaram duas telas.¹²

//

Naquele momento, queríamos monitores de 21 polegadas, mas não conseguimos justificar o custo. Usei um segundo monitor monocromático para permitir que o Turbo Debugger 386 mantivesse a tela principal no modo **gráfico** enquanto eu depurava o código passo a passo.

John Carmack - Programador

//

Você deve ter notado na **lista de modos VGA** que o modo 13h e modo 3h ((ver Figura 2.23, p. 54) não têm o mesmo endereço inicial de RAM. Isso permite um truque em que duas placas gráficas são conectadas ao mesmo PC. Uma MDA configurada em modo **texto** monocromático capta dados em 0xB8000, enquanto uma VGA mapeada em 0xA0000 executa o jogo normalmente.

¹²Nota de John Romero: "Tanto eu quanto o John tínhamos pequenos monitores monocromáticos âmbar de 12 polegadas que usávamos com o Turbo Debugger."


```
File Edit View Run Breakpoints Data Options Window Help READY...
[CPU 80486]
main:
cs:17FA55      push  bp
cs:17FB 8BEC      mov  bp,sp
#WL_MAIN#1606:
cs:17FD 9A1C3D614F  call far _CheckForEpisodes
#WL_MAIN#1608:
cs:1802 0E      push  cs
cs:1803 E80BEB      call _Patch386
#WL_MAIN#1610:
cs:1806 0E      push  cs
cs:1807 E863FA      call #WL_MAIN#1145
#WL_MAIN#1612:
cs:180A 0E      push  cs
cs:180B E8DBFD      call _DemoLoop
#WL_MAIN#1614:
48AB:0000 CD 20 FF 9F 00 EA FF FF = f 0
48AB:0008 AD DE E0 01 CC 15 AA 01 i k0S-0
48AB:0010 CC 15 89 02 27 10 99 01 S00'00
48AB:0018 01 01 01 00 02 FF FF FF 000 0
48AB:0020 FF FF FF FF FF FF FF FF

ax FF00 c=0
bx 4768 z=1
cx 0000 s=0
dx 4768 o=0
si 474A p=1
di 4768 a=0
bp FFEC i=1
sp FFE0 d=0
ds 824B
es 824B
ss 824B
cs 4D44
ip 17FA

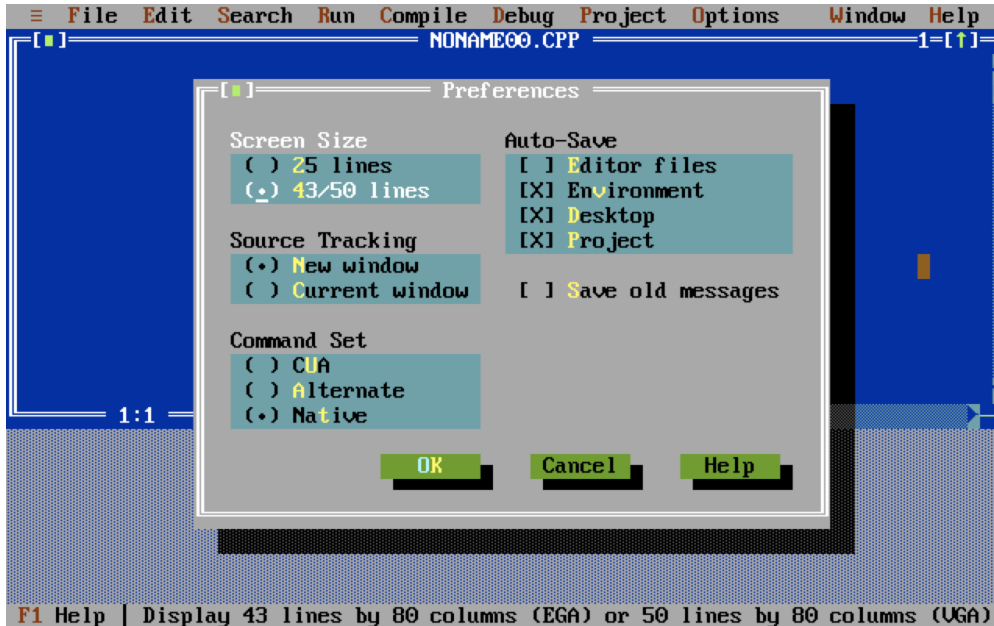
ss:FFE2 48BB
ss:FFE00170
ss:FFDE 3246
ss:FFDC 4D44
ss:FFDA 17FB

F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu
```

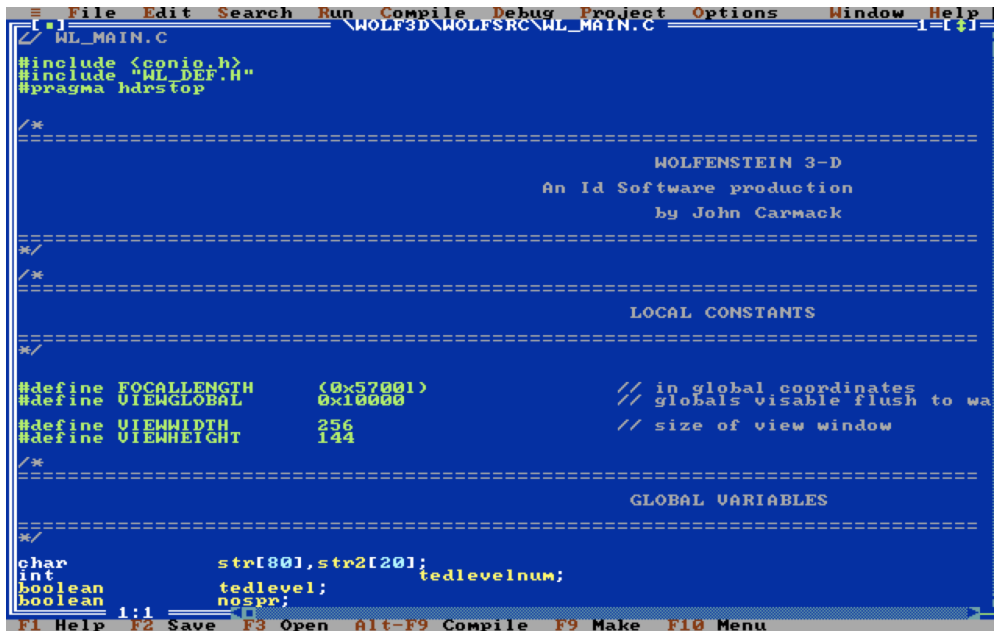
Acima, tela do MDA com o depurador Borland 3.1. Abaixo, a tela VGA executando o jogo.



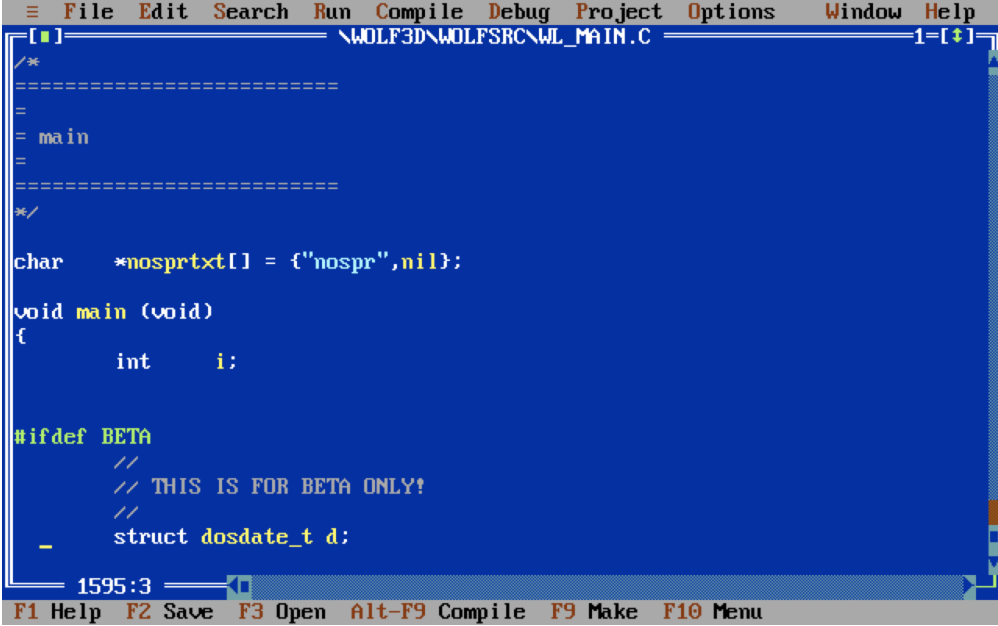
Outra forma de melhorar o **espaço disponível na tela** foi usar o modo de texto de "alta resolução" 50x80.



Os comentários ainda cabem perfeitamente na tela, já que apenas a resolução vertical foi duplicada.



O arquivo WL_MAIN.CA tela aberta em ambos os modos demonstra a relação de compromisso entre **legibilidade** e **visibilidade**.



```

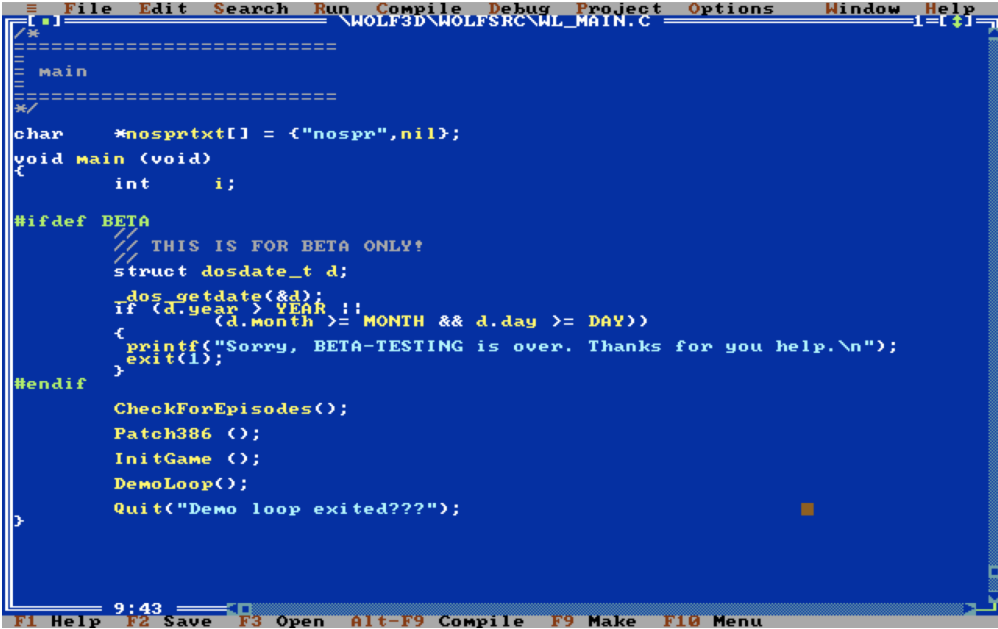
File Edit Search Run Compile Debug Project Options Window Help
[.] \WOLF3D\WOLF3SRC\WL_MAIN.C 1=[+]
/*
=====
=
= main
=
=====
*/

char *nosprtxt[] = {"nospr", nil};

void main (void)
{
    int i;

#ifdef BETA
    //
    // THIS IS FOR BETA ONLY!
    //
    struct dosdate_t d;
1595:3
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

```



```

File Edit Search Run Compile Debug Project Options Window Help
[.] \WOLF3D\WOLF3SRC\WL_MAIN.C 1=[+]
/*
=====
= main
=
=====
*/

char *nosprtxt[] = {"nospr", nil};
void main (void)
{
    int i;

#ifdef BETA
    //
    // THIS IS FOR BETA ONLY!
    struct dosdate_t d;
    _dos_getdate(&d);
    if (d.year > YEAR ||
        (d.month >= MONTH && d.day >= DAY))
    {
        printf("Sorry, BETA-TESTING is over. Thanks for you help.\n");
        exit(1);
    }
#endif

    CheckForEpisodes();
    Patch386 ();
    InitGame ();
    DemoLoop();
    Quit("Demo loop exited???");
}
9:43
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

```

3.3 Recursos Gráficos

Todos os elementos gráficos foram produzidos por [Adrian Carmack](#).¹³ Todo o trabalho foi feito com o Deluxe Paint (de Brent Iverson, Electronic Arts) e salvo no ILBM.¹⁴ arquivos (formato proprietário do Deluxe Paint).



Figura 3.7: O Deluxe Paint foi usado para desenhar todos os [elementos](#) do jogo.

Como o [VGA](#) é baseado em [paletas](#) (as cores não eram especificadas via [RGB](#) de 24 bits, mas por meio de [índices](#) que apontavam para uma [tabela de 256 cores](#)), o processo criativo foi [difícil](#). [Adrian](#) teve que tomar a decisão crucial de quais [cores](#) entrariam na [paleta](#).¹⁵ Em seguida, desenha tudo usando apenas essas cores.

¹³Kevin Cloud criou algumas [texturas](#) e também trabalhou no [design](#) e [layout](#) do [guia de dicas](#) de [Wolfenstein 3D](#).

¹⁴Bitmap Intercalado.

¹⁵Alguns jogos, como "Monkey Island", usavam várias paletas de cores dependendo da seção do jogo. A id Software optou por uma solução mais simples, com uma única paleta para todo o jogo.

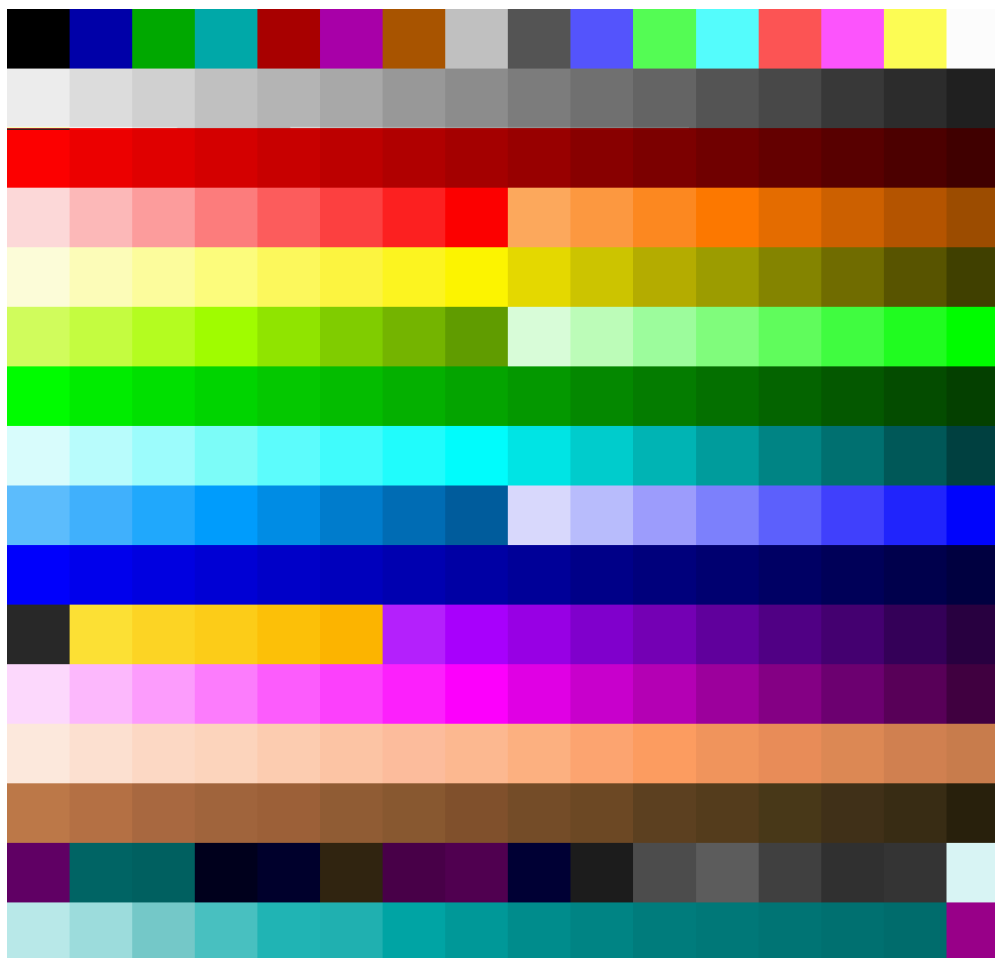


Figura 3.8: Paleta de cores de Wolfenstein 3D. Tudo no jogo é desenhado usando essas 256 cores.

As coordenadas da paleta são executadas 0x00 para 0xF0 horizontalmente e 0x00 para 0xF0 verticalmente. O gradiente azul horizontal na **parte inferior** começa em 0xF0 e termina em 0xFE. 0xFF (representado em rosa) é uma cor especial considerada **transparente** pelo mecanismo e sempre **ignorada** durante a **renderização**.

Todos os elementos gráficos foram desenhados à mão com o mouse. Como a placa de vídeo esticava o framebuffer ao exibi-lo na tela, Adrian teve que tomar cuidado para desenhar na mesma resolução em que o jogo rodaria (320x200).

Adrian e Kev no bot hw orked dirExatamente na Deluxe Paint, não tínhamos nenhuma. scanni ng para ols um t tempo.

John Carmack - Programador

Os recursos gráficos estão divididos em duas categorias:

- Itens do menu 2D **incluídos** no pacote do jogoVGAGRAPH, VGAHEAD,eVGADICT
- Itens da fase de ação 3D (paredes e sprites) enviados noVSWAParquivo

3.4 Fluxo de Trabalho de Ativos

Após os recursos gráficos, Após a regeneração, uma ferramenta (IGRAB) agrupou todos os ILBMs em um arquivo de arquivamos e geramos um arquivo cabeçalho com IDs de recursos. O mecanismo faz referência a um recurso. C diretamente usando esses IDs.

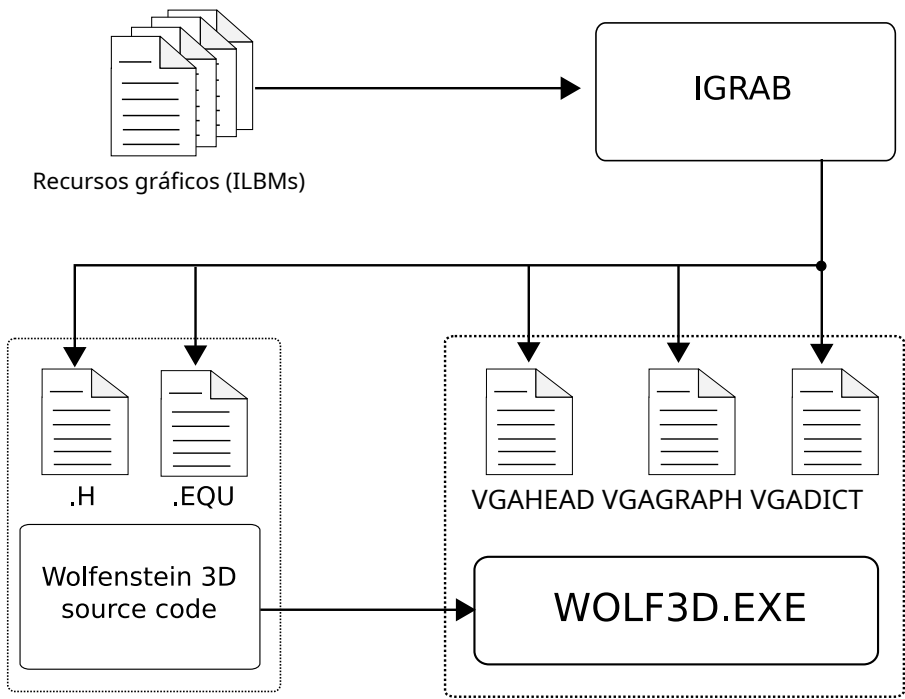


Figura 3.9:Fluxo de trabalho para criação de recursos para itens de menu 2D

```
// ////////////////////////////////////////

// Arquivo gráfico .H para .WL1
// IGRAB -ed em Dom 03 Mai 01:19:32 1992 //

// ////////////////////////////////////////

typedef      enumeração {
    // Carço Começar
    H_BJPIC=3,
    H_CASTLEPIC,           // 4
    H_KEYBOARDPIC,        // 5
    H_JOYPIC,              // 6
    H_HEALPIC,             // 7
    H_TREASUREPIC,        // 8
    H_GUNPIC,              // 9
    H_KEYPIC,              // 10
    H_BLAZEPIC,            // 11
    H_WEAPON1234PIC,      // 12
    H_WOLFLOGOPIC,        // 13
    ...
    PAUSEDPIC,             // 140
    GETPSYCHEDPIC,        // 141
}
```

No código do mecanismo, o uso de recursos é definido de forma fixa por meio de uma **enumeração**. Essa enumeração é um **deslocamento** para oCABEÇAtabela que fornece um deslocamento noDADOSarquivo. Com essa camada de indireção, os **recursos** poderiam ser **regenerados** e **reordenados** à vontade, sem qualquer modificação no **código-fonte**.

ODICTO arquivo contém a árvore de Huffman para descompactar os recursos; isso é discutido mais detalhadamente na seção Gerenciador de Cache, na página 118.

```
vazioCheckKeys (vazio) {
    se(Pausado) {
        LatchDrawPic (20-4,80-2*8, PAUSEDPIC); ...
    }
}

vazioPré-carregar gráficos(vazio) {
    LatchDrawPic (20-14,80-3*8, GETPSYCHEDPIC); ...
}
```

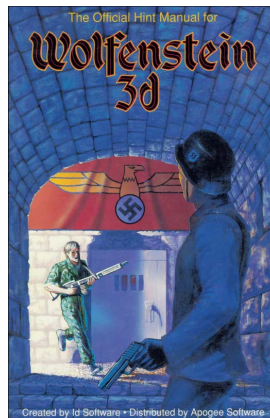
Curiosidades: Esse sistema causou **problemas** quando o código-fonte foi **liberado**. O .hOs arquivos de cabeçalho fornecidos não correspondiam aos **arquivos de recursos** das versões shareware ou das primeiras versões de Wolfenstein 3D. Os cabeçalhos liberados eram da Spear of Destiny. Você pode ver a bagunça gráfica que isso causou no artigo "Vamos compilar como se fosse 1992" em [\[link para o artigo\].fabiensanglard.net](http://link para o artigo.fabiensanglard.net).

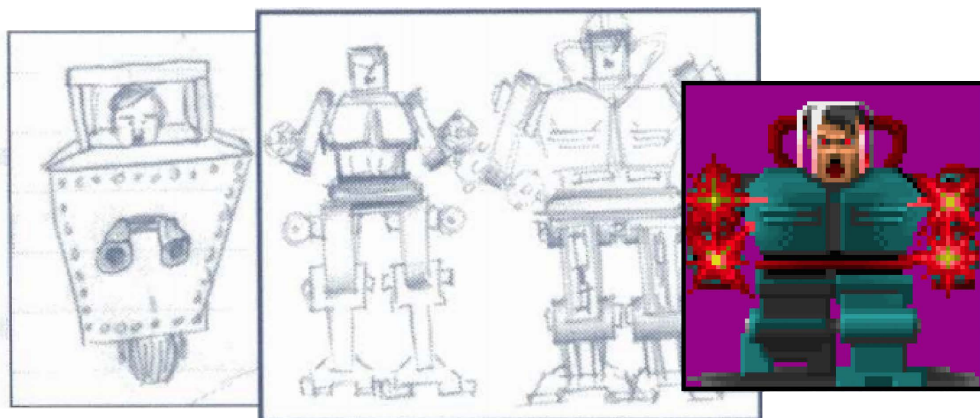
O "Manual Oficial de Dicas para Wolfenstein 3D", publicado em 1992, explica o processo criativo. Contém muitos desenhos de Tom Hall e mostra vários esboços de bastidores feitos por ele e pixelados pela **equipe gráfica**.

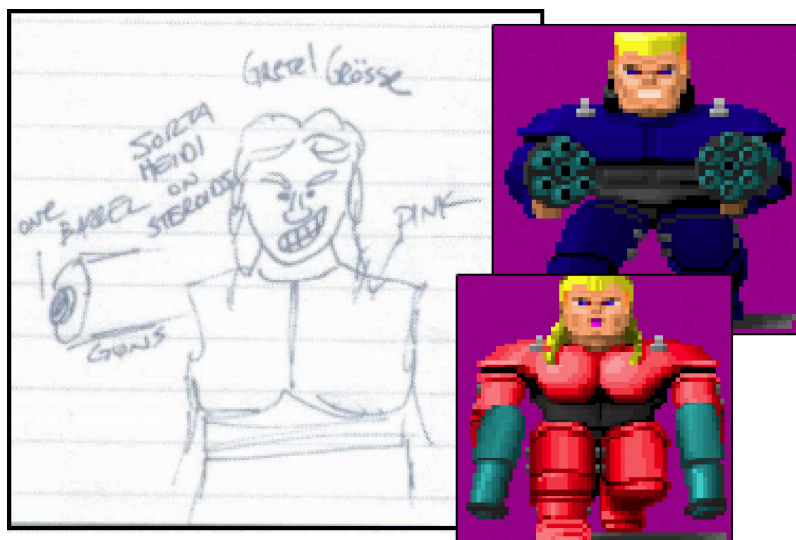
//

Quando o diretor criativo da Id, Tom Hall, tem uma ideia para uma tela, ele fornece um esboço para Adrian Carmack. Abaixo estão alguns dos primeiros designs de Tom para a tela de título. O terceiro esboço foi o escolhido.

//









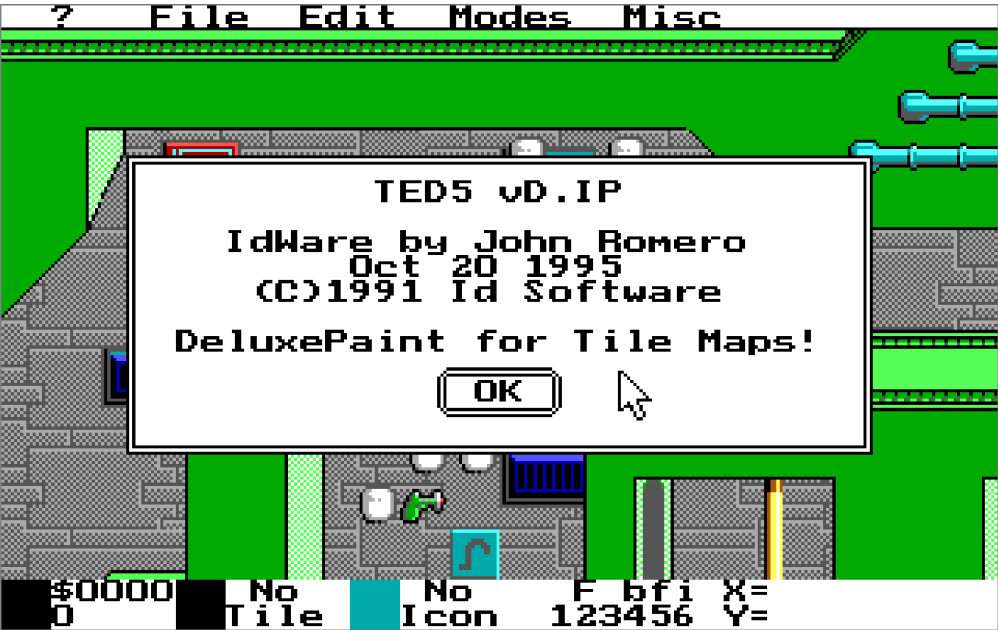
O manual de dicas também contém várias fotos da equipe antigamente; vale a pena lê-lo para entender o contexto.¹⁶

3.5 Mapas

Os mapas foram criados usando um **editor interno** chamado TED5, abreviação de Tile Editor. O TED5 não foi criado especificamente para Wolf 3D, mas foi originalmente desenvolvido para a série Commander Keen e aprimorado ao longo dos anos. Era uma ferramenta versátil, pois permitia a criação de mapas tanto para jogos de plataforma quanto para jogos com visão superior, como Rescue Rover e Wolf 3D.

O TED5 não é independente; para iniciar, ele precisa de um **arquivo de recursos** e do **cabeçalho** associado (conforme descrito na Figura 3.9 do fluxo de trabalho de recursos gráficos, na página 85). Dessa forma, os IDs de textura são codificados diretamente no mapa.

¹⁶O manual foi desenvolvido em uma NeXT ColorStation. Essa foi a única vez que a máquina de Steve Jobs foi usada para Wolfenstein 3D, embora a **id Software** a tenha comprado em dezembro de 1991. A série de estações de trabalho da NeXT tornou-se posteriormente um elemento fundamental no processo de produção de **Doom** em 1993.



Curiosidades: O sufixo "vD.IP" foi adicionado pela equipe do projeto Rise of the Triad em 1994. Ele significa "Developers of Incredible Power" (Desenvolvedores de Poder Incrível).





O TED5 permite a colocação de tiles em camadas chamadas "planos". Essa abordagem em **camadas** provou ser **poderosa** e **versátil**. Em **Commander Keen**, as camadas são usadas para o plano de **fundo**, para os tiles onde o herói pode ficar, para gerar bônus e assim por diante. Em Wolfenstein 3D, **duas** camadas são usadas: uma para as **paredes** e outra para posicionar **bônus** e **inimigos**.

Reutilizar o TED5 foi uma vitória dupla. Não só economizou **tempo de desenvolvimento da ferramenta**, como também reduziu o tempo de **adaptação**, já que todos os membros da equipe o utilizavam há anos. O TED5 era tão eficiente que permitia aos designers criar uma fase em questão de **minutos**.¹⁷

"

Depois de conversar com Romero e Tom, Scott descobriu que o grupo levava apenas um dia para criar uma fase do jogo. Que beleza! Em vez de apenas três episódios, por que não seis? Scott disse: "Se vocês conseguirem fazer mais trinta fases, levariam apenas quinze dias. E poderíamos oferecer a trilogia inclusa por trinta e cinco dólares ou os seis episódios por cinquenta dólares, ou, se alguém comprar o primeiro episódio e depois quiser os outros, custará vinte dólares. Assim, há um bom motivo para comprar todos!" Depois de pensar um pouco, a id concordou.

- Mestres da Perdição

"

John Romero e Tom Hall foram os responsáveis por todo o projeto dos mapas do TED5. Bobby Prince também colaborou e é creditado pelos mapas E6M2 e E6M3.

Curiosidades: O código-fonte do TED5 foi liberado vários anos depois. Entre os componentes do código-fonte estavam...Ce.Hera um misterioso _TOM.PIC¹⁸Descobriu-se que era uma caricatura adulta de Tom Hall feita por Adrian Carmack. A explicação veio de John Romero:

"

"Hahahaha! Nossa, eu tinha me esquecido completamente dessa foto. Não acredito que ela está nos arquivos originais do TED5! É basicamente um desenho que o Adrian fez do Tom [...] dizendo "Desculpe!".

É porque Tom e Adrian costumavam compartilhar uma **mesa de trabalho**. Tom sempre esbarrava na mesa enquanto Adrian desenhava gráficos com o mouse e dizia: "Desculpe!".

John Romero - Programador

"

¹⁷O TED5 foi utilizado em **33** jogos comercializados (incluindo Rise of the Triad em 1994).

¹⁸Proibida a reprodução aqui.

3.6 Áudio

3.6.1 Sons

Conforme mencionado no Capítulo 2.4, o **hardware de áudio** era altamente **fragmentado**. A id Software decidiu oferecer suporte a quatro placas de som e ao alto-falante padrão do PC, o que significava gerar arquivos várias vezes para cada uma e compactá-los com uma ferramenta interna chamada MUSE em um arquivo AU-DIOT (um formato proprietário da id Software):



Figura 3.10: Tela inicial do MUSE.

Três conjuntos de cada efeito sonoro foram incluídos com o jogo:

1. Para alto-falante de PC
2. Para AdLib
3. Para SoundBlaster, SoundBlaster Pro e Disney Sound Source

Todas as vozes foram gravadas por John Romero e Tom Hall, que simularam sotaques alemães.¹⁹o melhor que podiam²⁰.

3.6.2 Música

Toda a composição musical foi feita por Bobby Prince.

“ Nos primórdios das placas de som OPL, o software de sequenciamento considerado o "padrão ouro" era o **Sequencer Plus Gold** ("SPG") da **Voyetra**. Isso se devia ao fato de ele possuir um editor de instrumentos/bancos de instrumentos OPL.

Para esboçar as composições, usei o Cakewalk ("CW"). Já o utilizava há vários anos e estava tudo configurado para usar os equipamentos analógicos como saída de som. Ter sons "reais" desses equipamentos me ajudou a visualizar (audiolizar?) o que eu queria musicalmente. Eu salvava os arquivos do CW no formato *.mid e os carregava no SPG para criar o instrumento OPL para cada **faixa**. Criei bancos de instrumentos diferentes para os diferentes gêneros musicais.

Bobby Prince - Compositor

”

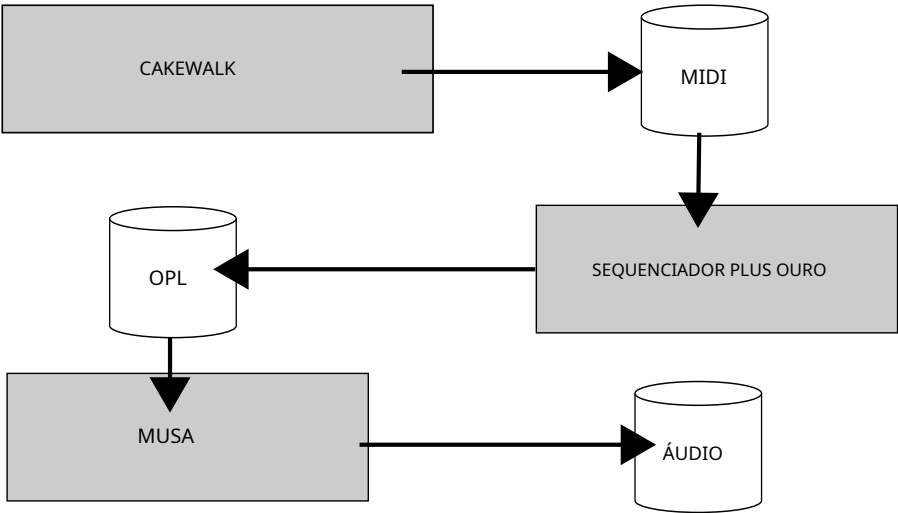


Figura 3.11:Canal de produção musical conforme descrito Por Bobby Prince.

¹⁹Mestres da Perdição, de David Kushner.

²⁰É possível reconhecer as vozes deles se você prestar atenção: o "guten tag" de Tom Hall é memorável.

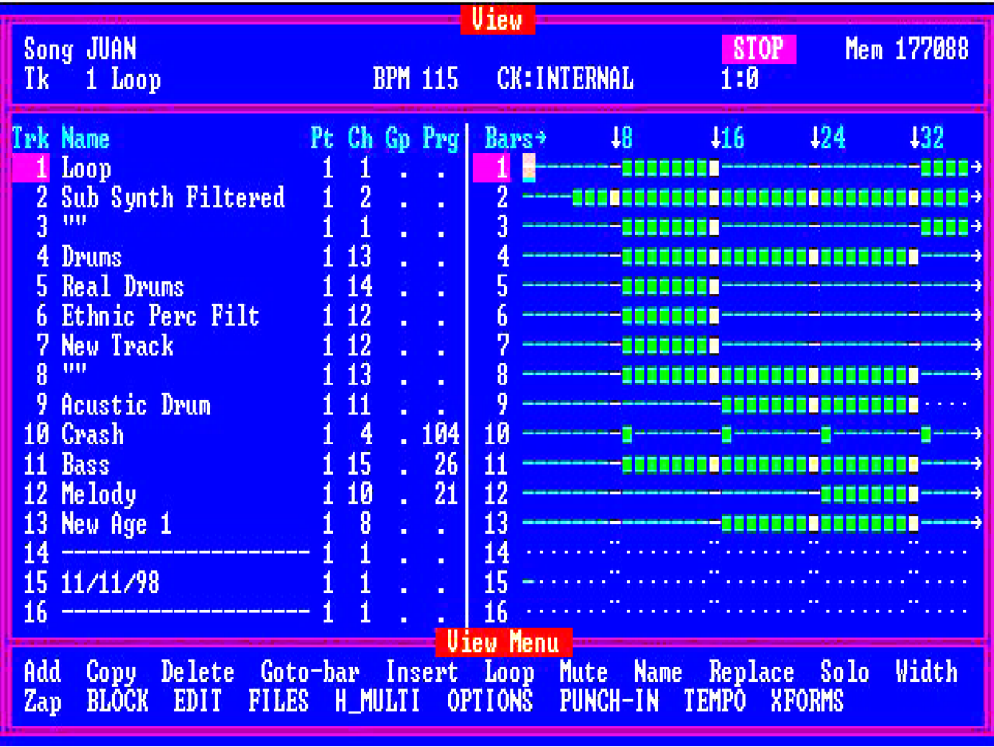


Figura 3.12:Sequencer Plus Gold ("SPG") da Voyetra.

O que vai dentro doÁUDIOO arquivo é um formato de música chamado IMF.²¹Como suporta apenas o YM3812, é **otimizado** para o **chip sem camadas de abstração**. Consiste em um **fluxo de comandos** em **linguagem de máquina** com os respectivos **atrasos**.²²

O **fluxo** controla os **nove canais** do **OPL2**. Um canal é capaz de simular um **instrumento** e reproduzir notas graças a dois **osciladores**, um atuando como **modulador** e o outro como **portadora**. Existem muitas outras maneiras de controlar um **canal**, como envelope, frequência ou oitava.

A forma como um **canal** é programado é descrita em detalhes na Seção 4.8.5.1, "Programação OPL2/YM3812", na página 222.

²¹Arquivo de música do software ID.

²²O formato do FMI é explicado detalhadamente na página 222.

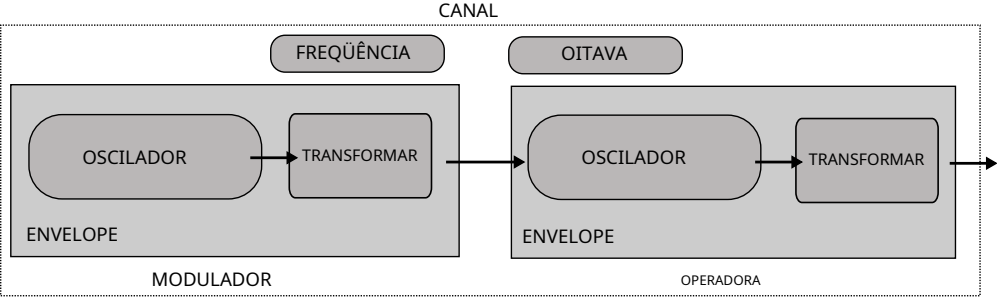


Figura 3.13:Arquitetura de um canal YM3812.

Curiosidades: A **sonoridade inconfundível** do YM3812 deve-se ao seu conjunto peculiar de **transformadores** de forma de onda (localizados logo após a **saída** de cada **oscilador** no diagrama). Quatro formas de onda estão disponíveis no OPL2: Sin 1, Abs-sin 2, Pulse-sin 3 e Half-sin 4.

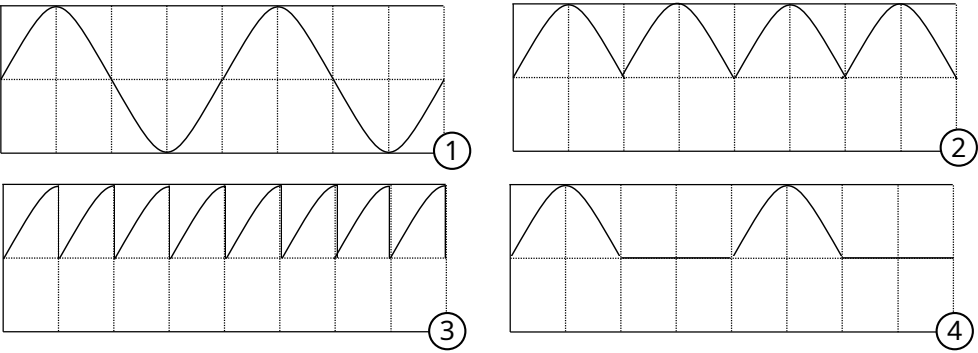


Figura 3.14:As quatro transformações de forma de onda disponíveis.

Curiosidades: No episódio 3, a música que toca contém uma mensagem oculta em código Morse: "Para o Lobo Mau. Para a Chapeuzinho Vermelho. Elimine Hitler. Imperativo. Complete a missão em 24 horas. Câmbio." O chefe final deste episódio é, de fato, Hitler em uma armadura mecânica (veja **dicamanual desenho**ng em p 88 anos) .

3.7 Distribuição

Às 4h da manhã do dia 5 de maio de 1992, o primeiro episódio do jogo foi carregado em Massachusetts. via BBS da Software Creations23servidor. Wolfenstein 3D foi distribuído como shareware; o

²³Buletina Board Sy stemsw ere serv és que um eido USers para connect vi aa cons ole e upload/d download próprio programa ams.

O motor de jogo e o primeiro episódio foram disponibilizados gratuitamente, com incentivo para que fossem copiados e distribuídos ao máximo de pessoas possível. Para obter os outros cinco episódios, cada jogador precisava pagar US\$ 50 à Id Software.

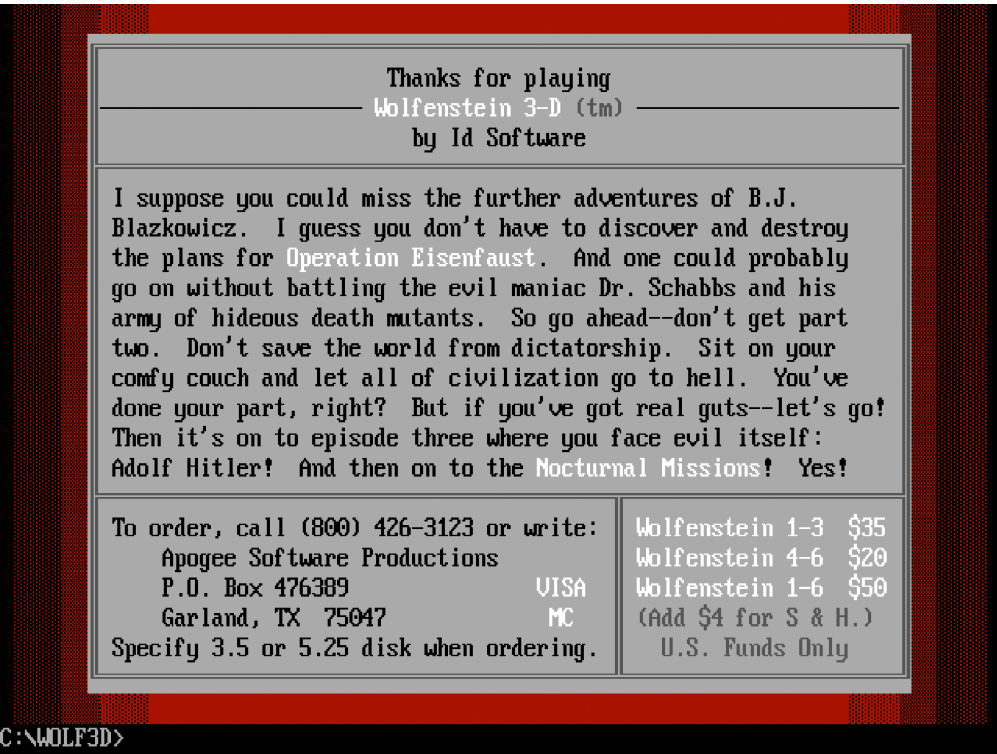


Figura 3.15:Sair do jogo descreve como obter a versão completa.

Para maximizar os lucros, era fundamental que o jogo fosse fácil de copiar e redistribuir. Em 1991, a internet ainda estava em seus primórdios e o melhor meio era o 3 1/2 disquete de 3 polegadas. Foi dada especial atenção ao tamanho total do jogo para que coubesse em um único disquete. Todos os arquivos combinados ocupavam 1.204 KiB, mas tudo foi compactado para 645 KiB (o formato de baixa capacidade comumente usado de 3 polegadas). 1/2 Um disquete de 1/4 de polegada de 1992 podia armazenar 720 KiB. Os seis episódios completos cabiam em dois disquetes. O jogo era distribuído da seguinte forma.

Os arquivos podem ser divididos em cinco partes:

- WOLF3D.EXE: Motor de jogo.
- VSWAP.WL1: Contém todos os recursos (sprites, texturas, sons digitalizados) necessários durante as fases de ação em 3D.

- Arquivos de música e efeitos sonoros utilizados durante as fases 3D e 2D:
 - AUDIOHED.WL1: Índice da carga útil emÁUDIOarquivo.
 - AUDIOT.WL1: Dados de áudio não comprimidos.
- Mapas:
 - MAPHEAD.WL1: Índice emMAPAS DE JOGOarquivo.
 - GAMEMAPS.WL1: Arquivos de mapas compactados.
- Imagens utilizadas durante a fase do menu 2D:
 - VGAHEAD.WL1: Índice emVGAGRAPHarquivo.
 - VGADICT.WL1: Árvore de Huffman para descomprimir cada imagem.
 - VGAGRAPH.WL1: Imagens compactadas agrupadas.

```

C:\WOLF3DU>dir
Directory of C:\WOLF3DU\
.                <DIR>                22-09-2018   9:16
..               <DIR>                22-09-2018   9:17
AUDIOHED WL1      1,156 31-12-1992 23:00
AUDIOT WL1      132,613 31-12-1992 23:00
CONFIG WL1       522 14-09-1994  0:00
FILE_ID DIZ       207 03-12-2011 11:06
GAMEMAPS WL1     27,425 31-12-1992 23:00
MAPHEAD WL1       402 31-12-1992 23:00
VGADICT WL1     1,024 31-12-1992 23:00
VGAGRAPH WL1    326,568 31-12-1992 23:00
VGAHEAD WL1       471 31-12-1992 23:00
USWAP WL1       742,912 31-12-1992 23:00
WOLF3D EXE      109,959 31-12-1992 23:00
    11 File(s)      1,343,259 Bytes.
     2 Dir(s)      262,111,744 Bytes free.

C:\WOLF3DU>_

```

Figura 3.16: Todos os arquivos distribuídos como shareware são exibidos da mesma forma que no prompt de comando do DOS.

Curiosidades: O executável do mecanismo é minúsculo, ocupando apenas 94 KiB em disco, pois está compactado em LZEXE. O tamanho descompactado é de aproximadamente 280 KiB, sendo 64 KiB dedicados ao mecanismo. Entrar tela e 768 bytes para a paleta.

As extensões de arquivo tinham, sim, um significado:

- WL1: Software compartilhado.
- WL3: Versão completa inicial com três episódios.
- WL6: Versão completa com seis episódios.
- WJ1: Software japonês de participação.
- WJ6: Versão completa em japonês.
- SOD: Lança do Destino.

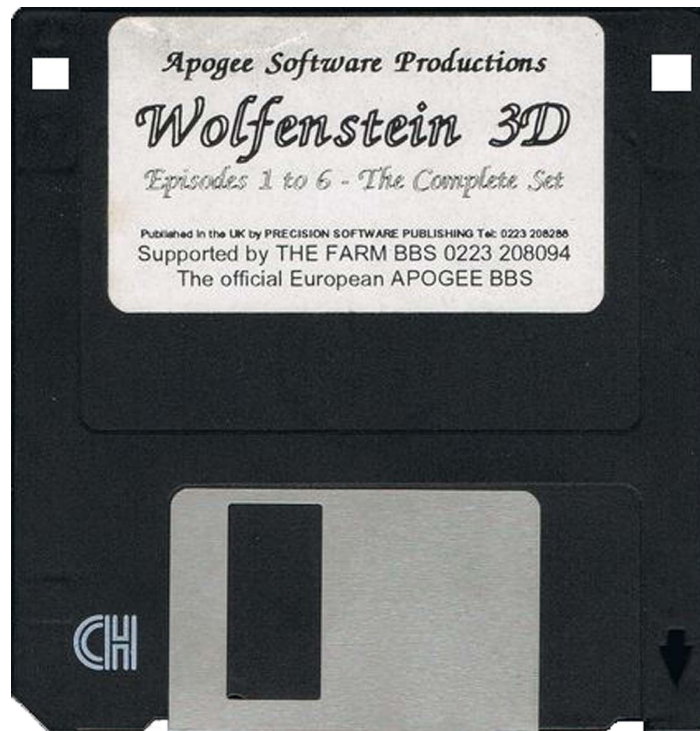


Figura 3.17

A Figura 3.17 mostra um 3 1/2 Disquetes de 1/4 de polegada são amplamente utilizados para **compartilhar** e **redistribuir** jogos. O orifício superior esquerdo permite que o leitor de disquetes reconheça se o disco é de baixa capacidade (cheio = 720 KiB) ou de alta capacidade (com orifício = versão de 1,44 MiB). O orifício superior direito possui uma trava deslizante que, quando aberta, impede a gravação no disco, tornando-o somente leitura. Uma vez inserido no leitor, o disquete permite que o leitor registre se o disco é de baixa capacidade (cheio = 720 KiB) ou de alta capacidade (com orifício = versão de 1,44 MiB).

No leitor de disquetes, a grande aba metálica desliza para a direita para expor o disco magnético.

Curiosidades: Algumas pessoas (incluindo o autor) economizaram dinheiro furando buracos em seus disquetes certificados de 720 KiB para que fossem reconhecidos como 1,44 MiB. Funcionou!

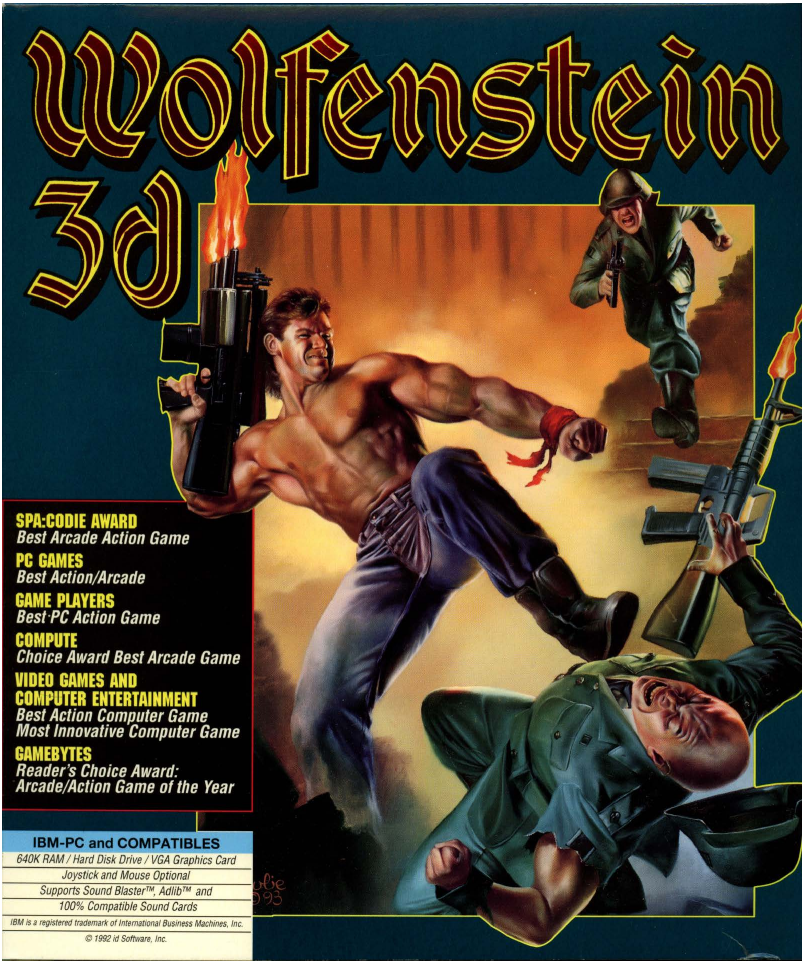


Figura 3.18: Wolfenstein 3D em formato físico, como um jogo de tabuleiro.

Embora tenha sido originalmente distribuído exclusivamente como **shareware**, as vendas posteriormente exigiram uma mídia mais "tradicional". A **GT Interactive** adquiriu **Wolfenstein 3D** em **1993** e o presenteou com uma caixa física, juntamente com o "Manual Oficial de Dicas", elaborado por Tom Hall e Kevin Cloud. Em **1995**, o jogo foi relançado em CD-ROM.

Capítulo 4

Software

4.1 Obtendo o código-fonte

O código-fonte do motor de jogo foi carregado no FTP da id Software.¹servidor em 21 de julho de 1995.

```
ftp://ftp.idsoftware.com/idstuff/source/wolfsrc.zip
```

Mais de vinte e dois anos depois, o arquivo ainda está localizado exatamente no mesmo endereço URL, um fato notável dada a natureza em constante mudança da web.

Alternativamente, você pode usargithub.comonde a id Software migrou todo o seu código de código aberto por volta de 2012. Não é apenas mais rápido, como também mais **confiável**.

```
$ git clone git@github.com :id-Software/wolf3d.git
```

4.2 Primeiro Contato

Após o download e a descompactação, o arquivowolfsrc.zipContém outro arquivo PKZIP autoextraível. Era uma conveniência antigamente, mas não é prático hoje em dia e é fácil de descompactar.

```
$unzip WOLF3D .1
```

cloc.plÉ uma ferramenta que analisa todos os arquivos em uma pasta e coleta estatísticas sobre o código-fonte. Ela ajuda a ter uma ideia do que esperar.

¹Protocolo de Transferência de Arquivos.

\$ cloc -1.64. pl WOLF3D

96 arquivos de texto.
94 exclusivo arquivos.
27 arquivos ignorado.

----- Linguagem				
	arquivos	em branco	comentário	código
----- C++				
	26	5750	6201	21169
C/C++ Cabeçalho	42	802	660	3900
Conjunto	10	669	732	2150
Lote DOS	1	1	0	4
----- SOMA:				
	79	7222	7593	27223

O código é 90% em C com código assembly.²para otimizações de gargalo e E/S de baixo nível, como vídeo ou áudio.³

O número de linhas de código (SLOC, na sigla em inglês) não é uma métrica significativa para uma base de código individual, mas se destaca na extração de proporções. Wolfenstein 3D, com suas 27.223 SLOC, é muito pequeno em comparação com a maioria dos softwares.curvar (Uma ferramenta de linha de comando para baixar conteúdo de URLs tem 154.134 linhas de código. O navegador Chrome do Google tem 1.700.000 linhas de código. O kernel do Linux tem 15.000.000 linhas de código.

A divulgação do código-fonte foi extremamente bem recebida, e em meio aos muitos elogios, alguns comentários extraordinários deixaram lembranças inesquecíveis.

“

Naquela época, nossos editores não tinham corretores ortográficos, e eu sempre errava muito na ortografia. A palavra "coluna" aparece dezenas de vezes no código-fonte. Depois que publiquei o código-fonte, um dos e-mails que me marcou dizia:

É "COLUNA", seu idiota!

John Carmack - Programador

”

²Toda a montagem em Wolf3D é feita com TASM (também conhecido como Turbo Assembler da Borland). Ele usa a **notação Intel**, onde o **destino** vem antes da origem:fonte de destino instr.

³A id Software só passou a usar C++ a partir de Doom 3, por volta do ano 2000.

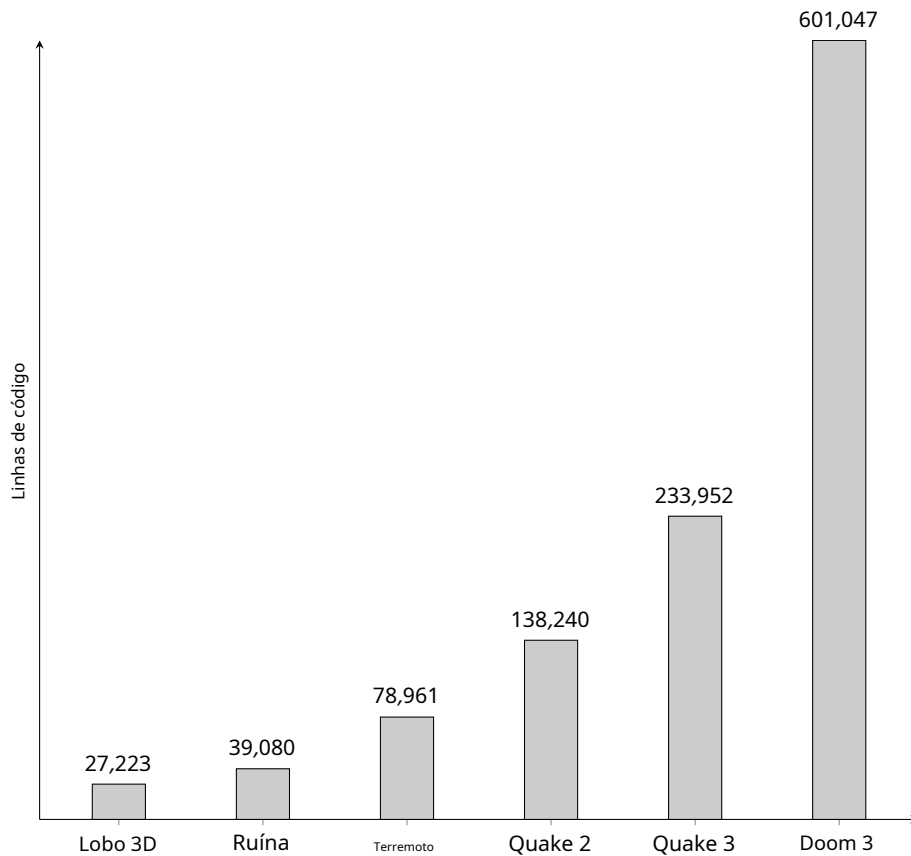


Figura 4.1: Linhas de código de motores de jogos da id Software.

O arquivo contém mais do que apenas o código-fonte; ele também **inclui**:

- GOODSTUF.TXT: Dois e-mails de fãs (um ex-prisioneiro de guerra e um funcionário da Microsoft) demonstrando o sucesso do jogo.
- SIGNON.OBJ: A tela de inicialização que exibia as características do sistema (RAM, EMS, XMS, Joystick, Placas de Som) estava vinculada ao binário. Essa escolha de design peculiar será explicada mais adiante.
- GAMEPAL.OBJ: Paleta de cores do jogo. Codificada e vinculada no executável para o mesmo. razão como SIGNON.OBJ.
- LEIA-ME: Como compilar. Você também pode encontrar um tutorial completo em "Vamos compilar como se fosse...". 1992" emfabiansanglard.net.
- Vários arquivos resultantes de uma tentativa de compilação anterior.

4.3 Visão Geral

O motor do jogo está dividido em três blocos:

- Motor de menu 2D que permite aos usuários configurar o jogo.
- Renderizador de jogos 3D onde os usuários passam a maior parte do tempo.
- Sistema de som que funciona simultaneamente com o renderizador 2D ou 3D.

Os três sistemas comunicam-se através de **memória compartilhada**. O **renderizador** grava solicitações de **música** e **som** na **RAM** (garantindo também que os recursos estejam prontos). Essas solicitações são lidas pelo "loop" de som. O sistema de som também grava na **RAM** para os renderizadores, já que é responsável pelo **pulso** de todo o mecanismo. Os **renderizadores** atualizam o mundo de acordo com o tempo real rastreado por **Contagem de tempo** de temporizável.

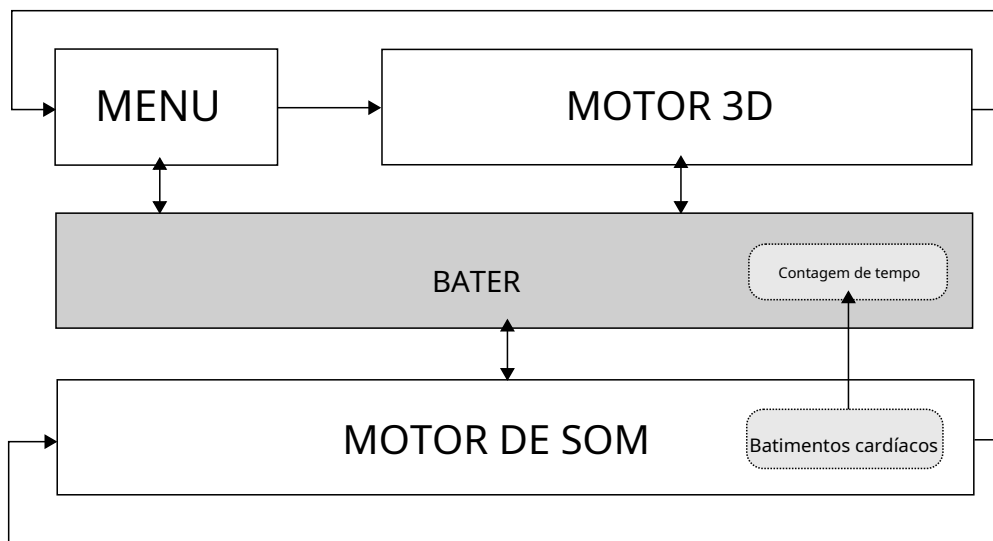


Figura 4.2: O motor de jogo possui três sistemas principais.

4.3.1 Laço Desenrolado

Com a **visão geral** em mente, podemos mergulhar no **código** e desdobrar o **loop principal** a partir de **void main()**. Os dois **renderizadores** são **loops regulares**, mas devido a limitações explicadas posteriormente, o sistema de som é controlado por **interrupção** e, portanto, está fora de controle principal. Devido ao modo real, C os tipos não significam o que as pessoas esperariam de uma arquitetura de 32 bits.

- inteiropalavra de 16 bits.
- longoepalavra de 32 bits.

A primeira coisa que o programa faz é verificar os **recursos** disponíveis através de `VerificarEpisódios()`.

```
vazioprincipal (vazio) {
    VerificarEpisódios();
    Patch386    ();
    InitGame    ();
    DemoLoop();
}
```

Como a máquina alvo operava em modo real, o código foi compilado usando apenas instruções de **16 bits**. Para operações em longo (Em sistemas de 32 bits, a Borland utilizava sua própria biblioteca matemática. `Patch386` Wolfenstein 3D detecta se a CPU é uma **386** e modifica seu próprio **código** para substituir a divisão integral da Borland por instruções que utilizam registradores de 32 bits. `eaxedx`).

```
movimento eax,[bp+8]
cdq
idiv[DWORD PTR bp+12]
movimento eax,edx
shr  edx,16
```

`EmInitGame`, O motor liga e exibe todos os gerentes.

```
vazio InitGame () {
    MM_Startup ();           // Gerenciador de memória

    SignonScreen();         // Exibir configuração do sistema

    VW_Startup  ();          // Vídeo   Gerente
    IN_Startup  ();          // Entrada Gerente
    PM_Startup  ();          // Página Gerente
    PM_DesbloquearMemóriaPrincipal ();
    SD_Inicialização ();     // Som    gerente
    CA_Startup  ();          // Cache gerente
    Startup dos EUA ();      // Fonte gerente
    InitDigiMap  ();
    ReadConfig  ();
    CA_CacheGrChunk(STARTFONT); // Carregar fonte MM_SetLock (&
    grsegs[STARTFONT],verdadeiro); // Fonte de bloqueio
    CarregarMemLatch();       // Carregar recurso de imagem na VRAM //
    CriarTabelas ();          // Tabelas de consulta seno/cosseno/vista //
    ConfigurarParedes ();     // Texturas de parede da tabela de consulta

}
```

Em seguida, vem o loop principal, onde o renderizador 2D e o renderizador 3D são chamados **indefinidamente**.

```

vazioDemoLoop () {
  IniciarCPMusic(INTROSONG);
  PG13();// Exibir tela de Carnificina Profunda enquanto
  (1) {
    CA_CacheScreen      (TÍTULO FOTOGRÁFICO);
    CA_CacheScreen      (CRÉDITOS DA IMAGEM);
    DrawHighScores      ();
    PlayDemo (0);
    GameLoop();          // Renderizador 2D (menu)
    ConfigurarNívelDoJogo ();
    IniciarMúsica ();
    PM_VerificarMemóriaPrincipal ();
    Pré-carregar gráficos ();
    DesenharNível();
    PlayLoop () ;// Renderizador 3D (ação) StopMusic ();

  }
  Desistir("O loop de demonstração foi encerrado ???");
}

```

Reproduzir em loopContém o renderizador 3D. É bastante padrão, com a abordagem de obter **entradas**, atualizar o **mundo** e **renderizar** o mundo.

```

vazioPlayLoop () {
  PollControls ();          // Obter entrada do jogador

  MoveDoors();              // Mover portas
  MovePWalls ();            // Mover parede secreta
  para(obj = jogador; obj; obj = obj -> próximo)
    DoActor (obj);          // Os inimigos pensam

  ThreeDRefresh () {        // Renderizar visualização 3D
    VGAClearScreen();        // Empate  piso/teto
    WallRefresh();           // Empate  paredes
    DesenharEscalas ();      // desenhar coisas em
    DesenharArmaDoJogador(); escala // desenhar arma
    [...]                    // Inverter framebuffer através do controlador CRT
  }
  AtualizarLocalizaçãoDoSom(); // Localização de som estéreo
}

```

O sistema de som é iniciado através do Gerenciador de Som emSDL_SetTimerSpeed.Embora exista uma **biblioteca** famosa para desenvolvimento de jogos chamada **Simple DirectMedia Layer** (SDL), o prefixo SDL_Não tem nada a ver com isso. Significa **Sound Low level** (Simple DirectMedia Layer).

nem sequer existia em 1991).

A razão para as interrupções é explicada detalhadamente no Capítulo 4.8 "Áudio e Batimento Cardíaco". Resumidamente, com um sistema operacional que não suporta processos nem threads, essa era a única maneira de executar algo simultaneamente com o restante do mecanismo.

Uma **rotina de serviço de interrupção** (ISR, na sigla em inglês) é instalada na tabela de vetores de interrupção para responder a interrupções disparadas pelo mecanismo. Observe como a ISR pode ser chamada em frequências de 140 Hz, 700 Hz ou até mesmo 7000 Hz, dependendo das necessidades do sistema de som.

```
# definir TickBase 70

typedef enum{
    sds_Off,
    sds_PC,
    sds_SoundSource,
    sds_SoundBlaster
} Modo SDS;

externo    Modo SDS    DigiMode;

vazio estático SDL_SetTimerSpeed(vazio) {
    palavra    avaliar;
    vazio    interromper    (*isr)(vazio);

    se(( DigiMode == sds_PC) && DigiPlaying) {
        taxa = TickBase * 100;           // 7000 Hz
        isr = SDL_t0ExtremeAsmService;
    }
    senão se(música | | (( DigiMode == sds_SoundSource))
        && DigiPlaying) {
        taxa = TickBase * 10; isr        // 700 Hz
            = SDL_t0FastAsmService;
    }
    outro {
        taxa = TickBase * 2; isr =        // 140 Hz
            SDL_t0SlowAsmService;
    }
    setvect(8,isr);
    SDL_SetIntsPerSec(taxa);
}
```

4.4 Arquitetura

O código-fonte está estruturado em duas camadas. Os arquivos WL_* são camadas de alto nível que **dependem** de subsistemas ID_* de baixo nível, chamados **Gerenciadores**, que interagem com o **hardware**.

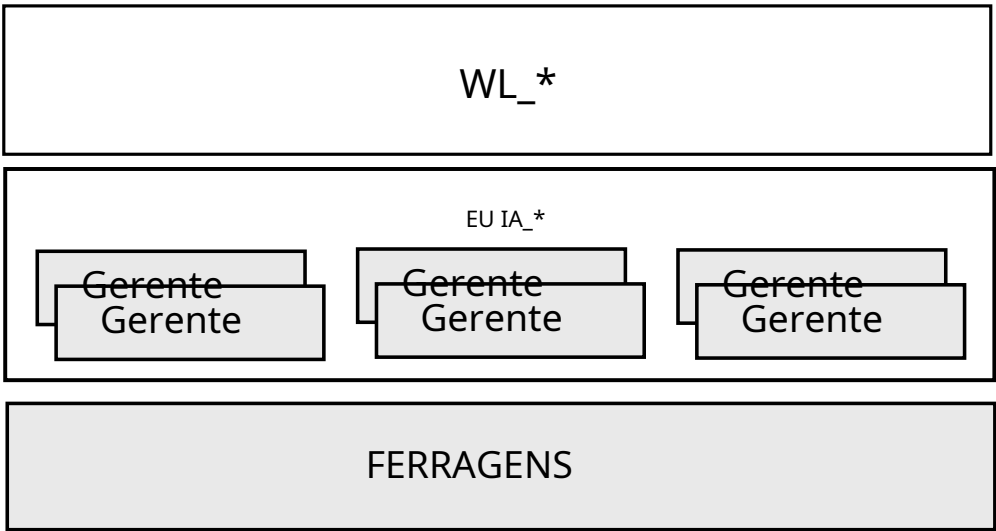


Figura 4.3:Camadas do código-fonte de Wolfenstein 3D.

Há sete gerentes no total:

- Memória
- Página
- Vídeo
- Cache
- Som
- Usuário
- Entrada

O **conteúdo** do WL_ foi escrito especificamente aliado para Wolf3D enquanto os gerenciadores de ID_ foram reutilizados com base em jogos anteriores (Hovertank O). (ne e Catacomb 3-D) e aprimorados para as necessidades de o novo motor.

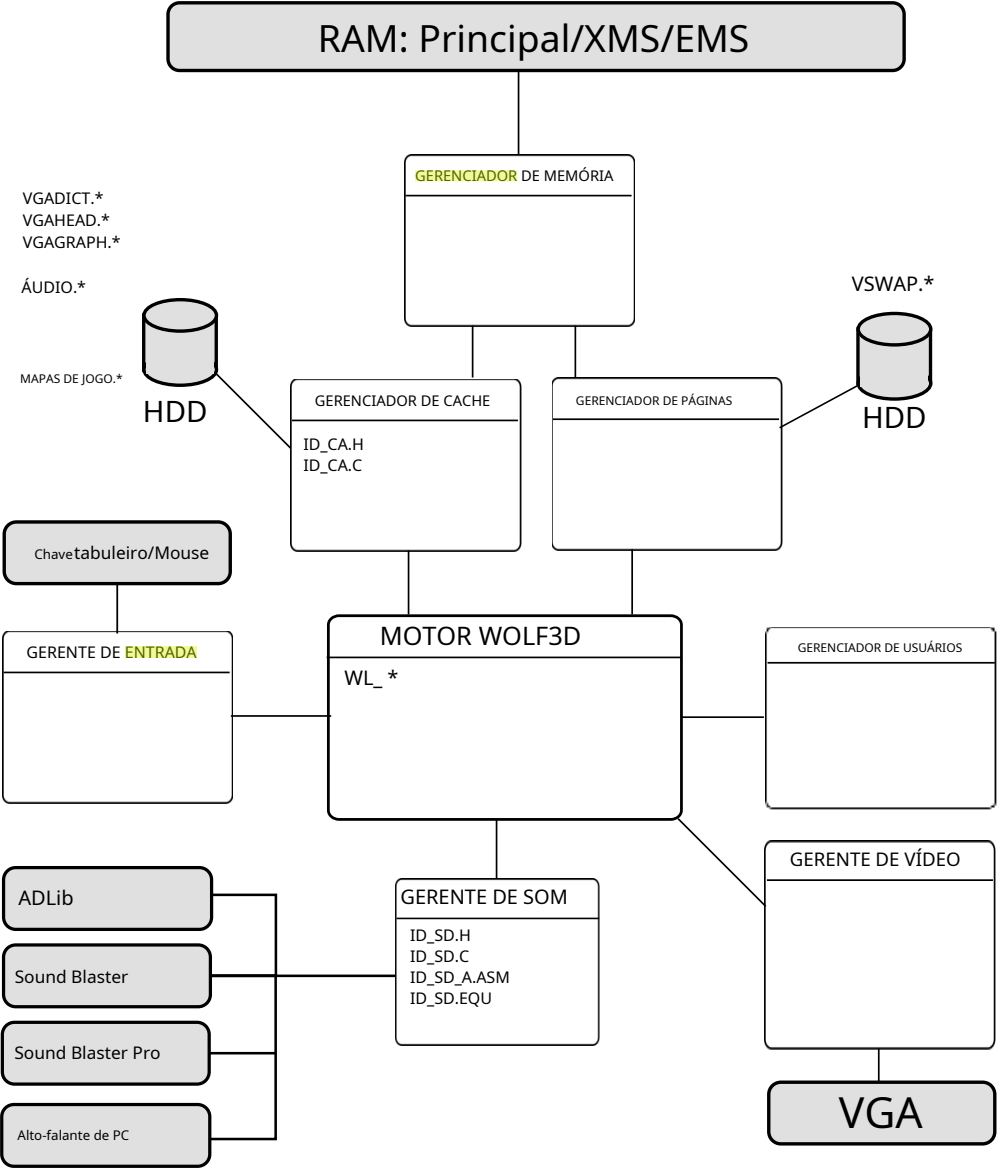


Figura 4.4:Arquitetura com o motor e subsistemas (em branco) conectados às entradas/saídas (em cinza).

Ao lado dos discos rígidos (HDDs), você pode ver os arquivos compactados conforme descrito no Capítulo 3. Equipe.

4.4.1 Gerenciador de Memória (MM)

O motor não depende de malloc para gerenciar a memória convencional, que pode levar à **fragmentação da memória** e à impossibilidade de **compactar o espaço livre**, o sistema possui seu próprio **gerenciador de memória**, composto por uma **lista encadeada** de "blocos" que controlam a **RAM**. Um bloco aponta para um ponto **inicial** na **RAM** e possui um tamanho **definido**.

```
typedef struct      mmblockstruct
{
    sem assinatura   início, comprimento;
    sem assinatura   atributos;
    memptr          * useptr;
    estrutura mmblockstruct far *next; }
mmblocktype;
```

Um bloco pode ser marcado com atributos:

- LOCKBIT: Este bloco de RAM não pode ser movido durante a compactação.
- PURGEBITS: Quatro níveis disponíveis: 0 = não **purgável**, 1 = **purgável**, 2 = não **utilizado**, 3 = purgar primeiro.

O gerenciador de memória começa alocando toda a RAM disponível através de malloc/farmalloc cria um **TRANCA** de tamanho 1 KiB no final. A lista encadeada usa dois ponteiros: **CABEÇA** e **ROVER** que apontam para o **penúltimo bloco**.

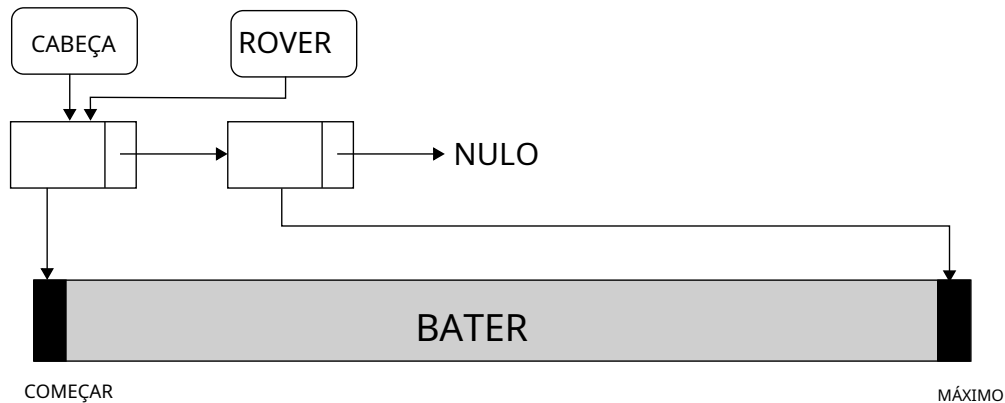


Figure 4.5: Estado inicial do gerenciador de memória.

O motor interage com o Gerenciador de Memória solicitando RAM (**MM_GetPtr**) e liberando RAM (**MM_FreePtr**). Para alocar memória, o gerenciador procura por "espaços livres" entre os blocos. Isso pode levar até três passagens com complexidade **crescente**:

1. Depois do rover.
2. Depois da cabeça.
3. Compactação e depois passagem pelo rover.

O caso mais simples é quando há espaço **suficiente** após o **rover**. Um novo nó é simplesmente adicionado à lista encadeada e o rover **avança**. No próximo desenho, três solicitações de alocação foram bem-sucedidas: A, B e C.

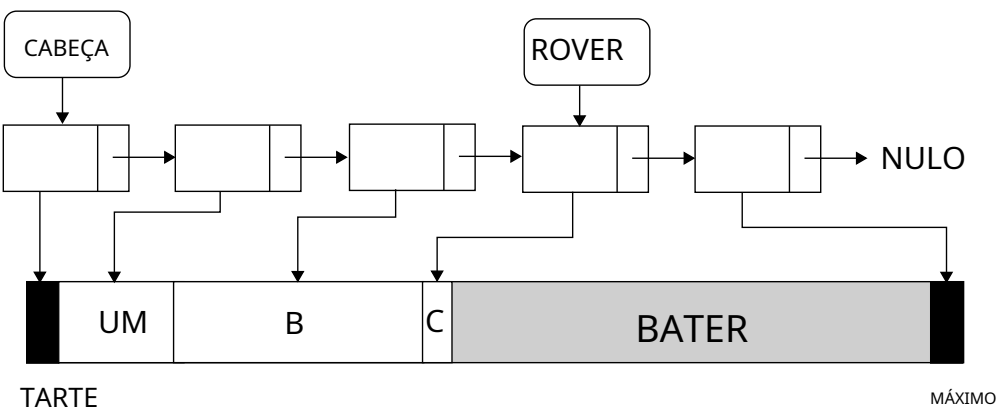


Figura 4.6:Estado interno do MM após três alocações de passagem 1.

Eventualmente, a **memória RAM livre** se esgotará e a primeira tentativa falhará.

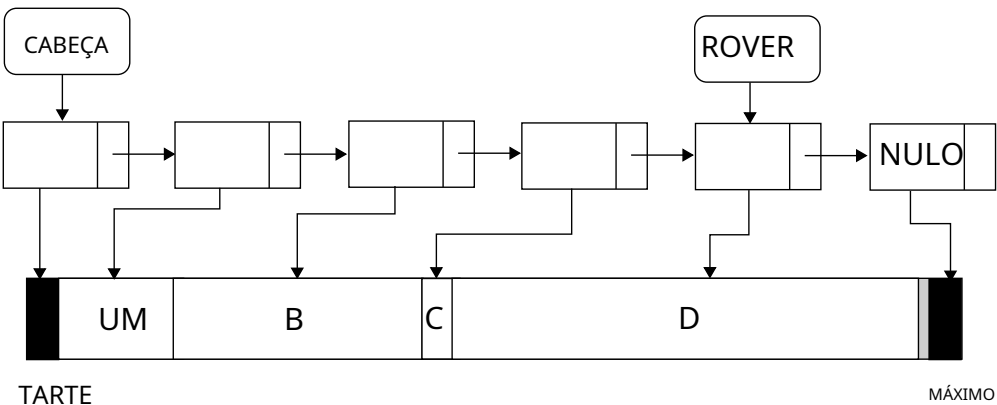


Figura 4.7:Falha na primeira passagem: RAM insuficiente após o ROVER.

Se a primeira passagem falhar, a segunda procura por uma "brecha" entre a cabeça e o rover. Esta passagem também **removerá blocos não utilizados**. Se, por exemplo, o bloco B estivesse marcado como PURÍVEL, Ele será excluído e substituído pelo novo bloco E. Nesse ponto, a fragmentação começa a aparecer (como se...).mallocfoi usado).

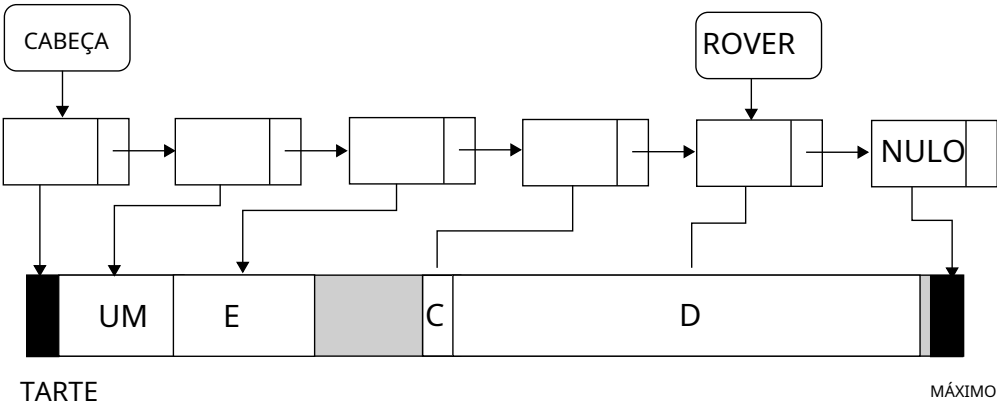


Figura 4.8:Bfoi expurgado.Efoi alocado na passagem 2.

Se a primeira e a segunda tentativas falharem, não há nenhum bloco contínuo de memória grande o suficiente para atender à solicitação. O **gerenciador** então percorrerá toda a lista encadeada e fará duas coisas: excluirá os blocos marcados como **removíveis** e **compactará a RAM** movendo os blocos.

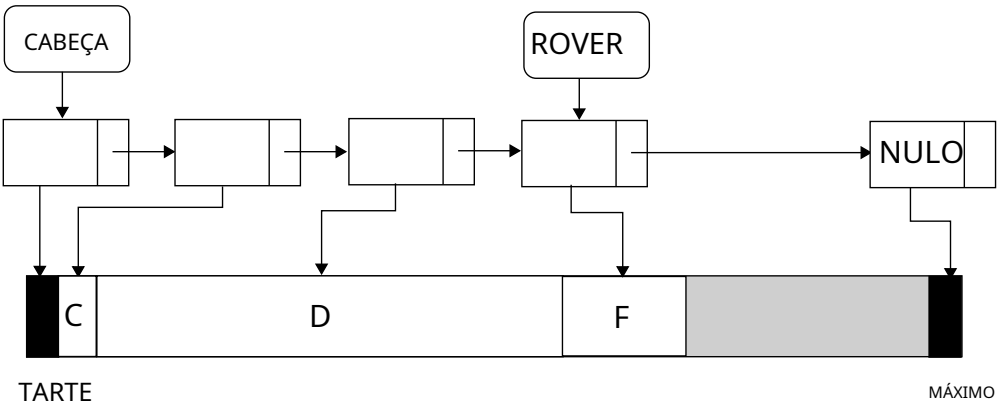


Figura 4.9:UMeEforam expurgados.CeDcompactado.Falocado na passagem 3.

Mas se a memória for **realocada**, como as alocações **anteriores** ainda **apontam** para o que faziam antes da fase de **compactação**? Observe queum mmblockstructtem umuseptrpoeu nter qual

Aponta para o **proprietário** de um **bloco**. Quando a memória é movida, o proprietário do bloco também é atualizado.

Como alguns blocos estão marcados como **TRANCADO**, a compactação pode ser **interrompida**. Ao encontrar um bloco **travado**, a compactação **para** e o **próximo** bloco será movido imediatamente após o bloco travado, mesmo que haja espaço disponível entre o último bloco e o bloco travado.

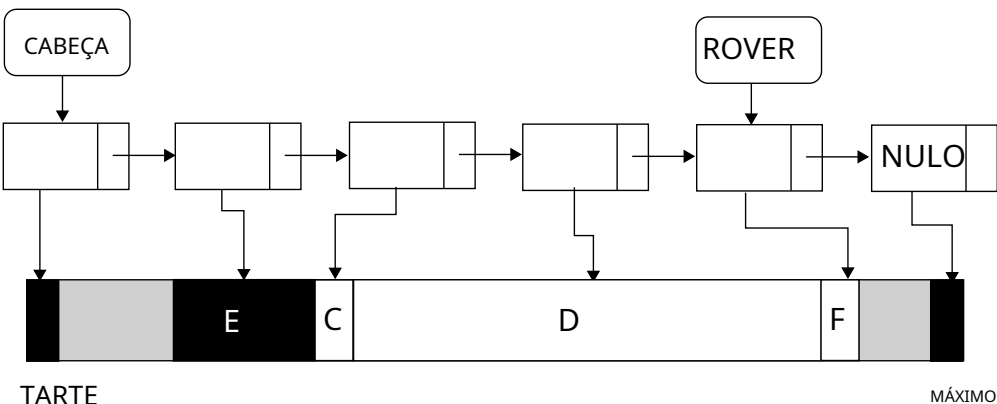


Figura 4.10: E está travado e não pode ser compactado.

No desenho acima, o componente C foi movido para depois do componente E, embora pudesse ter sido movido para antes. Evitar esse **desperdício** tornaria o gerenciador de memória mais complexo, portanto, o desperdício foi considerado **aceitável**. Frequentemente, ao projetar um **componente**, é preciso ser **prático** e estabelecer um certo **equilíbrio** entre **precisão** e **complexidade**.

Um **gerenciador de memória dedicado** provavelmente se **justificava** para o **Wolf**, mas eles são um enorme fonte de **bugs**, e eu imploro às pessoas que não façam isso!

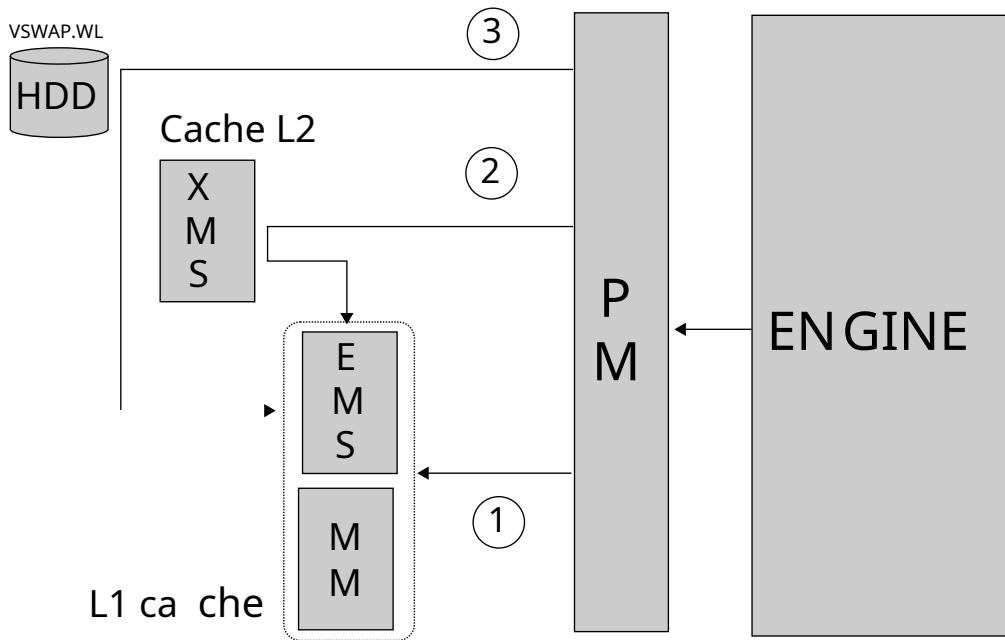
John Carmack - Programador

4.4.2 Página Manager (PM)

O **Página Manager** é de **ditado** para o **3D e nGine**. Seu **task** é para fazer **e** sobre **e** avaliação **tais como** **imagens, sprites, som** **efeitos de armazenamento** **vestir** **HDD** **a** **estão disponíveis** **em** **RAM** **para** **a** **CPU** **para** **uso**. **Jason Blochowiak** parece ter sido o principal autor e sua experiência anterior **A** **influência** do **Unix** foi evidente no design deste componente. Ele é baseado nos conceitos de **paginação** e **troca de memória**.

Em vez de usar um endereço de memória para identificar uma página, como no Unix, utiliza-se um ID de recurso. Esses IDs são gerados pelo IGRAB. Cada recurso consome uma "página" inteira. Assim como no Unix, todas as páginas têm o mesmo tamanho, 4 KiB. Quando o mecanismo precisa de um recurso, ele solicita uma página com o ID de recurso ao Gerenciador de Páginas. Todos os tipos de RAM (convencional, EMS e XMS) são utilizados, mas existe uma hierarquia.

Originalmente, todos os recursos para sequências 3D estão no disco rígido em formato de arquivo.VSWAP.WL1. Quando uma solicitação de um recurso é recebida, o cache L1 (composto por RAM convencional e EMS) é consultado primeiro¹. Em caso de falha, o cache L2 é consultado². Se a página for encontrada lá, ela é transferida para o L1. Se a página ainda não for encontrada no L2, ela é carregada diretamente do disco rígido para o L1³. O L2 só é gravado quando uma página é removida do L1. Cada vez que uma página é acessada, ela também é marcada com o número do quadro atual. Essa marcação é usada para aplicar a política de remoção.



A arquitetura do nível de Gerenciamento de páginas É interessante, pois se comporta ótimas XMS a sa último reso rt cache L2 enquanto EM A RAM RAM é para nós como uma memória RAM Ory. Isto é porque EM S acionada é severa eral ti mes mais rápido que convencional controlada por XMS e quase tão f o mais rápido possível n- memória cional. Este tópico c é detalhado i está no Apêndice B, página 29. 7.

Para minimizar o custo da página falhas, o motor pré-lo anúncios a página e cache antes ré Um nível começa. O usuário experimenta isso como o "Prepare-se para a adrenalina". tela:



Figura 4.11: O "termômetro" mostra o processo de pré-carregamento do Gerenciador de Páginas.

O mecanismo de pré-carregamento não é particularmente **inteligente**: ele carrega o máximo de páginas possível do arquivo de troca. Ele não tenta analisar o que é realmente usado na fase, mas sim carrega os recursos na ordem em que estão armazenados. VSWAP arquivo no disco rígido. Em uma máquina com pouca memória (menos de 1 MB), a **política de remoção** (LRU) estabiliza o **cache** após alguns minutos em um nível.

Este design tem uma pequena, porém irritante, falha. Em máquinas com pouca memória, ocorrerá um erro de cache quando o jogador abrir a última porta de uma fase e estiver prestes a enfrentar o poderoso chefe final. Este inimigo nunca foi visto antes e, portanto, não está no cache do Gerenciador de Páginas. Isso acarreta o pior caso possível de erro de cache: um longo acesso ao disco rígido é necessário, causando lentidão e, frequentemente, resultando em uma morte injusta (e humilhante).

O tamanho do arquivo de troca varia dependendo da versão do jogo que está sendo executada: VSWAP.WL1 (O shareware) tem 742 KiB enquanto VSWAP.WL6 (A versão completa tem 1.500 KiB. Em ambos os casos, uma máquina com 2 MiB de RAM, além do 1 MiB fornecido de fábrica, é suficiente para ter

Todos os recursos carregados durante o pré-carregamento.

Surra: Quando o sistema precisa liberar páginas, mas acaba recarregando o mesmo recurso no mesmo frame, ocorre o "thrashing". O disco rígido é sobrecarregado e a taxa de quadros cai. O thrashing pode acontecer se muitos recursos diferentes estiverem visíveis na tela. Para ajudar os desenvolvedores a equilibrar sua criatividade com a necessidade de uma taxa de quadros decente, o mecanismo detecta o thrashing e pisca a borda da tela em vermelho quando em modo de desenvolvimento.

Os sons são especiais: Como as placas de som são alimentadas por um sistema de interrupções, o gerenciador de som não consegue se recuperar de uma falha de página. Portanto, todos os recursos de som são carregados primeiro (eles estão localizados no início do arquivo). VSWAP.WL1) e apenas na memória convencional.

4.4.3 Gerenciador de Vídeo (VL e VH)

O gerenciador de vídeo possui duas partes. Há um componente de nível superior e um de nível inferior:

- OVL_* A camada é composta tanto de C quanto de ASM, onde as funções em C abstraem a manipulação dos registradores VGA por meio de rotinas em assembly.
- A camada de alto nível (VH_*) É dedicada a desenhos de menus em 2D e, naturalmente, é cliente da camada inferior.

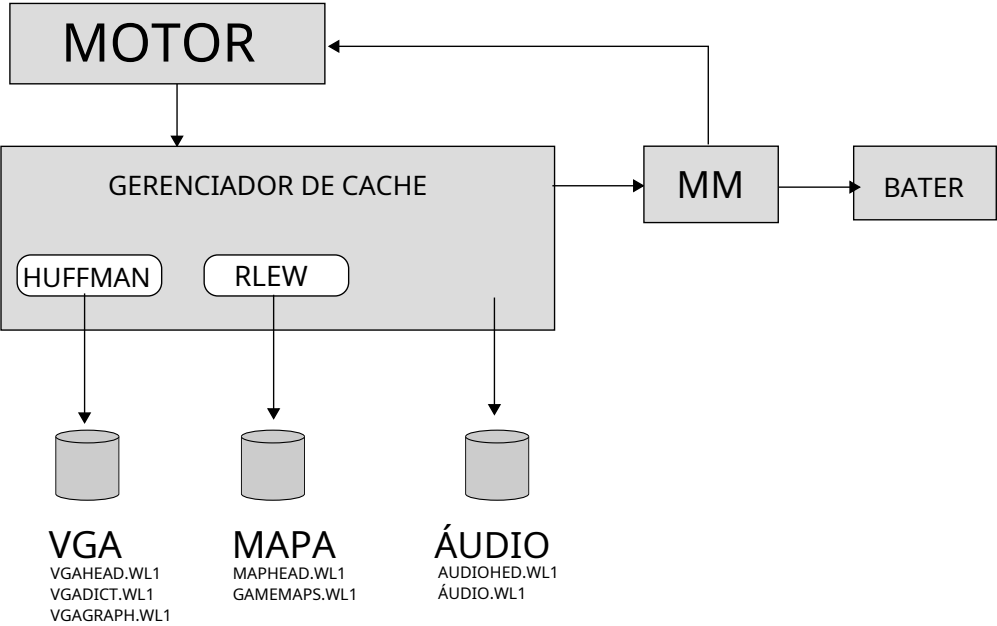
4.4.4 Gerenciador de Cache (CA)

O gerenciador de cache é um componente pequeno, porém essencial. Ele carrega e descompacta mapas, gráficos 2D e recursos de áudio armazenados no sistema de arquivos, disponibilizando-os na RAM. Os recursos de cada tipo são armazenados em dois arquivos. Um arquivo de cabeçalho contém o deslocamento para permitir a tradução do ID do recurso para o deslocamento em bytes no arquivo de dados.

Todos os recursos são comprimidos. No caso de mapas e áudio, a compressão é codificada diretamente no mecanismo. Para gráficos, no entanto, um terceiro arquivo (DICT) contém o dicionário de compressão para descomprimir cada recurso. O Gerenciador de Cache lida com a descompressão de forma transparente, utilizando dois métodos de compressão. Os recursos usam o método Huffman tradicional, mas os mapas são comprimidos duas vezes com uma passagem "carmacizada", que é uma redescoberta independente da abordagem LZ (Lempel-Ziv).⁴.

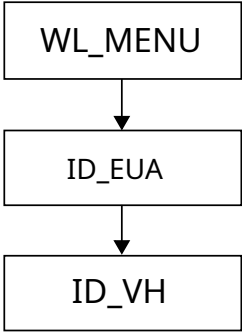
Curiosidades: Existe algum erro de digitação no nome do arquivo? AUDIOHED.WL? A grafia correta seria AUDIOHEAD.WL1. Onde está o UM? Na verdade, essa é uma limitação do sistema operacional. O DOS permite apenas nomes de arquivo de 8,3 caracteres (no máximo oito caracteres seguidos de um ponto e, em seguida, no máximo três caracteres para a extensão).

⁴John Carmack mencionou em diversas ocasiões como era difícil encontrar livros de programação na década de 90. Ele "inventava" algo apenas para descobrir mais tarde que outros já o tinham feito, e melhor do que ele!



4.4.5 Gerenciador de Usuários (NÓS)

O gerenciador de usuários é amplamente baseado no código Catacomb 3-D. e foi escrito por Jason Bloch, muito wiak. O copiar/colar é visível, já que 90% da função declarada no cabeçalho. (ID_US.H) não são, na verdade, gerenciadores mentado em ID_EUA.C. É sa com nomes ruins, como geralmente acontece. y cuida do layout do texto, ut. Quando um WL_* rotina de alto nível n precisa desenhar uma string, é passado para Impressão nos EUA que faz um ele fará a medição (ex: Dr. aw string centralizada) e então passa essa informação para o vídeo. eo Gerente (VW_DrawPropString), que cuida da entrega sobre.



4.4.6 Gerenciador de Som (SD)

O Gerenciador de Som abstrai o Speaker, o Criação com suporte para todos os quatro sistemas de som: PC e AdLib e o Sound Blaster, e não é executado Disney Sound Source. É um sistema complexo, pois é chamado dentro do mecanismo. Em vez disso, via IRQ em uma frequência muito mais alta do que... O motor (o motor funciona a um máximo de 70Hz, enquanto o gerenciador de som varia de 140Hz a 7000Hz) precisa ser executado rapidamente e, portanto, não só é escrito em assembly, como seus recursos também recebem prioridade na alocação de memória. Todos os seus recursos são carregados.

na memória convencional para evitar uma falha de cache no gerenciador de páginas.

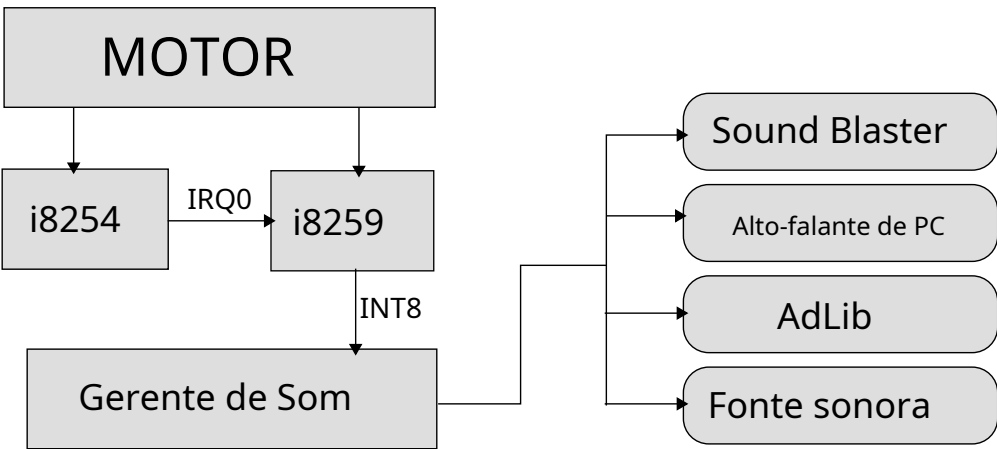


Figura 4.12:Arquitetura de sistema de som.

O gerenciador de som é descrito detalhadamente na seção "Som e Música".

4.4.7 Gerenciador de Entrada (IN)

O gerenciador de entrada abstrai as interações com joystick, teclado e mouse. Ele apresenta o código padrão necessário para lidar com portas PS/2, Serial e DA-15, cada uma utilizando seus próprios endereços de E/S.

4.5 Startup

Assim que o motor do jogo inicia, ele precisa lidar com o as dificuldades descritas no Capítulo 2 "Difícil-hardware. É aqui que as coisas realmente ficam interessantes. interessante.

4.5.1 Login

O primeiro problema (leve) Lidar com isso era lidar com ecossistema heterogêneo de PCs no mercado. placas de Com motorista diferente o hs carregado e diferente som, o motor tem que descobrir como poderá muito A memória RAM está instaladaed e se o lem estava ou não. funcionar. Caso contrário, tem que deixar o usuário saber Qual é o problema? Isso era impo p solução de rtant como a id Software era uma equipe pequena, sem lá fontes para ajudar problemas personalizado problemas er.

É para isso que serve a tela de "login": autodiagnóstico.



Figura 4.13:Tela de login.

Além de exibir dispositivos reconhecidos, como mouse, joystick e placas de som, a métrica mais importante da tela de login é rotulada como "PRINCIPAL". Devido à arquitetura descrita no Capítulo 2 "Hardware", um programa DOS tem apenas 640 KiB de RAM disponíveis. Cada driver carregado pelo usuário consome desses 640 KiB. Se o DOS não conseguir carregar o executável na RAM, o usuário verá a seguinte mensagem de erro:

Sem memória

A tela de login mostra que o Wolf3D precisa de pelo menos 320 KiB de RAM convencional. John Romero escreveu uma nota de lançamento para ajudar as pessoas a entenderem o que estava acontecendo e evitar reclamações. Você pode lê-la no Apêndice, na seção "A Barreira dos 640 KB".

No momento em que a tela de login é exibida, o único gerenciador carregado é o Gerenciador de Memória. Ainda não existe um sistema de arquivos para o mecanismo acessar. É por isso que a paleta de cores e a tela de login são compiladas dentro do executável. Isso é feito para que sejam carregadas na RAM pelo carregador do sistema operacional (DOS). Tudo o que o mecanismo faz é carregar a paleta de cores na VGA.

Copie o bitmap de login da RAM para a VRAM e preencha os blocos do "termômetro" com verde ou amarelo, dependendo do que foi detectado.

```
// Conteúdo do arquivo GAMEPAL.OBJ:
// contabiliza 256 * 3 = 768 bytes. externo byte
longe gamepal;

// Conteúdo de SIGNON.OBJ:
// corresponde a 320 x 200 = 64.000 bytes. caractere
externo introscn distante;

vazio Tela de login (vazio) {

    sem assinatura      início do segmento, comprimento do segmento;

    VL_SetVGAPlaneMode    ();
    Conjunto de Paletas de Teste VL    ();
    VL_SetPalette (& gamepal);

    se(! realidade virtual)
    {
        VW_SetScreen (0x8000 ,0); VL_MungePic (&introscn
        ,320 ,200); VL_MemToScreen (&introscn ,320 ,200 ,0 ,0);
        VW_SetScreen (0,0);

    }

    // recuperar a memória da tela de login vinculada início do segmento =
        FP_SEG (& introscn);
    comprimento do segmento = 64000/16;
    se(FP_OFF (& introscn)){
        segstart ++;
        comprimento do segmento --;
    }
    MML_UseSpace (segstart,seglenth);
}
```

Depois dessa tela, ointroscaA variável (que utiliza 320x200 bytes = 64.000 bytes) é descarregada da RAM para liberar mais espaço para o tempo de execução.

Curiosidades: O corpo da funçãoTela de loginA referência à "realidade virtual" foi adicionada devido a uma empresa que licenciou o motor gráfico para construir máquinas de arcade de realidade virtual. Décadas depois, John Carmack participaria de um renascimento da realidade virtual com o Oculus VR.

4.5.2 Resolvendo o problema VGA

O Capítulo 2 "Hardware" deixou-nos com uma questão por resolver: todos os modos VGA carecem de capacidade de buffer duplo.

O modo mais atraente (13h) oferece um único framebuffer com resolução de 320x200 pixels não quadrados e 256 cores indexadas. O chipset Chain-4 no circuito VGA mapeia automaticamente a RAM a partir de A0000h para os quatro bancos de VRAM. Quando ativos, o desenvolvedor não precisa se preocupar com os bancos. A tela pode ser limpa com uma função simples, como segue.

```
personagem far *VGA = (byte far*)0xA0000000L;

vazio ClearScreen(vazio){
    asm movimento eixo ,0x13
    asm inteiro 0x10

    para(inteiro i=0 ; i < 320*200 ; i++)
        VGA[i] = 0x00;
}
```

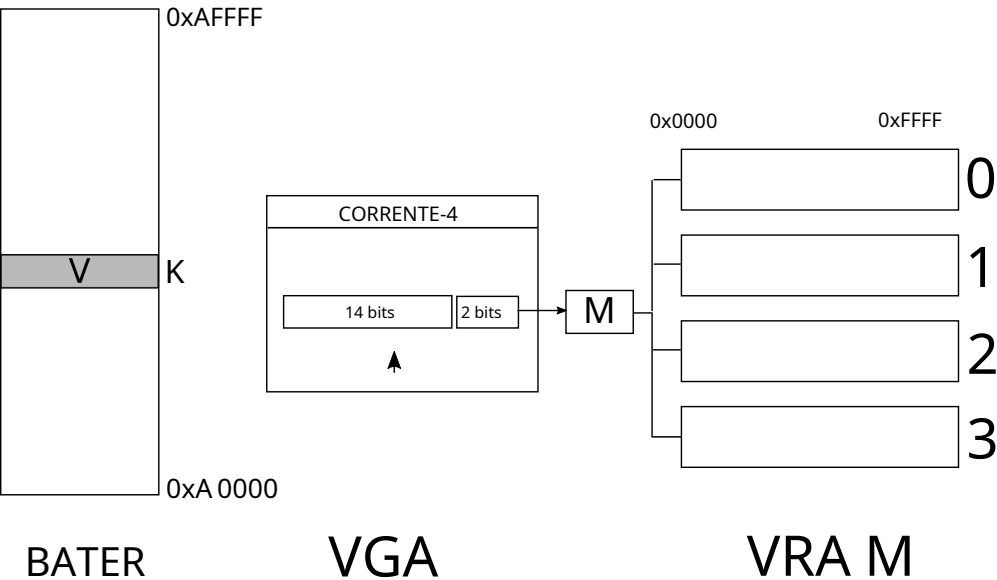


Figura 4.14: Chipset Chain-4 entre RAM e VRAM. A VRAM encaminha as operações de E/S.

Esse sistema de mapeamento também é chamado de "encadeamento". Isso ocorre porque 2 bits fora do anúncio vestidos são usados

Para direcionar uma operação de leitura/escrita para um banco, apenas 14 bits são usados para o deslocamento real no banco. Como 14 bits podem endereçar apenas 16384 valores, esse sistema resulta em 75% da VRAM inutilizável.

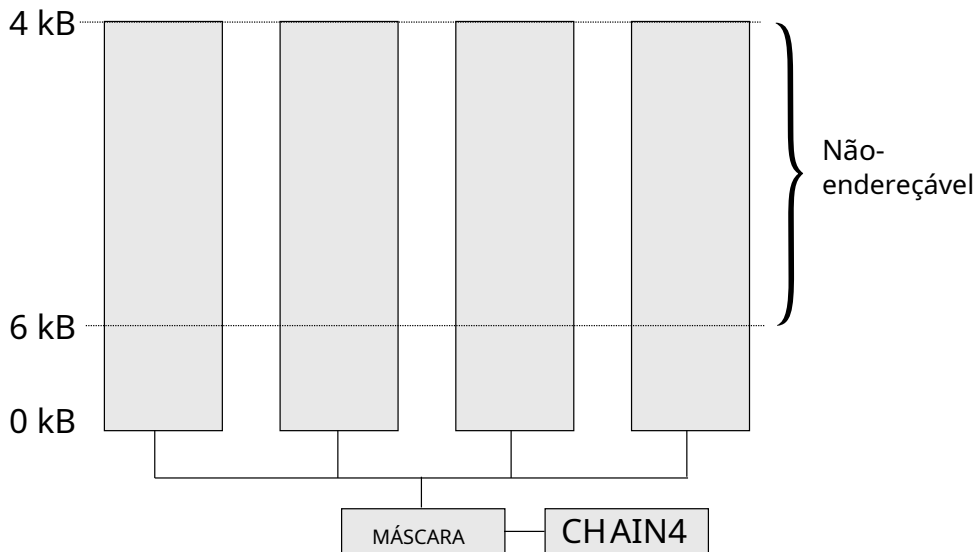


Figura 4.15: Chain-4 falso e um contínuo VRAM bancado, mas desperdiça 75% do VRAM.

O desperdício é realmente a culpa do chip da Cadeia-4. Porém, acontece que é possível para des-
 uma isso. A tecnologia 991. nique wa s popularizado por Michael A bprecipitado. *Dr. Dobb's Jornal* de julho
 1 Em seu artigo operações ele descreve ibed o que ele moeda e Modo d-X. Um não-documento ed sequ ence de
 Odesativando Cadeia g- 4 permite uma resolução Osolução de quadrado 320x240 pixels (si nce o
 ratio é 4:3) e completo acesso t de 256 KiB de R UM.

C olfenstein 3D faz ecoisas ligeiramente diferente. Isso desativa es Chain-4 mas continue ps resol ução em
 320x200. Este mo de era c oModo-Y integrado y ouvido mais tarde ⁵.

T Aqui estão duas razões o t enão usei 32 de-Y 60x240 square pixels. Despi te seu adv formiga
 fou artistas, uma tela usando M o320x200 = com 64.000 pixels, que representam Sentes 17% menos
 pixels comparada a Modo-X 320x240 = 7 6. 800 pixels els. O motor al pronto struggled
 talcançar um aceitar capaz fra mgerar, então Modo-X era sim ply muitos pix luxe ls por fra eu. Isso
 C também poderia ser een inco nveniente para artistas desde De Paint correu no pipelinModo 13h qual
 hum pi não quadrado xels. Isto teria criado e um constrangedor de ward quando re ativos seria
 hforam criados e rasgar ere em diferentes re soluções .

⁵src.games.programmer, 10 de fevereiro de 1992

```

vazio VL_SetVGAPlaneMode (vazio) {
    // Chama o 16º vetor de interrupção com o valor 0x13 // (Solicita
    // à BIOS que configure a VGA no modo 13h) asm
    movimento eixo, 0x13
    asm inteiro 0x10

    // Desacorrentar (chamado de desativação no motor)
    VL_DePlaneVGA ();
    VGAMAPMASK (15);
    VL_DefinirLarguraDaLinha (40);
}

```

A mágica acontece na função VL_DePlaneVGA onde os registradores VGA são manipulados para ajustar o que a BIOS havia configurado no Modo 13h.

```

# definir SC_INDEX      0x03c4
# definir SC_DADOS      0x03c5

# definir CRTC_INDEX    0x03d4
# definir CRTC_DADOS    0x03d5

# definir MODO_DE_MEMÓRIA 0x04
# definir CRTC_UNDERLINE 0x14
# definir MODO_CRTC      0x17

vazio VL_DePlaneVGA () {
    // Alterar a forma como a VRAM é escrita (Desativar a cadeia -4)
    saída(SC_INDEX, MODO_DE_MEMÓRIA);
    saída(SC_DATA, (inp(SC_DATA)&~8));

    // Limpar todos os quatro bancos, já que a BIOS só limpou // os primeiros
    // 16K de cada banco ao configurar o modo 13h. VL_ClearVideo (0);

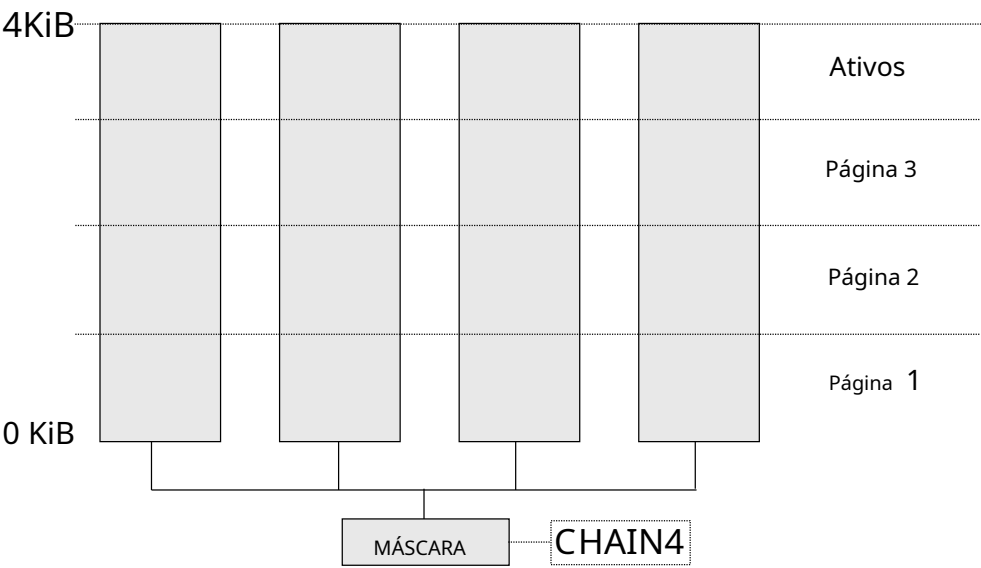
    // Altera a forma como a VRAM é lida pelo CRTC
    // O modo de endereçamento é selecionado através do registrador
    // CRTC_MODE. saída(CRTC_INDEX, UNDERLINE);
    outp(CRTC_DATA, 0x00);
    // Modo de endereçamento CRTC definido como byte.
    saída(CRTC_INDEX, CRTC_MODE);
    outp(CRTC_DATA, 0xa3);
}

```

Os registradores VGA do Controlador de Sequência e do Controlador CRT são configurados para dividir o

256 KiB de VRAM divididos em quatro partes.

- 64.000 bytes para o Framebuffer 0
- 64.000 bytes para o Framebuffer 1
- 64.000 bytes para o Framebuffer 2
- 70.144 bytes para recursos gráficos



Como o motor não foi feito para o modo Y resolve um grande problema, introduzindo dois modos: um referente à velocidade e outro sobre a credibilidade.

Vamos analisar a velocidade. A cadeia-4 de texto da imagem o desenvolvedor é capaz de selecionar o banco de dados para escrever nele. Isso pode ser facilmente feito com uma função simples.

```
#definir SC_M 0x02
#define MASK 0x02

void selecionarPlano(c haravião) {
    saída(SC_IND) EX, S C_MAPMASK);
    saída(SC_DAT A, 1 << plano);
}
```

Nagora o código sample de texto do capítulo de hardware e na página 59 para limpar a tela e adicionar um módulo.

```

vazioTela limpa (inteirocor) {
    para(inteiry=0 ; y < 200 ; y++) {
        para(inteirox=0; x < 320 ; x++) {
            selecionarPlano(x % 4);
            escreverPixel(x, y, cor);
        }
    }
}

```

O código parece inofensivo, mas, por mais simples que seja, não consegue rodar a mais do que alguns quadros por segundo.⁶O problema é que substituímos algo feito em hardware por algo feito em software. saídaAs instruções são simplesmente lentas demais.

A solução é mudar a forma como desenhamos na tela. Em vez de desenharmos primeiro na horizontal, precisamos desenhar primeiro na vertical para minimizar o número de trocas de bancos de caracteres.

```

vazioTela limpa (inteirocor) {
    para(inteirox=0; x < 320 ; x++) {
        selecionarPlano(x % 4);
        para(inteiry=0 ; y < 200 ; y++) {
            escreverPixel(x, y, cor);
        }
    }
}

```

Este código é executado duas vezes mais rápido.⁷já que apenas 640 lentosaídainstruções são usadas.

Essa consideração de velocidade tem um impacto fundamental no motor gráfico: para desenhar qualquer coisa rapidamente com a placa de vídeo, ela precisa ser desenhada verticalmente. Tudo no motor gráfico é desenhado dessa forma: paredes, sprites, menus. As ramificações dessa limitação de hardware são sentidas até mesmo na forma como os recursos são armazenados na RAM: rotacionados em 90 graus e entrelaçados para corresponder ao layout dos bancos de memória da placa de vídeo. Isso é descrito em detalhes na seção 4.7.8 "Desenho de Sprites".

A segunda questão introduzida pelo Modo-Y diz respeito à correção. Com três páginas disponíveis, o mecanismo desenha na página 1, depois na página 2, depois na página 3 e, em seguida, retorna à página 1. Isso resolve o problema de tearing e permite que o mecanismo nunca fique bloqueado durante a sincronização vertical (vsync), já que sempre há um framebuffer válido para desenhar. A troca de páginas é feita instruindo o Controlador CRT a escanear um framebuffer em um deslocamento diferente após a próxima sincronização vertical.

O deslocamento de varredura do controlador CRT é um valor de 16 bits que é atualizado com um pequeno trecho de código assembly.

⁶5 quadros por segundo em um 386DX-40 com placa VGA Cirrus Logic.

⁷10 quadros por segundo em um 386DX-40 com placa VGA Cirrus Logic.

⁸Atingir 70 quadros por segundo é possível usando menos instruções graças a REP STOSWinstrução.

Escrevendo diretamente em um registrador VGA: primeiro o byte mais significativo e depois o byte menos significativo, da seguinte forma.

```
asm mov cx, startScanOffset

asm mov dx, 0x3d4; 3d4h é o registro do CRTC

asm movimento al ; Informe ao CRTC que desejamos atualizar ; o
,0x0c asm fora dx, al ; registro de endereço inicial alto
asm inc dx
asm movimento al, ch
asm fora dx, al ; definir o byte mais significativo
asm dec dx
asm movimento al ; Informe ao CRTC que desejamos atualizar ; o
,0x0d asm fora dx, al ; registrador de endereço inicial baixo
asm inc dx
asm movimento al, cl
asm fora dx, al ; definir o byte menos significativo
```

Este código parece funcionar, mas tem uma falha grave. Se você o executar, de vez em quando a tela esperada mostrada abaixo será exibida...



...em vez disso, apareceria distorcido:



Essa falha mostra tanto desalinhamento quanto partes de duas páginas. Esse problema está relacionado à atomicidade. O endereço inicial do CRTC é um valor de 16 bits, mas oforaA instrução só pode escrever 8 bits por vez. Se as páginas forem configuradas uma após a outra desta forma.

x0000	0x3E80	0x7000	
Página 1	Page 2	Page 3	

Na VRAM, a página 1 está em 0x0000, a página 2 em 0x3E80 e a página 3 em 0x7000. Instruir o CRTC a usar a página 2 em vez da página 1 requer a atualização do byte mais significativo (0x00) para 0x3E e do byte menos significativo (0x00) para 0x80. Como as atualizações não são atômicas, um erro de temporização pode resultar no CRTC selecionando o valor 0x3E00 em vez de 0x3E80.

```
asm      movimento  CX, startScanOffset

asm      movimento  dx,0x3d4      ; 3d4h      é o registro do CRTC

asm      movimento  al,0xc      ; Informe ao CRTC que desejamos atualizar ; o
asm      fora        dx,al      registro de endereço inicial alto
asm      inc         dx
asm      movimento  al,ch
asm      fora        dx,al      ; definir o byte mais significativo

;***** INICIAR LEITURA DO CRTC AQUI      !!!!!      *
;***** E MOSTRA 2 FRAMEBUFFERS PARCIAIS *****

asm      movimento  al,0xd      ; Informe ao CRTC que desejamos atualizar ; o
asm      fora        dx,al      registro de endereço inicial baixo
asm      inc         dx
asm      movimento  al,cl
asm      fora        dx,al      ; definir o byte menos significativo
```

Como atualizar atômicamente um valor de 2 bytes com uma operação de 1 byte? Veja como Wolfenstein configura suas páginas:

```
# definir  TELA INTEIRA      80
...
# definir  TAMANHO DA TELA      (SCREENBWIDE *208)
# definir  PÁGINA 1 INÍCIO      0
# definir  PÁGINA 2 INÍCIO      (TAMANHO DA TELA)
# definir  PÁGINA 3 INÍCIO      (TAMANHO DA TELA *2u)
# definir  INÍCIO GRÁTIS      (TAMANHO DA TELA *3u)
```

Observe como ele usa o valor 208 para a altura do framebuffer. A princípio, isso não faz sentido, já que a tela tem 200 pixels de altura. Pensei que fosse um erro de digitação (afinal,0e8(embora visualmente semelhantes), isso foi feito intencionalmente. O truque aqui é usar um pequeno espaço após cada página para que os endereços difiram apenas pelo valor do byte mais significativo.

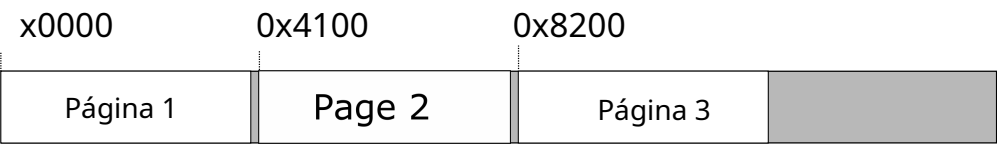


Figura 4.16:Um pequeno espaço entre as páginas faz com que todos os endereços iniciais sejam múltiplos de 256.

Agora, a página 0 está em 0x0000, a página 1 em 0x4100 e a página 2 em 0x8200. A mudança de qualquer página para outra requer a atualização apenas dos 8 bits mais significativos, o que torna a troca de buffer uma operação atômica.

4.5.3 Carnificina Profunda

Após a tela de login, aparece a tela de "classificação". Não havia classificações oficiais para videogames em 1991, desde a criação do ESRB.⁹A classificação indicativa só seria implementada em 1994, em resposta às críticas de violência excessiva (como em Doom) ou conteúdo sexual. Mesmo assim, o jogo ainda exibia um aviso, mais uma ocasião para a id Software demonstrar sua irreverência.

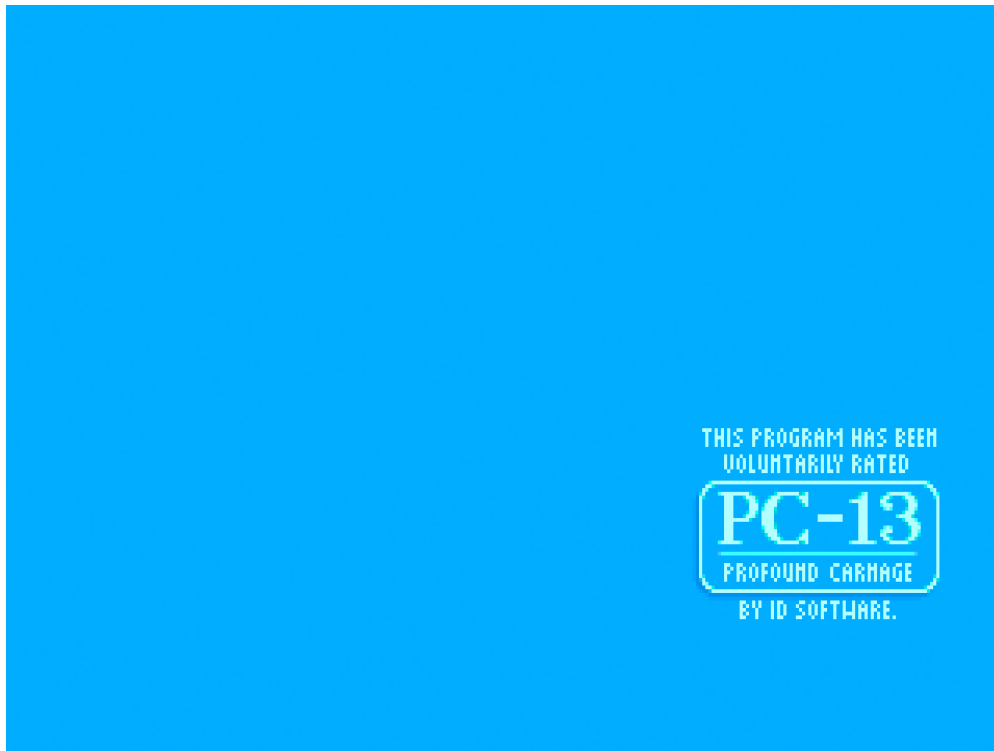


Figura 4.17: Este programa foi classificado voluntariamente como PC "Carnificina Profunda" - 13.

A tela de classificação PC-13 foi, obviamente, inspirada no logotipo da classificação PG-13 do sistema oficial de classificação de filmes da Motion Picture Association of America.

⁹O Entertainment Software Rating Board é a organização responsável por atribuir classificações etárias e de conteúdo.